LLM Memory, Records, and Continued Use

Content, Provenance, Updates, and Authorization

Kentaro Sato
Independent Researcher
info@metaclan.jp

I CONTENT AND PROVENANCE

Role-specific information, shared records, and revocation.

II SCOPE OF UPDATE CONTROL

Field-level comparisons, continued updating, and rollback.

III JUDGMENT AND AGREEMENT

Judgment stability, computational support, and agreement quality.

IV ACTUAL VOTES AND APPROVAL STATES

Three-role votes, individual states, and group agreement.

V MULTI-ROLE FAILURES AND DESIGN PRINCIPLES

Failure analysis, seven principles, and a reference architecture.

Four experimental reports and a concluding design synthesis.

Collected Edition v1.4
3 November 2026 • MRSI-COLLECTED-1.4

Series Summary

From correct answers to traceable records and decisions

A correct answer can carry an incorrect source identifier. A system can faithfully read a stale record or faithfully follow a poor agreement. These four exploratory reports examine what must be evaluated separately when LLM roles retain their own information and judgments while sharing records and coordinating action. Synthetic two- and three-role tasks trace generated responses through actual saved notes or votes, the inputs supplied to later calls, and subsequent use.

Paper 1 — Content and provenance. Correct decisions coexisted with incorrect origin IDs that persisted into later responses. Model-based selection followed by deterministic copying preserved the targeted shared observations, while defects in other fields and subsequent readout prevented an established end-to-end advantage over rewriting. A revocation diagnostic distinguished completing a justified change from appropriately deferring when its target could not be identified.

Paper 2 — Scope of update control. A correct joint answer could conceal stale private observations. Conversely, repairing those observations could leave overall success unchanged because a shared-field defect remained. Comparisons using the same update proposals exposed both relationships. Continued use supported two cycles of observation updates, yet a separate customer-update diagnostic showed old notices undoing legitimate new preferences.

Paper 3 — Judgment and agreement. Supplying computational summaries earlier supported correct intermediate role-specific judgments, even when final-use outcomes were correct under both timing conditions. Correct own judgments and available candidate totals nevertheless coexisted with acceptance of a suboptimal agreement, which was then used faithfully. Fixed-input repetitions further showed that the same combined failure could conceal different defective components.

Paper 4 — Actual votes and approval states. A recorded hold on an appropriate proposal was faithfully respected by subsequent readers. In a matched three-state comparison, one missing vote became an explicit hold in the readout while agreement and execution were correctly deferred. Appropriate voting, fidelity to received votes, and accurate individual approval records therefore require separate checks.

Technical synthesis — From observations to design. Six failure observations motivate seven design principles, a reference architecture, and executable examples. The central task is to preserve whose information or expressed position justifies each next action: bind content to provenance, identify the scope of updates and controls, separate own judgments from joint assignments, and distinguish proposal assessment, actual votes, agreement, and execution authorization.

Scope. The collection retains each study's evaluation units and limitations; counts are not pooled. Small synthetic tasks do not establish general reliability or the causal effect of increasing role count. The synthesis adds no new experiment, and the complete proposed architecture has not been validated in an end-to-end LLM comparison.

Master Contents

PAPER 1 / GLOBAL PDF PAGES 8–33

Maintaining Role-Specific Information and Shared Records in LLM Memory Updates

An Exploratory Evaluation of Content, Provenance, and Continued Use

English v1.0 26 pages

Section	PDF page	Paper page
Title and abstract	8	1
1. Introduction	9	2
2. Related Work	9	2
3. Task and Memory System	10	3
4. Evaluation Methods	12	5
5. Results	15	8
6. Discussion	18	11
7. Limitations and Reproducibility	19	12
8. Conclusion	20	13
9. Ethics, Competing Interests, Author Contributions, and Data Availability	21	14
References	22	15
Appendix A. Scope of the Included Trials	23	16
Appendix B. Evidence and Reproduction Resources	28	21
Appendix C. Implementation and Evaluation Principles	29	22
Appendix D. Interval Estimates and Post Hoc Descriptive Supplements	29	22
Appendix E. Post Hoc Reference Analysis after a One-Brace Correction	33	26

PDF page is the page in this collected edition; Paper page is the page number printed in the standalone report. Click a section or use the PDF bookmarks to navigate. The individual reports retain their own pagination and evidence links.

Master Contents

PAPER 2 / GLOBAL PDF PAGES 34–51

Scope of Update Control in Private and Shared LLM Records

Field-Level Comparisons and Post-Update Rollback

English v1.0 18 pages

Section	PDF page	Paper page
Title and abstract	34	1
1. Introduction	35	2
2. Related Work	35	2
3. Task and System	37	4
4. Evaluation Methods	39	6
5. Results	42	9
6. Discussion	45	12
7. Scope and Limitations	46	13
8. Conclusion	47	14
9. Data Availability and Declarations	47	14
References	49	16
Appendix A. Entry Points and Supplementary Analyses Preserving Formal Outcomes	50	17
Appendix B. Implementation and Aggregation Details	50	17

PDF page is the page in this collected edition; Paper page is the page number printed in the standalone report. Click a section or use the PDF bookmarks to navigate. The individual reports retain their own pagination and evidence links.

Master Contents

PAPER 3 / GLOBAL PDF PAGES 52–66

Separating the Stability of Role-Specific Judgments from the Quality of Joint Agreements

Timing of Computational Support and Fixed-Input Repetitions in a Two-Role LLM Task

English v1.0 15 pages

Section	PDF page	Paper page
Title and abstract	52	1
1. Introduction	53	2
2. Related Work	53	2
3. Task: Separating a Role's Recommendation from Its Joint Assignment	54	3
4. Methods	55	4
5. Results	59	8
6. Discussion	61	10
7. Scope and Limitations	62	11
8. Conclusion	63	12
9. Ethics, Author Contributions, and Data Availability	63	12
References	64	13
Appendix A. Task and Evaluation Scope Established by the Preceding Exploration	65	14
Appendix B. Progression Criteria, Constructed Diagnostics, and Implementation	65	14

PDF page is the page in this collected edition; Paper page is the page number printed in the standalone report. Click a section or use the PDF bookmarks to navigate. The individual reports retain their own pagination and evidence links.

Master Contents

PAPER 4 / GLOBAL PDF PAGES 67–83

Boundaries Between Role-Specific Judgment, Individual Approval, and Group Agreement in Three LLM Roles

Carrying Actual Votes Forward and Comparing Matched Approval States

English v1.0 17 pages

Section	PDF page	Paper page
Title and abstract	67	1
1. Problem Setting and Contributions	68	2
2. Related Work	68	2
3. Task and Evaluated Boundaries	69	3
4. Methods	72	6
5. Results	74	8
6. Discussion	77	11
7. Scope and Limitations	78	12
8. Conclusion	79	13
9. Data Availability and Research Disclosures	79	13
Appendix A. Cases, Replicates, and Response Counts	80	14
Appendix B. Output Contracts and Metric Definitions	80	14
Appendix C. Execution Settings and Reproducibility Scope	81	15
Appendix D. Evidence Entry Points	82	16
References	83	17

PDF page is the page in this collected edition; Paper page is the page number printed in the standalone report. Click a section or use the PDF bookmarks to navigate. The individual reports retain their own pagination and evidence links.

Master Contents

TECHNICAL SYNTHESIS / GLOBAL PDF PAGES 84–113

Failure Analysis and Design Principles for Multi-Role LLM Systems

Boundaries of Records, Approval, and Joint Use in Two- and Three-Role Tasks

English v0.3 30 pages

Section	PDF page	Paper page
Title and abstract	84	1
1. Goal: Preserve Each Role's Judgment While Working Together	85	2
2. What the Roles Were Asked to Do	85	2
3. Units of Failure Analysis: Record Fields and Stage Boundaries	87	4
4. Failure Analysis: What Was Tested and What Each Actor Got Wrong	88	5
5. Seven Design Principles for Connecting Private and Shared Activity	94	11
6. A Reference Architecture Connecting the Principles	99	16
7. Evaluation: Allow Appropriate Progress and Stop Inappropriate Progress	101	18
8. Scope and Further Development of the Design Principles	102	19
9. Conclusion	103	20
10. Author Contributions and Data Availability	103	20
References	104	21
Appendix A. From Failure Observations to Design and Verification	105	22
Appendix B. A One-Page Record for Design Review	105	22
Appendix C. Experimental Records and Organizational References	105	22
Appendix D. Complete Code, Inputs, and Execution Instructions	107	24

PDF page is the page in this collected edition; Paper page is the page number printed in the standalone report. Click a section or use the PDF bookmarks to navigate. The individual reports retain their own pagination and evidence links.

Maintaining Role-Specific Information and Shared Records in LLM Memory Updates

An Exploratory Evaluation of Content, Provenance, and Continued Use

Kentaro Sato
Independent Researcher
Contact: info@metaclan.jp

English version v1.0

Repository and research materials | Apache License 2.0

Abstract

When LLM agents share records over successive tasks, can they preserve information specific to each role and its provenance? We ran the same LLM as two roles in separate contexts on synthetic tasks requiring each role to retain its assigned customer's dog-or-cat preference while combining both roles' observations to choose paper or digital guidance. Saved notes were passed to subsequent calls to evaluate decision correctness, content–provenance correspondence, and revocation: invalidating a previously recorded observation using its provenance.

Exploratory trials identified correct decisions accompanied by incorrect origin IDs that persisted into subsequent responses. We then compared model-based rewriting of shared observations with model-based row selection followed by deterministic copying, using 64 previously unused groups. Selection plus copying preserved all shared observation rows correctly, but both methods achieved end-to-end success in 59/64 groups, with one rewriting group unresolved. Conservative 95% intervals for selection plus copying minus rewriting were calculated for every completion of the unresolved outcome. Their envelope was −17.36 to 16.60 percentage points. End-to-end superiority was not established. The remaining failures under selection plus copying occurred in the generation of role-specific information and in subsequent readout.

A diagnostic experiment varied stored origin IDs and the information in revocation notices. Revocation was completed when the target could be identified; otherwise, the system preserved information and deferred resolution. Tracking each stage from storage to subsequent use distinguished provenance defects accompanying correct answers, local storage success from end-to-end outcomes, and appropriate revocation from appropriate deferral.

Keywords: LLM agents; shared memory; provenance; state updates; revocation; exploratory evaluation

1. Introduction

Agents working together over time need to use shared information while retaining facts specific to their respective roles. When new information arrives, they must update observations while preserving assigned facts for which there is no reason to change. Final-answer correctness alone cannot establish whether these requirements were satisfied together.

For example, role A serves a customer who prefers a dog design, while role B serves one who prefers a cat design. Each role checks whether paper or digital guidance reaches everyone in its assigned area. Choosing a common guidance channel requires both observations, but does not require the customers' preferences to become identical. Moreover, a correct channel choice does not establish that the record still identifies whose observation came from which source.

This study examines **where content–provenance correspondence is preserved or damaged as roles retain their assigned facts and combine observations into a joint decision**. We report an adaptive exploratory series that revised the storage contract, followed by a method comparison and a revocation diagnostic motivated by its observations.

Table 1. Three layers of evaluation and the findings they reveal. Section 3.4 provides a worked example; Section 5 gives denominators and uncertainty.

Evaluation layer	Question	Observation
Decision	Does the model produce a valid answer from saved notes?	Dog/cat and channel decisions could be correct despite provenance defects.
Record	Are persons, values, and revisions correctly associated with their sources?	Successful storage of shared observation rows could coexist with defects in role-specific information or subsequent readout.
Subsequent operation	Can the system use provenance to revoke information, or defer when the target is unidentified?	It revoked identifiable targets and preserved non-target information while deferring under insufficient information.

Linking these layers to actual saved states and subsequent inputs makes it possible to evaluate both record defects behind correct answers and the appropriateness of deferral.

2. Related Work

External-memory architectures include MemGPT's management of context across memory tiers, G-Memory's three-layer graph of collective experience, and MIRIX's coordination of multiple memory types, updates, and retrieval. We focus on content–provenance correspondence when such records are updated, saved, and read as subsequent inputs. MemGPT; G-Memory; MIRIX.

Table 2. Connections to the processes and evaluation targets in related work. The table covers the methods and experiments discussed here, rather than every feature of each system.

Work	Mechanism or evaluation considered	Connection to this study
CoMem	Storage of individual experience, evaluation-based promotion to shared memory, and retrieval from both; evaluated on ALFWorld and PDDL	Separating role-specific assigned facts from shared observations and evaluating their preservation through storage and reuse
MAP-Graph	Provenance graphs, permissions, trust, and action constraints evaluated on synthetic tasks	Tracing the association between generated observations and their origins from storage to subsequent responses

Continued on the next page

Work	Mechanism or evaluation considered	Connection to this study
Collaborative Memory, Sections 3–5	Immutable provenance identifying contributors, resources, and timestamps; dynamic access control; accuracy and resource-use evaluation under changes to sharing and permissions	Separating an observation's origin from its current supporting evidence; revoking observation validity rather than access permission
Governed Shared Memory, Section 7.3	Traversal of derivation chains with recorded parent IDs, evaluating ancestor reachability, writer identity, and depth	Examining the generation of content–provenance associations as well as the storage and retrieval of provenance
AIM, Section 5.2	Exact operation matching alongside a state-aware measure that accounts for conditions such as targets not created upstream	Reporting strict and content-only outcomes together to distinguish provenance defects when content is correct
PROV-DM	A data model distinguishing entities, activities, agents, derivation, and related relations	Representing observation origins and current supporting input documents in separate fields of a custom JSON format
CaMeL	Separation of control and data flow for trusted requests and untrusted data, with capability-based policy enforcement	Assigning separate responsibilities to the model and controller through input/output validation and storage operations
Agentic Unlearning, Section 3	SBU addresses both parameters and persistent memory, combining dependency-based removal or invalidation with parameter updates	Retaining values and origins in external records as history while evaluating invalidation state and subsequent use

In particular, **faithfully preserving recorded provenance and associating stored content with the correct provenance are distinct evaluation targets**. Comparison with the cited Governed Shared Memory experiment clarifies our focus on the connections among generation, storage, and subsequent use. If its category of *provenance collapse* is understood broadly as loss of traceability, the stale-ID associations and generation-time ID errors reported below identify specific locations of damage within that problem area. This mapping is our interpretation of that paper's classification. Governed Shared Memory, Section 4.

3. Task and Memory System

3.1 Assigned facts and joint decisions

Each group comprises two roles, A and B. Each role has an assigned customer who prefers either a dog or a cat design. This preference is a fact to preserve when shared information changes. The exploratory scenarios involved take-home bags at a community craft workshop and reception name tags at an exhibition. The method comparison used new materials generated under the same task rules.

The two roles observe whether paper and digital guidance reach everyone in their respective areas. The areas are disjoint and collectively cover the whole population. An observation is recorded as ALL if a channel reaches everyone and NOT_ALL otherwise. If exactly one channel reaches everyone in both areas, it is selected for the group. If both or neither qualify, the decision is deferred as UNKNOWN. The model must also defer if the input lacks necessary information about the other area.

Both channels can reach everyone in a role's own area even when a unique group-wide choice cannot yet be made. Replacing both observation values with UNKNOWN at storage time loses information needed for a later decision. Appendix A.7 describes a supplementary experiment on this distinction between recording and deciding.

3.2 Records and evaluation terminology

Each role holds a private note and a local copy of the current shared note. The shared note contains both roles' observations. Before exchange, A's copy may contain A's new observation and B's old observation, while B's

copy contains A's old observation and B's new observation.

Table 3. Terms and corresponding record keys. Evaluation criteria are distinguished from units of aggregation.

Term	Record key or designation	Function
Private note	`private_note`	Retains the role's assigned customer, observations, and provenance for that role's subsequent use
Shared note	`shared_note`	Stores A/B observations in each role's copy for exchange with the other role
Origin ID	`origin_id`	Identifies the source of an observation or assigned fact
Declared evidence references	`read_receipt` and related fields	Identifies current input documents supporting the adopted content; saved separately from the notes
Structural/reference validation	Structural/reference validation	Checks syntax, types, and the existence of referenced input documents
Strict evaluation	`strict`	Requires correct content, provenance, and support from the current input
Content-only evaluation	`content`	Retains person, value, revision, and decision requirements while omitting provenance, reference, and input-support requirements

If an observation originally has ID X and arrives through a shared note, its origin ID remains X, whereas its immediate supporting document is the shared note. Declared evidence references are output claims; they do not measure internal attention or the actual reading process.

The evaluation units are observation rows, responses, and groups. In the method comparison, *end-to-end success* requires all four responses in a group—both roles' integration/storage responses and subsequent decision responses—to pass. Strict and content-only outcomes are reported at the same group level.

3.3 Responsibilities of the model and storage controller

Every model call uses a fresh context. The controller constructs subsequent inputs from notes that were actually saved. The model selects evidence and makes decisions; the controller handles delivery, structural/reference validation, storage, and condition-dependent row copying. Fixed-rule Python code scores persons, values, revisions, provenance, and decisions (Appendix C). Errors that remain structurally usable are saved and passed onward. If a structural defect prevents construction of a subsequent input, that dependent model call is not issued.

Under **rewriting (REWRITE)**, the model rewrites the persons, values, revisions, and origin IDs of shared observations. Under **selection plus copying (SELECT)**, it specifies row locations in the input, and the storage controller copies every field of each selected row. There are five candidates: one private observation row, two rows in the role's own shared copy, and two in its partner's shared copy. If the model selects an old row or a row belonging to the wrong person, that row is still saved. Private notes and declared evidence references are generated by the model under both methods.

3.4 A worked example: correct decisions with defective provenance

The **truncated customer origin-ID example (E1-G49/B)** traces the same input through storage and subsequent use. The customer's input record was as follows. JSON excerpts show only the fields needed for the explanation.

```
{"animal":"CAT","origin_id":"RAD9D1FEC60F1FA2","subject":"B"}
```

Rewriting preserved this record. Under selection plus copying, however, the model-generated customer field in the private note lost the final character of the origin ID:

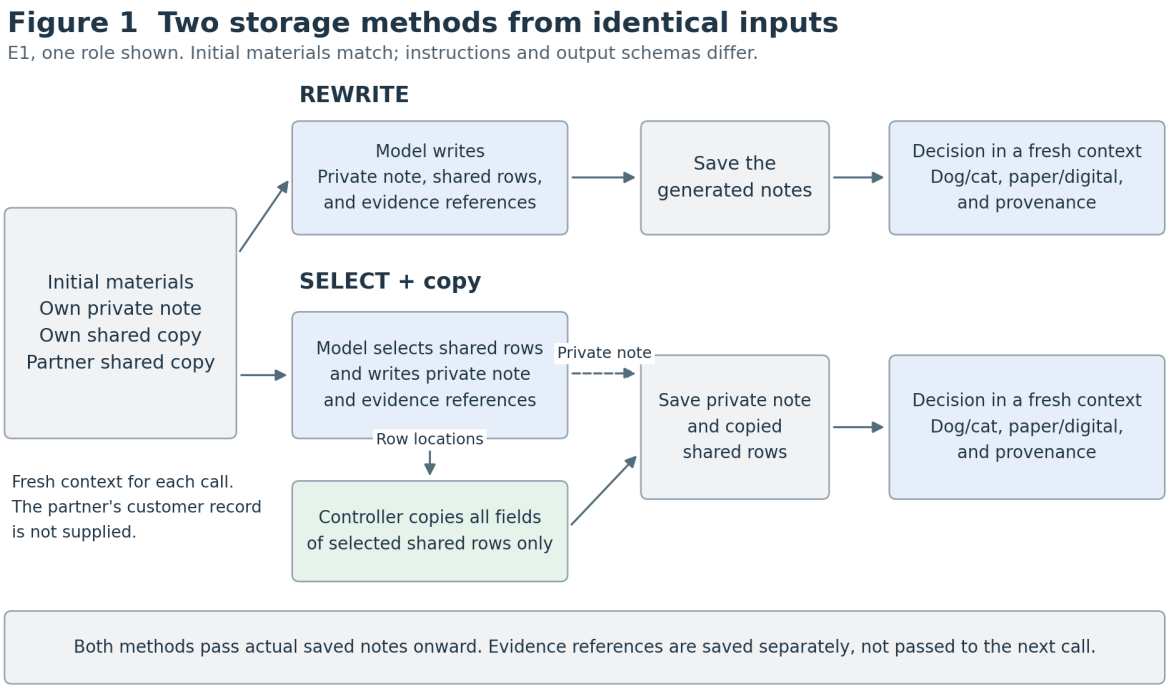


Figure 1. Workflow of the transcription-method comparison (E1). Both methods receive the same initial materials and use their own saved notes for subsequent decisions. One role is shown; the other follows the same procedure. Deterministic copying applies to shared observation rows. The update stage used separately in the exploratory series is shown in Appendix A.


```
{"animal":"CAT","origin_id":"RAD9D1FEC60F1FA","subject":"B"}
```


Table 4. Storage and subsequent decisions from the same input. Both methods saved the shared observation rows correctly.

Stage	Rewriting	Selection plus copying
Input customer record	Cat; origin ID `RAD9D1FEC60F1FA2`	Same
Saved customer record	Cat; origin ID `RAD9D1FEC60F1FA2`	Cat; origin ID `RAD9D1FEC60F1FA`
Subsequent decision	Cat/paper; correct origin ID	Cat/paper; truncated origin ID inherited

The subsequent SELECT response retained the following fields:


```
{"decision":{"mascot":"CAT","channel":"PAPER"},"origin_ids":{"mascot":["RAD9D1FEC60F1FA"]}}
```


The latest observations indicated that paper reached everyone in both areas, while digital guidance did not. Cat/paper was therefore correct. The defect arose in an individual field outside the scope of deterministic copying and passed into the next response alongside a correct decision. Section 5.2 reports outcomes for the complete methods. Supplement S2: worked examples links the excerpts to complete responses and saved states; its explanatory text is in Japanese.

4. Evaluation Methods

4.1 From exploratory trials to two additional experiments

The 11 earlier trials constituted an adaptive exploratory series using four known groups across two scenarios. Observed failures motivated revisions to the storage contract. Table 5 introduces the four studies central to the main text.

Table 5. Studies discussed in the main text and their purposes. Appendix A lists all 11 exploratory trials and their dependencies.

Descriptive name	ID	Purpose
A further update cycle	SM02	Test preservation of assigned facts, observations, and provenance during continued updates to saved notes
Exploratory transcription-method comparison	RC01	Compare rewriting with selection plus copying on the same four known groups
Transcription-method comparison on unused materials	E1	Compare local storage and end-to-end outcomes on 64 new groups
Revocation diagnostic	E2	Evaluate revocation and deferral under different origin IDs and notice information in eight separate cases

An observation of a stale origin ID attached to correctly updated content motivated the exploratory transcription comparison. Appendix A documents the materials and rules fixed before each run, execution order, dependencies, and all outcomes.

RC01's pre-run prediction was selection plus copying minus rewriting ≥.25: an engineering prediction of at least one additional successful group among four known groups. After observing 3/4 versus 4/4 under a rule prohibiting further collection based on the sign of the difference, we designed E1 with unused materials and E2 with separate materials. Both plans were fixed before execution. RC01 pre-run plan.

4.2 E1: a paired comparison on 64 unused groups

E1 evaluated both rewriting and selection plus copying on each of 64 newly generated groups. The material-generation seed was 2026092101. An 8-by-8 cross of scenario names and venue names varied customers, current and old observation values and revisions, and presentation order. We checked equivalence under A/B, paper/digital, and dog/cat reversals after excluding names and IDs, and excluded overlap with earlier materials and E2. Origin IDs consisted of `R` followed by 15 uppercase hexadecimal characters, without prefixes encoding role or revision.

Each group required eight responses: two methods × two roles × integration/storage and subsequent decision. There were 512 responses in total. Initial materials for the two methods were byte-identical. Each role's subsequent decision input was built from the notes it had actually integrated and saved. Deterministic copying in SELECT applied to shared observation rows; the model generated private notes, declared evidence references, and subsequent decisions under both methods. The allocation seed was 2026092102. Execution was divided into four blocks of 16 groups. Within each block, each method went first for eight groups, with the order reversed at the subsequent decision stage. Calls ran serially in fresh contexts.

The primary outcome was the proportion of groups in which all four integration/storage and subsequent decision responses, across both roles, satisfied requirements for persons, observation values, revisions, origin IDs, current references, input support, and subsequent decisions. We defined Δ as selection plus copying minus rewriting. The conjunction of all requirements was chosen to measure how often assigned information, shared records, and subsequent decisions succeeded together. If correct initial materials led to an incorrect saved record that was then faithfully read, the group still failed the end-to-end criterion.

To distinguish local storage performance, stage-specific components and content-only evaluation were specified as secondary outcomes before execution. Content-only evaluation retained person, value, revision, and decision requirements while omitting provenance and reference requirements. Aggregating the four shared rows at group level, breaking results down by scenario and venue, and describing output length were post hoc supplements (Appendix D). No additional significance tests were performed for secondary outcomes.

The pre-run directional prediction was $\Delta>0$. We fixed in advance a two-sided exact McNemar test for complete data and the following interval procedure. We constructed 97.5% Clopper–Pearson intervals separately for the proportions of SELECT-only and REWRITE-only successes, then combined their lower and upper bounds to obtain a conservative 95% interval for the difference using Bonferroni coverage. With missing outcomes, we considered all completions consistent with known results. The minimum and maximum Δ give the missing-outcome bounds, while the envelope of the corresponding 95% intervals describes inferential uncertainty. Under missingness, the primary p-value is not calculated and the denominator remains 64 groups. Inference uses independent, identically distributed groups as a working assumption. Pre-run design.

The pre-run power calculation assumed N=64, no missingness, independent and identically distributed groups, and a two-sided exact McNemar test at α=.05. Power was 82.7% for a true difference of 20 percentage points with a discordance rate of .30, 66.8% for the same difference with discordance .40, and 23.6% for a difference of 10 points with discordance .30. The 20-point scenario represented a compromise between cost and detectable effect size and informed the fixed N. Pre-run budget and power estimates (see the evidence index).

4.3 E2: origin IDs crossed with notice information

E2 used eight cases that did not overlap with E1. It manipulated provenance in otherwise correct private and shared notes prepared by the study controller. The target observer was A in four cases and B in four. Across all copies of the target observation, its origin ID was set to the correct ID, an old source-record ID, or an ID absent from the source-record history. Persons, values, and revisions were held constant. Incorrect IDs did not collide with other valid rows or customer information. OLD and ABSENT were historical categories available to the analysis; the model received the current notes and notice.

Revocation notices came from a trusted issuer. An ID-only notice named the target origin ID; a detailed notice added the observer, revision, and paper/digital values to the same ID. The fixed contract prioritized an ID match. If that failed, the details had to uniquely identify an observation matching the observer, revision, and both values. All copies of an identified observation were invalidated while retaining their original values, revisions, and origin IDs as history. If the target could not be identified, all rows were preserved and the notice was recorded as unresolved.

The saved notes and revocation record were passed to the subsequent decision stage. A resolved revocation required an UNKNOWN/REVOKED channel outcome; an unresolved notice required UNKNOWN/PENDING. Evaluation also covered preservation of the customer's preference and provenance, active observations, the set of revoked observers, and unresolved notices.

The design comprised eight cases × three origin-ID conditions × two notice conditions × two roles × two stages, totaling 192 responses. Each condition retained a denominator of eight cases. Across both roles and stages, we separately counted (1) supported target resolution and completion, and (2) evidence-appropriate handling. Completion in all 8/8 correct-ID cases under each notice type was a gate for interpreting the diagnostic, and all six conditions were reported. No p-values were assigned to this small, fixed diagnostic.

Inability to complete revocation with an incorrect ID and an ID-only notice follows from the materials and matching contract. The empirical question in these conditions was whether the system would avoid guessing a target, preserve assigned information and non-target observations, and carry the unresolved state forward. Detailed notices used uniquely matching materials. Post hoc intervals were nominal 95% Clopper–Pearson intervals treating the eight cases in each cell as independent, identically distributed binomial trials; they were not simultaneous intervals for all six conditions.

4.4 Missingness, continuation after a stop, and verification

Each component was classified as success, failure, or unresolved. When aggregating a group's end-to-end outcome, a known failure took precedence over unresolved components. Missing-outcome bounds with fixed denominators were distinguished from inferential confidence intervals.

After one local validation stop, the original no-resumption rule was changed and the change documented before the remaining 63 responses were obtained. The original stop record was preserved, and the continuation was aggregated separately (Appendix A.8).

E1's malformed JSON response remained unresolved in the formal analysis. A post hoc analysis removing one extra brace from a derived copy is reported separately in Appendix E. Separate checking code verified raw responses, saved states, subsequent inputs, and hashes, and regenerated the main tables from saved responses (Appendix B).

5. Results

We report group or case counts first, followed by differences in proportions and their uncertainty. Table 6 distinguishes the interval types; Appendix D gives their calculation components.

Table 6. Intervals used in this section.

Interval	Target and interpretation
Main-comparison interval	E1's conservative 95% interval for the method difference, calculated using the procedure fixed before execution and taking the envelope over all completions of unresolved outcomes; the same procedure applies to content-only evaluation
Supplementary component interval	A nominal 95% interval added post hoc after aggregating shared observation rows at group level
Revocation-diagnostic interval	A nominal 95% binomial interval added post hoc for the eight cases in each E2 condition; not simultaneous across the six conditions

5.1 Exploration: provenance defects alongside correct decisions and in later responses

In the further-update exploration (SM02), end-to-end success was 4/4 groups under content-only evaluation and 3/4 under strict evaluation. In the **stale origin-ID example (G03/B)**, B correctly saved partner A's new values and revision 3 but retained the old origin ID `SM3A` instead of the new `SM2G3A`. Both subsequent decision responses correctly selected dog/digital and also inherited the old ID. Exploratory results.

The exploratory transcription comparison (RC01) generated fresh responses under both methods from those pre-integration materials. Strict end-to-end success was 3/4 for rewriting and 4/4 for selection plus copying; both achieved 4/4 under content-only evaluation. In the **origin-ID-absent-from-input example (REWRITE G03/B)**, the model associated correct values, person, and revision with `SM3G3A` and carried that ID into a correct dog/digital decision. SELECT chose the row with the correct `SM2G3A` and copied all its fields. Exploratory comparison results.

5.2 E1: successful copying targets and failures in surrounding stages

Both methods achieved end-to-end success in 59/64 groups. Depending on the unresolved REWRITE group's outcome, the success counts would be equal or REWRITE would have one additional success. The main-comparison interval was −17.36 to 16.60 percentage points. End-to-end superiority of selection plus copying was not established.

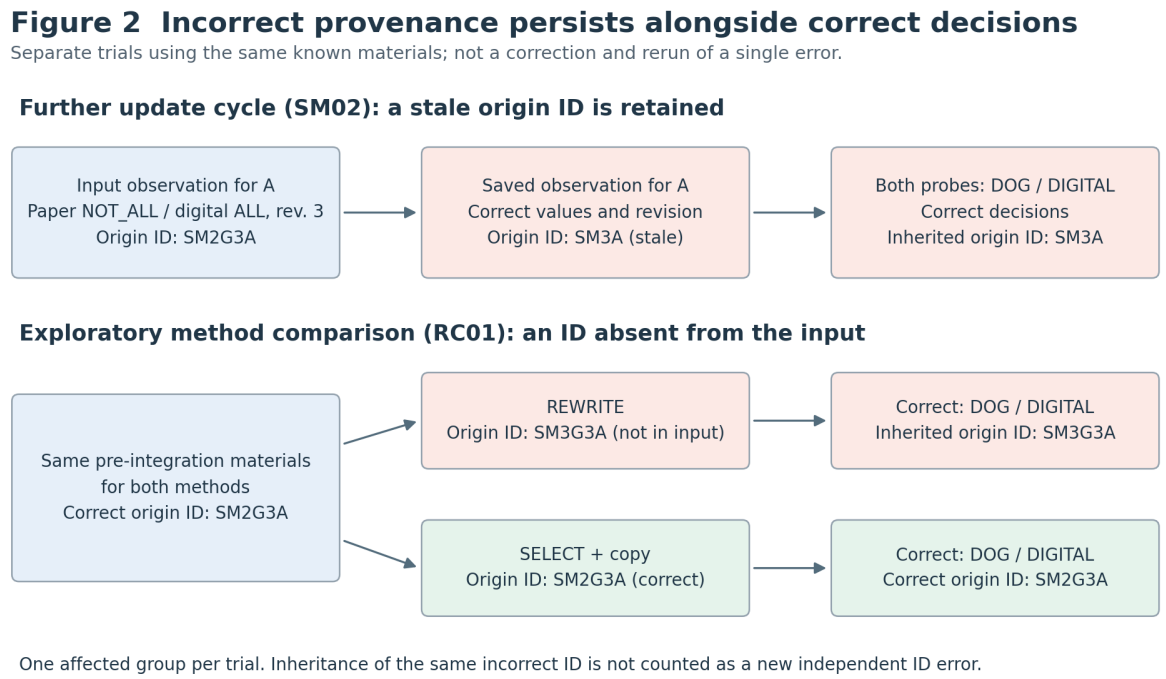


Figure 2. Provenance traces from two trials sharing known materials. The further-update trial retained a stale origin ID; the exploratory transcription comparison generated an ID absent from the input. Both defects coexisted with correct content and subsequent decisions. These were distinct errors in separate trials, and the method difference was localized to one group.

At the stage level, all shared observation rows under SELECT passed strict evaluation. The defects in its five unsuccessful groups occurred in the generation of role-specific information or in subsequent readout. Table 7 places the copying target alongside the surrounding stages.

Table 7. Stage-specific results under strict evaluation. Rows, responses, and groups aggregate the same materials at different levels.

Stage and evaluation target	Rewriting	Selection plus copying
Integration/storage: shared observation rows	253/256 rows	256/256 rows
Integration/storage: assigned customer in private note	128/128 records	127/128 records
Integration/storage: own observation in private note	128/128 records	128/128 records
Subsequent decision: complete response	124 successes, 3 failures, 1 unresolved / 128 responses	123 successes, 5 failures / 128 responses
End-to-end: both roles and both stages	59 successes, 4 failures, 1 unresolved / 64 groups	59 successes, 5 failures / 64 groups

Evaluation of shared rows included content, provenance, and support from the current input through declared evidence references. For example, one REWRITE group (G11) failed a reference requirement. These results therefore measure more than exact copying of selected rows. Aggregating the four shared rows across both roles gave 61/64 groups for REWRITE and 64/64 for SELECT: a difference of +4.69 percentage points, with a supplementary component interval of −5.87 to 14.44 points (Appendix D.2).

SELECT defects included an incorrectly saved and subsequently inherited customer origin ID (G49 in Section 3.4) and a truncated origin ID during readout of a correctly saved observation (G16/A). G03, G16, and G49 passed content-only evaluation, whereas G27 and G45 also failed the channel-decision requirement. The five groups and six responses are detailed in the supplementary component audit.

We also observed incorrect saved content affecting a later decision. One REWRITE role (G36/A) changed

partner B's paper-coverage value from ALL to NOT_ALL during integration. Its subsequent decision deferred in accordance with that incorrect saved record. This distinguished inheritance of a storage defect from a defect newly introduced during readout. Raw-response and component inspection (see the evidence index).

Table 8. Formal primary results and the content-only evaluation specified before execution, for the fixed 64 groups. Differences are SELECT minus REWRITE. Missing-outcome bounds describe completions of unresolved results within these 64 groups; they are distinct from inferential confidence intervals.

End-to-end criterion	Rewriting: success / failure / unresolved	Selection plus copying: success / failure / unresolved	Missing-outcome bounds for the difference	Main-comparison interval
Strict, primary	59 / 4 / 1	59 / 5 / 0	−1.56 to 0.00 percentage points	−17.36 to 16.60 percentage points
Content-only	60 / 3 / 1	62 / 2 / 0	+1.56 to +3.13 percentage points	−11.35 to 16.38 percentage points

Both methods succeeded in 54 groups; only REWRITE succeeded in five; only SELECT succeeded in four; and one group had an unresolved REWRITE outcome with a successful SELECT outcome. All 512 planned responses were obtained, but one final REWRITE response (G23/B) contained an extra `}`, leaving 511 structurally evaluable responses. Under the missingness rule fixed before execution, **the formal primary p-value was not calculated**. Formal aggregate results.

Appendix D reports interval components for each completion and breakdowns by scenario and venue. The post hoc one-brace correction is reported separately in Appendix E.

5.3 E2: completion and appropriate deferral

The revocation diagnostic produced 192 saved responses and 96 sets of post-revocation notes. Revocation was completed in every case under conditions that identified the target, and resolution was deferred in every case under conditions that did not. The requirements to preserve assigned customer information and non-target active observations and to carry the processing state forward were also satisfied.

Table 9. Completion and deferral by stored provenance and notice information. Every cell uses the same eight cases and combines both roles and both stages.

Stored origin ID	Revocation notice	Completion	Evidence-appropriate handling	Subsequent state
Correct ID	ID only	8/8	8/8	Revoked
Correct ID	With details	8/8	8/8	Revoked
Old ID	ID only	0/8	8/8	Pending
Old ID	With details	8/8	8/8	Revoked
ID absent from history	ID only	0/8	8/8	Pending
ID absent from history	With details	8/8	8/8	Revoked

The correct-ID baseline passed with both notice types. Even with an incorrect stored ID, uniquely matching details led to invalidation of all copies of the target observation. With ID-only notices, the model preserved records instead of guessing a target to revoke. Evidence-appropriate handling was observed in 192/192 responses and 8/8 cases within each condition. Continuation and combined results.

For example, the subsequent response to an ID-only notice given notes containing an ID absent from history (G01/A) included:

```
{"channel":"UNKNOWN","channel_status":"PENDING","revoked_observers":[],"pending_notice_ids":["R07D126
↪ D1015A6A8"]}
```


The channel decision was deferred, no observer was revoked, and the unresolved notice ID was retained. The complete response also preserved the customer and active observations. It was scored as incomplete but evidence-appropriate (Supplement S2).

The channel decision was UNKNOWN in every condition. Evaluating target identification, invalidation state, and unresolved notices separately distinguished completion from appropriate deferral. The revocation-diagnostic intervals were 63.06–100% for 8/8 and 0–36.94% for 0/8. The 0/8 completion result with an incorrect ID and an ID-only notice follows from the information and matching contract specified in Section 4.3.

6. Discussion

6.1 Evaluate content, provenance, and subsequent operations separately

The two exploratory examples and the customer origin-ID example in Section 3.4 showed that correct decisions can coexist with incorrect origin IDs, and that those defects can pass from saved states to subsequent responses. Linking raw responses, actual saved states, and subsequent inputs makes content correctness, content–provenance correspondence, and inheritance separately observable.

E2 examined a subsequent operation that uses provenance through a fixed diagnostic. It distinguished matching by a correct ID, matching by unique details, and deferral when the target remained unidentified, while checking preservation of assigned information and non-target observations. This separated whether sufficient information for completion was available from whether the operation was appropriate to the supplied evidence.

6.2 Inspect copying targets together with surrounding stages

Selection plus copying assigns row selection to the model and faithful copying of the selected row to the controller. RC01's 16/16 correct selections and 16/16 exact copies, and E1's 256/256 successful shared observation rows, are outcomes observed under this division of responsibility. In E1, defects in all five unsuccessful SELECT groups occurred outside the copying target—in individual fields or subsequent decisions. The evaluation thus identified both successful shared-row storage and remaining end-to-end failures.

This suggests three targets for design evaluation: the scope of copied information, generation outside that scope, and decisions that read the saved states. The small difference favoring SELECT under content-only evaluation did not persist under the primary criterion incorporating provenance and references, further motivating separate evaluation of these components. Because both intervals include zero, this breakdown supplies hypotheses for subsequent comparisons.

E1's upper bound, obtained using the interval procedure fixed before execution, was 16.60 percentage points, below the 25-point difference observed in the four earlier groups. Under the working model of independent, identically distributed groups, the results do not support extrapolating an average improvement of that magnitude to E1's materials. E1's own pre-run prediction was $\Delta > 0$; its comparison with the results is reported in Section 5.2.

6.3 Separate origins from current evidence to locate defects

Keeping origin IDs separate from declared evidence references helped identify which relationship failed. An exploratory case carrying forward old reference names mixed historical references with current direct references (Appendix A.7). The stale and input-absent origin IDs in Section 5.1 instead broke the association between saved

content and its origin. E1 separately recorded customer origin-ID errors introduced at storage, errors introduced during readout of correctly saved observations, and missing or excessive evidence references.

This distinction clarifies whether to inspect the reference-field contract or the transcription of observation rows. A future isolated comparison could distinguish a reference contract requiring only support for adopted rows from one also requiring documents consulted for comparison.

6.4 Evaluate role-specific information and shared state through paired retention and update requirements

The task required A and B to retain their respective customer facts while selecting a common guidance channel from shared observations. Defining in advance which role-specific differences should remain and which decisions should agree after sharing made their coexistence evaluable at each stage.

E2 also tested preservation of assigned facts and active observations for which no change was warranted while a target observation was invalidated. Pairing retention and update requirements provides a basis for extending evaluation to inconsistent notices or repeated runs under the same contract for assigned facts and observations.

7. Limitations and Reproducibility

Sample and statistical inference

The 11 earlier trials were adaptive exploration on four known groups in two scenarios. E1's 64 groups and E2's eight cases were fixed before execution, but were generated by shared rules and evaluated with one model. They were not random samples from multiple domains. Groups and cases are the inferential units; roles, rows, stages, or aggregation with earlier trials do not increase N. E1's intervals assume independent, identically distributed groups. Their conservatism does not necessarily accommodate dependence due to common templates or labels. Resolving small differences or establishing equivalence was outside the design's scope.

Run-to-run variability

Each planned response in E1 and E2 was measured once. Observed discordance can include run-to-run variability, but this design cannot separately estimate within-material, within-method variation and heterogeneity in method differences across materials. A retry after a stop and repeated final decisions in parts of the exploratory series do not substitute for planned replication. Scenario- and venue-level breakdowns are descriptive (Appendix D).

Operationalization and external validity

The measured information consisted of assigned customer preferences and guidance-coverage values expressed in a fixed vocabulary, rather than spontaneously formed personalities or values. We evaluated short trajectories using explicit revisions, self/other distinctions, controller-managed delivery, and external notes. Extension to practical settings requires evaluation with ambiguous descriptions, contradictions, multiple sources, larger histories, decision costs, more roles, and longer operation.

Conditions of the revocation diagnostic

Notices were trusted, incorrect IDs did not collide with other rows, and details uniquely identified targets. Ambiguous details, details with no matching target, contradictory details, autonomous recovery, provenance authentication, adversarial inputs, and defenses against information leakage were not evaluated.

Nor were naturally occurring defects from the method comparison fed directly into E2. The Section 3.4 example concerned customer provenance, whereas E2 targeted guidance-coverage observations. Combining the method-

comparison error rate with the diagnostic completion rate therefore cannot estimate operational loss. Intervals for conditions made unresolvable by design are not estimates of an unknown ability to resolve targets.

Intervention and mechanism

The method comparison evaluated complete methods, including instructions, output formats, output amounts, and storage operations. Appendix D.7 reports output lengths, but the separate effects of these factors remain unidentified. Finding SELECT's remaining defects outside the copying target does not establish that copying causally moved failures to another stage.

The exploratory additions of shared information and changes to reference fields also changed multiple factors. They did not isolate the effects of shared formation, partner editing, or separation of declared references (Appendices A.4 and A.7). Effects of ID format and the internal mechanisms of ID errors were not measured (Appendix A.9).

Execution environment and audit

The model configuration was `gpt-5.6-luna`, reasoning `medium`, no tools, fresh contexts, and serial execution through an authorized ChatGPT account route. This is the identifier used at submission, not a published set of fixed model weights. Generalization to other models and third-party regeneration of identical outputs have not been tested. E2 included a documented rule change and continuation after interruption (Appendix A.8). Development, execution, fixed-code scoring, and editorial auditing took place within the same project; independent third-party blinded rescoring was not performed.

Reproduction resources and scope of comparison

A package regenerates the main tables from saved responses (Appendix B). This computational reproducibility is distinct from stability under rerunning the model. We compared methods and evaluation targets in related work; we did not perform direct performance comparisons, an exhaustive priority survey, or PROV-conformance testing. Publication status and declarations are given in Section 9.

8. Conclusion

This study **traced answer correctness, content–provenance correspondence, and subsequent operations using that correspondence from storage to the next input**. It identified correct content and decisions accompanied by incorrect origin IDs, with defects carried into subsequent responses.

In E1's comparison on unused materials, SELECT achieved strict success on 256/256 shared observation rows. Defects in its five unsuccessful end-to-end groups occurred in the generation of role-specific information or subsequent readout. Although an end-to-end method advantage was not established, the evaluation identified successful local storage alongside remaining challenges in surrounding stages. Under E2's fixed information conditions and matching contract, it distinguished completed revocation from appropriate deferral due to insufficient evidence, while evaluating preservation and subsequent use of assigned facts and non-target observations.

The coexistence of role-specific information and shared state can be examined by separately specifying facts to retain for each role, information to update jointly, its provenance, and subsequent operations. This report provides a single-model, synthetic-task case study that carries out this evaluation on actual saved states.

9. Ethics, Competing Interests, Author Contributions, and Data Availability

The materials comprise fictional customers and synthetic observations. Model outputs are experimental materials; no real customer records or human-participant responses were used. The author has yet to confirm the determination of whether ethics review was required and any review history.

LLM assistance was used in design, implementation, execution procedures, scoring rules and code, and writing. Responses from the model under study were scored by fixed Python code (Appendix C.2). Interpretation of examples and LLM reader feedback were separate from the process that produced formal outcome labels. Detailed author contributions, funding, and competing-interest declarations will be finalized by the author before public release.

Data and code are distributed in the accompanying MRSI repository under the approved retention policy with no deletion deadline (Appendix B). The repository provides distribution scope, Apache 2.0 license information, and current reproduction instructions. A permanent archive DOI for this paper has not been assigned. Authentication credentials, account identifiers, and raw communication headers are excluded from the distributed materials.

References

The versions below were checked on September 20–22, 2026. arXiv versions are distinguished from the W3C standard document. Conference and journal acceptance status was not comprehensively verified.

1. Chengxin Yu et al. (2026). CoMem: Collective-Individual Memory Synergy for Evolutionary Multi-Agent Systems. arXiv:2609.15009v1.
2. Yiqi Wang et al. (2026). MAP-Graph: Provenance-Aware Shared Memory for Multi-Agent Workflows. arXiv:2608.10509v1.
3. Yanki Margalit, Nurit Cohen-Inger, Erni Avram, Ran Taig, and Oded Margalit (2026). Governed Shared Memory for Multi-Agent LLM Systems. arXiv:2606.24535v1.
4. Zachary Johnson et al. (2026). AIM: A Privacy-Aware Interoperable Memory Framework for Multi-Agent Multi-User LLM Systems. arXiv:2609.12320v1.
5. W3C (2013). PROV-DM: The PROV Data Model. W3C Recommendation, 30 April 2013.
6. Alireza Rezazadeh et al. (2025). Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control. arXiv:2505.18279v1.
7. Guibin Zhang et al. (2025). G-Memory: Tracing Hierarchical Memory for Multi-Agent Systems. arXiv:2506.07398v2.
8. Yu Wang and Xi Chen (2025). MIRIX: Multi-Agent Memory System for LLM-Based Agents. arXiv:2507.07957v1.
9. Charles Packer et al. (2024; initial version 2023). MemGPT: Towards LLMs as Operating Systems. arXiv:2310.08560v2.
10. Edoardo Debenedetti et al. (2025). Defeating Prompt Injections by Design. arXiv:2503.18813v1.
11. Bin Wang et al. (2026). Agentic Unlearning: When LLM Agent Meets Machine Unlearning. arXiv:2602.17692v2.

Appendix A. Scope of the Included Trials

A.1 Complete trial inventory

The first table lists the 11 earlier exploratory trials; the second lists the two additional experiments. These 13 studies constitute the report. Planned/executed counts refer to logical response slots and actual research API calls, not independent samples. Structural missingness and dependent calls not issued are listed separately. Square brackets denote missing-outcome bounds. MET and NOT_MET compare the fixed sample with the pre-run prediction; they do not indicate suitability for publication or acceptance of a method.

ID and full name	Planned / executed	Structural missing / dependent calls not issued	Saved primary result	Sensitivity or supplementary result	Original designation
SC01 SHARED-CARRY-01	64 / 64	0 / 0	E_BOTH=4/8 group–replicate combinations; formation 2/4	E_VALUE=4/8; final BOTH decisions 16/16	NOT_MET
SU01 SHARED-UPDATE-01	72 / 67	1 / 5	C_PAIR=0/4 groups	Content C_PAIR=[0,.25]	NOT_MET
RD01 RECORD-DECISION-01	32 / 32	0 / 0	ALL/ALL preservation: with decision rules 1/8; recording only 8/8	Difference +.875; four source records × two repetitions per condition	LOCAL_POSITIVE
SC02 SHARED-CARRY-02	48 / 12	4 / 36	E_BOTH=[0,1]; formation F=[0,1]	E_NOREF=[0,1]	INCONCLUSIVE
SC03 SHARED-CARRY-03	48 / 48	0 / 0	E_BOTH=8/8 group–replicate combinations; formation 4/4	E_NOREF=8/8	MET
SU02 SHARED-UPDATE-02	48 / 48	0 / 0	C_PAIR=3/4 groups	Content-only 3/4	NOT_MET
SR01 SHARED-RECENCY-01	32 / 32	0 / 0	E_PAIR=0/4 groups across both orders	Content-only 4/4	NOT_MET
SM01 SHARED-MERGE-01	32 / 32	0 / 0	M_PAIR=4/4 groups	Content-only 4/4	MET
SM02 SHARED-MERGE-02	32 / 32	0 / 0	M_PAIR_2=3/4 groups	Content-only 4/4	MET
RC01 ROW-CARRY-01	32 / 32	0 / 0	M_END: REWRITE 3/4; SELECT 4/4 groups	Content-only 4/4 for both	MET
RC02 ROW-CARRY-02	24 / 24	0 / 0	M_CONT=4/4 groups	Content-only 4/4	MET

SU02 predicted success in all four groups, whereas SM02 predicted at least 3/4; the same 3/4 result therefore received different original designations. We do not reclassify them under a common pass/fail threshold. SC02 could not be evaluated beyond its structural missingness. SC01's E_VALUE is not equivalent to the content-only criterion used later.

The collection and analysis scope of the additional experiments is shown below. E2's continuation and E1's post hoc sensitivity analysis do not add independent samples.

Experiment	Planned responses / actual API calls	Responses, structure, and stops	Formal result
E1 ROW-CARRY-CONFIRM-01	512 / 512	512 responses; 511 structurally valid; one malformed JSON response	64 groups; REWRITE 59 successes, 4 failures, 1 unresolved; SELECT 59 successes, 5 failures; primary p-value not calculated
E2 PROVENANCE-REVOKE-01	192 / 193	129 initial responses plus 63 after authorized continuation; 192 structurally valid; one stopped attempt retained	Eight cases × six conditions; evidence-appropriate handling 8/8 in each; completion 0/8 for each incorrect-ID + ID_ONLY condition and 8/8 otherwise

Together, the experiments involved 705 actual API calls, 704 response texts, and 703 structurally valid responses plus E1's one malformed JSON response. E2's additional block involved 63 calls over 14.15 minutes, bringing cumulative E2 time to 43.31 minutes. E1's post hoc reference analysis (Appendix E) issued zero new requests and did not replace the original response.

A.2 Dependencies and decisions fixed before execution

The earlier exploratory series comprises SC01 through RC02. Within each trial, materials, instructions, planned response slots, scoring, missingness rules, resource limits, and stopping rules were fixed before live requests. The manuscript's organization and central claims were developed after observing these results. The 11 trials are therefore not treated as one preregistered confirmatory study. They form an adaptive exploratory series in which earlier failures informed later storage contracts.

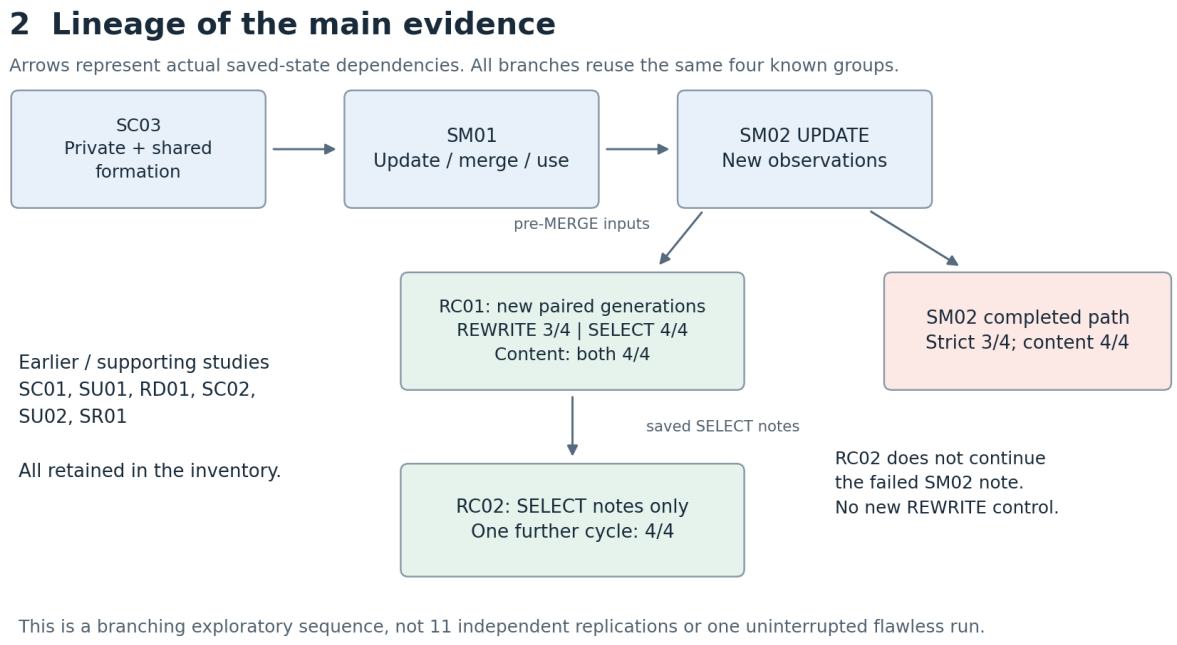


Figure A1. The main sequence based on actual saved states. Both RC01 conditions generated fresh responses from the materials after SM02's UPDATE and before its MERGE. RC02 inherited RC01 SELECT's saved notes. This was neither a correction and rescoring of SM02's defect nor a third cycle continuing directly from SM02's failed notes. The tables in Appendix A include the 11 earlier trials containing this main sequence and the subsequently added E1 and E2.

SC03 allocated 48 responses: eight private records, four shared drafts, four partner edits, and 32 subsequent decisions. Decisions crossed private-only versus private-plus-shared input, both roles, and two repetitions. SM01 and SM02 each allocated 32 responses: eight updates, eight integrations, and 16 decisions, with only

the final decision repeated twice on identical inputs. RC01 used four groups × two roles × two conditions × integration and decision, totaling 32 responses. RC02 used four groups × two roles × update, integration, and decision, totaling 24.

RC01 used the same initial materials for each role under both conditions and checked byte equality by hash. Condition order was balanced across roles and groups and reversed for subsequent decisions. Within-stage order used the fixed seed 2026091905. RC02 used seed 2026092001 and balanced LOCAL_GROUP/PARTNER_GROUP presentation order with two instances of each order per role. These procedures controlled execution order; they did not make the four groups random samples from a population.

RC01's directional prediction arose from SM02's retention of an old origin ID alongside new observation values and revision. Its pre-run plan left row selection to the model while replacing transcription of persons, values, revisions, and provenance with controller-side copying to prevent within-row mixing. Exact copying is a property of that mechanism, so the empirical question was whether the model could select the correct row, retain role-specific information, and support a correct subsequent decision. RC01 pre-run plan, Sections 1 and 5.

The prediction was SELECT−REWRITE ≥.25, or at least one additional successful group among four known groups. This was a small-sample engineering prediction motivated by a known defect, not an estimated population effect of 25 percentage points or a probability expressing confidence. The plan explicitly required closure as NOT_MET even if both methods achieved 4/4 and the difference was zero, and prohibited further collection if the difference was negative. This local prediction is distinct from E1's later extension to unused materials.

A.3 Success, content-only evaluation, and missingness

Evaluation of the main sequence checked the assigned customer, observer, observation values, revision, origin ID, current references, and whether the actual input supported the output. Intermediate saved states were evaluated alongside final decisions. Later primary outcomes were the proportion of groups whose two roles satisfied every required stage.

Trial	Primary outcome unit and definition	Pre-run local prediction
SC03	`E_BOTH`: group–replicate combinations with successful shared formation and both roles' decisions and evidence under BOTH; four groups × two repetitions	Formation 4/4; E_BOTH=1; customer difference L=0; channel difference A=1
SM01	`M_PAIR`: groups in which both roles' UPDATE, MERGE, and two PROBE responses all pass / 4	≥.75
SM02	`M_PAIR_2`: groups satisfying the same end-to-end criterion in the second cycle / 4	≥.75
RC01	`M_END`: groups in which both roles' MERGE and PROBE responses—all four—pass under a method / 4	SELECT−REWRITE ≥.25
RC02	`M_CONT`: groups in which both roles' UPDATE, MERGE, and PROBE responses—all six—pass / 4	≥.75

These thresholds were local predictions for small trials, not general reliability guarantees or power-based acceptance criteria. A trial whose prediction was met at 3/4 was not described as error-free.

From SM01 onward, content-only evaluation followed a rule fixed before execution: omit origin-ID, declared-reference, and input-support requirements while retaining persons, values, revisions, and final decisions. It did not replace the primary result. Earlier trials used different sensitivity definitions, so their original metric names are retained here. Primary and sensitivity results across all 11 trials are not averaged as if they shared one scale.

With missing data, the lower bound is the number of known successes divided by the fixed denominator; the upper bound adds unresolved outcomes to known successes. A group with a known failed component is a failure even if another response is unresolved. These are identification bounds due to missingness, not confidence

intervals. Responses that never reached content evaluation because of structural defects are distinguished from responses with observed content errors.

A.4 Observed coexistence of role-specific information and joint decisions

SC03 achieved shared formation in all four groups. `E_BOTH` was 8/8 group–replicate combinations, as was `E_NO REF`, which relaxed provenance requirements. With private notes alone, customer designs were correct in 16/16 responses and the group-wide channel was appropriately deferred in 16/16. With private plus shared notes, both customer and channel decisions were correct in 16/16. On these materials, adding shared information enabled a group-wide decision while preserving differences between assigned customers. SC03 results.

Adding shared information also supplied information about the partner's area. The shared draft and edited JSON were identical in all four groups. The result establishes successful decisions using shared information; Section 7 discusses the inability to isolate shared formation or partner editing itself.

SM01 gave each role's SC03 saved notes its own new notice, integrated both new observations, and proceeded to subsequent decisions. Both the primary end-to-end outcome and content-only evaluation were 4/4 groups. The 16 saved note states and 16 final decisions met their respective fixed criteria. In these four groups, one integration cycle incorporated the partner's new observation while retaining the role's own new observation. SM01 results.

A.5 Rewriting versus copying complete selected rows

RC01 generated new REWRITE and SELECT responses from SM02's pre-integration materials. Strict end-to-end success was 3/4 groups for REWRITE and 4/4 for SELECT, a difference of .25. Content-only evaluation was 4/4 for both. RC01 results.

Matched group	REWRITE end-to-end	SELECT end-to-end	Content-only evaluation
G01	Success	Success	Both succeed
G02	Success	Success	Both succeed
G03	Provenance failure	Success	Both succeed
G04	Success	Success	Both succeed
Total	3/4	4/4	4/4 each

Target	REWRITE	SELECT
Entire integrated note correct under strict evaluation	7/8	8/8
Shared observation rows correct under strict evaluation	15/16	16/16
Saved assigned facts and own observations	8/8	8/8
Final customer and channel decisions both correct	8/8	8/8

REWRITE G03/B changed the correct origin ID `SM2G3A` to `SM3G3A`, which did not occur in the input. Values, person, and revision were correct, and the declared reference pointed to the appropriate partner document, but that document did not support the incorrect ID. The defect persisted into a correct DOG/DIGITAL decision. This differs from SM02's retention of the old `SM3A`.

Under SELECT, the model selected the latest row in 16/16 instances, and the controller's materialized row exactly matched the selected row in 16/16. The first is an observed model-selection outcome; the second verifies copying behavior. Exact copying is not counted as an additional improvement in model capability. The mechanism does not correct a wrongly selected row, and it can preserve an ID already miswritten during UPDATE.

The difference was localized to one group. The intervention jointly changed instructions, output schema, selection space, output amount, and storage processing, so it cannot identify the isolated effect of copying. No difference was observed in final decisions or retention of assigned facts in this comparison.

A.6 One further cycle from SELECT's saved states

RC02 used the eight notes actually saved by the roles under RC01 SELECT and performed UPDATE→MERGE→PROBE once each. All 24 responses passed; both `M_CONT` and content-only evaluation were 4/4 groups. Both roles' observations advanced from revision 3 to revision 4. All four groups correctly chose the opposite channel from the preceding cycle while preserving customer dog/cat preferences. Latest-row selection and copying all fields of those rows each succeeded in 16/16 instances. RC02 results.

This was a continuation conditioned on known materials and successful SELECT saved states. Without a new REWRITE control, it is not a replication of a between-method difference. Each PROBE was also run only once, so this trial does not evaluate stability over repetitions of the same input.

A.7 Early failures and supplementary diagnostics

SC01 achieved shared formation in 2/4 groups and end-to-end `E_BOTH` in 4/8 group–replicate combinations. Some groups failed provenance requirements during shared formation despite correct final BOTH decisions. In SU01, initial ALL/ALL observations were replaced with UNKNOWN/UNKNOWN and passed onward. Primary `C_PAIR` was 0/4, with one structurally missing response and five dependent calls not issued. Later successes do not overwrite these results. SC01 results; SU01 results.

RD01 compared recording the same initial source records with and without three lines of group-decision rules in the instructions. For four ALL/ALL source records × two repetitions, correct observation preservation was 1/8 with the decision rules and 8/8 with recording-only instructions. For mixed-value source records, both conditions achieved 8/8. This offers local evidence for separating recording from decision instructions, but does not isolate instruction meaning, length, or allocation of attention. RD01 results.

SC02 executed 12 of 48 planned responses. Four shared drafts returned arrays instead of A/B objects, so 36 dependent calls were not issued. The primary outcome bounds were [0,1], yielding INCONCLUSIVE. This is neither a 0% content-success result nor a success obtained by converting arrays after the fact. SC02 results.

SU02 achieved 3/4 groups under both primary and content-only evaluation, below its prediction of success in all groups. In G03's change condition, the partner with correct notes deferred the channel decision in one of two repetitions of the same input. SR01 varied the arrival order of old and new versions. Content-only success was 4/4 groups, while strict end-to-end success across both orders was 0/4. None of the final 16 decisions reverted to old content, but 10 of 16 saved states retained historical reference names in the current direct-reference field. This differs from confusing observers or corrupting observation values. SU02 results; SR01 results.

A.8 Interruption and continuation

The initial run saved all 512 E1 responses and 129 E2 responses. The next E2 request received an HTTP 200 and a completed provider response, but local output validation timed out at its two-second limit, triggering a stop under the fixed rule. The cause remains unidentified. After the stop report, the user authorized changing the original no-resumption rule. The stopped response slot and 62 unissued slots were then run. The stopped slot was retried once with byte-identical request content, while the original 129 responses and stop record were retained. Instructions, scoring, and the two-second validation limit remained unchanged during continuation. Original stop report; change record before continuation (see the evidence index).

All 63 continuation responses in E2 were structurally valid. Including the stopped attempt, E2 involved 193 model calls and the combined E1/E2 total was 705, with 704 saved responses. Planned responses, actual calls, and saved responses are recorded separately.

A.9 Mechanistic distinctions and the earlier workflow

In SC03, adding shared information also increased information about the partner; drafts and edited records were identical in all four groups. The transition from SR01 to SM01 also changed multiple elements. Neither design isolated shared formation, partner editing, or separation of declared references.

E1's ID format was not randomly compared against the earlier format, and the internal mechanism of ID errors was not measured. SM02's UPDATE references were not passed to MERGE, so the study did not establish a causal path from those references to stale-ID retention.

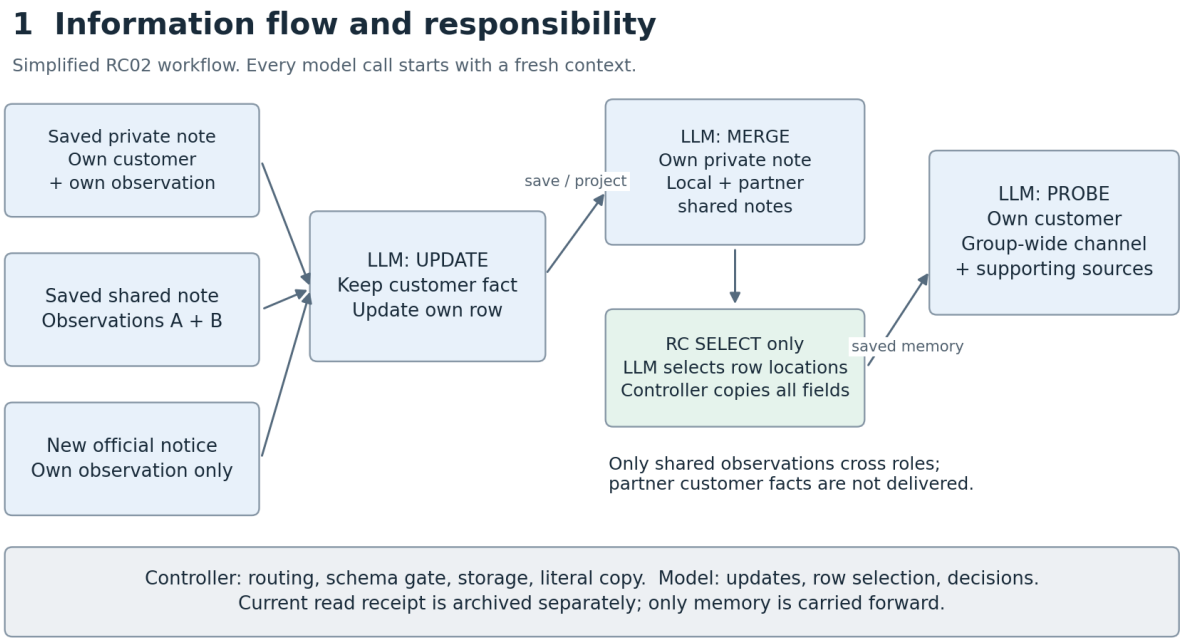


Figure A2. RC02's update, integration, and use workflow. One role is shown; its partner follows the same process. Shared portions are exchanged, and MERGE uses selection and copying for shared observation rows. UPDATE, private notes, and declared evidence references are model-generated. Unlike E1 in Figure 1, this workflow includes an update stage and uses different comparison conditions.

Appendix B. Evidence and Reproduction Resources

Supplement S1: reproduction package provides a portable manuscript, figures, implementation supplement, materials, instructions, allocations, responses, saved states, subsequent inputs, scoring code, hashes, and computational reproduction. The repository includes English and Japanese manuscripts, figures, and supporting documentation. Links in the PDF point to versioned GitHub resources; the Markdown edition uses relative paths within the repository.

The package contains 1,127 original responses. Frozen evaluators rescore 704 E1/E2 responses and 88 SM02/RC01/RC02 responses, totaling 792. The other eight trials contribute 335 responses preserved with their original evidence, saved aggregates, and frozen code. The 31 files containing the earlier scoring code and its local dependencies retain their original bytes. Formal and post hoc reference tables are regenerated separately from saved evidence, without issuing model requests with changed instructions or scoring.

Retention and versions. Explicit authorization on September 22, 2026 established local retention with no deletion deadline for the specified reproduction materials, ZIP, supplementary code, and repackagings of the same evidence. File lists and hashes are retained. The current retention policy supersedes the old October 12 deadline remaining in original configurations. This exception does not encompass whole original run directories or unrelated experiments. Extracted older versions and current materials remain available after removal of older ZIPs. Availability record (see the evidence index).

Source indexes: evidence mapping for the 11 earlier trials (see the evidence index); formal combined E1/E2 analysis; pre-run design; change record after the stop (see the evidence index); supplementary calculations (see the evidence index). Original study reports and procedural documents are in Japanese.

Appendix C. Implementation and Evaluation Principles

C.1 Share the contract and construct subsequent inputs from actual saved states

The schema in the instructions and the post-response validator use a shared contract for required fields, types, and current references. Hashes link each role's saved notes to subsequent inputs. At each stage, the controller projects the private portion for its owner and the shared portion for exchange. Current declared references are stored separately as audit records.

C.2 Separate structural validation from fixed-rule content scoring

Structural/reference validation checks syntax, types, and the existence of referenced documents. Scores are produced by deterministic Python code comparing persons, values, revisions, origin IDs, input support, decisions, and revocation states against fixed expectations. E1 uses frozen RC02 scoring through `e1_evaluate`; E2 uses `e2_evaluate`. No separate LLM or human interprets each free-text answer to assign outcome labels. Structurally usable errors are saved and passed onward, and their inheritance is scored.

C.3 Verify selection, copying, and subsequent use separately

Under SELECT, the model specifies a row in the actual input, and the controller copies its person, values, revision, and origin ID together. Agreement with the selected row verifies a property of the storage operation. Correct row selection and subsequent decisions are evaluated as research outcomes. Private notes and declared references remain model-generated. Missingness is recorded with planned IDs and parent dependencies, and formal analyses are separated from post hoc derived analyses.

Pseudocode, the codec, trial-specific scoring entry points, input/output validation, retry and stopping rules, E2 continuation, and structural correction are documented in Supplement S3: implementation details, in Japanese. The frozen scorers and dependencies (see the evidence index) and Appendix B's offline reproduction link implementation to reported outcomes.

Appendix D. Interval Estimates and Post Hoc Descriptive Supplements

D.1 E1 intervals: all completions and their components

Let a=`SELECT-onlysuccesses`, b=`REWRITE-onlysuccesses`, and Δ=`SELECT-REWRITE`. For the two discordant proportions, use 97.5% Clopper–Pearson intervals and form the difference interval as $[L_a - U_b, U_a - L_b]$. By Bonferroni, the marginal intervals jointly cover their targets with probability at least 95%. Each marginal uses a binomial model, requiring the working assumption of independent, identically distributed groups. For

unresolved outcomes, take the envelope over all completions. All numerical bounds below are percentages or percentage points.

Criterion / G23 REWRITE completion	a	b	La	Ua	Lb	Ub	Difference lower bound	Difference upper bound
strict / False	5	5	2.160258	18.762297	2.160258	18.762297	-16.602039	16.602039
strict / True	4	5	1.398607	16.644683	2.160258	18.762297	-17.363690	14.484426
content / False	4	2	1.398607	16.644683	0.262821	12.098875	-10.700269	16.381862
content / True	3	2	0.749783	14.435629	0.262821	12.098875	-11.349092	14.172807

Strict envelope: -17.363690 to 16.602039 percentage points.

Content-only envelope: -11.349092 to 16.381862 percentage points.

The primary-analysis envelope is −17.363690 to 16.602039. The post hoc brace correction corresponds to the completion in which G23 succeeds, yielding −17.363690 to 14.484426. The lower endpoints coincide because that completion already determines the envelope's lower endpoint. The calculation does not set marginal lower bounds to zero. This table connects the original formula to the reported values; the formal primary p-value remains uncalculated.

An independent implementation inverted direct sums of binomial probabilities by bisection. It agreed within 1e-10 with the beta quantiles used by the original code and SciPy's exact-interval API. It reads saved scores without importing the live research runner.

D.2 Group-level aggregation of shared observation rows only

The number of groups in which all four shared rows—A/B observations in both roles' copies—passed strict evaluation was 61/64 for REWRITE and 64/64 for SELECT. REWRITE failures were G06, G11, and G36. This is a post hoc group-level aggregation of existing component scores, without an additional test. Strict evaluation includes current-input references as well as content and provenance, so it does not identify the isolated effect of copying. The 256 rows are not treated as an independent N.

The observed difference was SELECT−REWRITE = 3/64 = +4.6875 percentage points. Applying the same pair of 97.5% Clopper–Pearson intervals and difference construction as in D.1 to three SELECT-only successes and zero REWRITE-only successes gives a post hoc nominal 95% interval of −5.867991 to 14.435629 points. The marginal intervals were 0.749783–14.435629% for SELECT-only success and 0–6.617775% for REWRITE-only success. This descriptive supplement was added in Japanese manuscript v0.7. It depends on the working independence assumption and does not adjust for post hoc selection of this component. The local sample difference is reported alongside uncertainty about generalization. As specified in the original plan, no additional significance test is applied to secondary outcomes.

Stage-specific components were already secondary outcomes in the original design. It is this aggregation and presentation, rather than component evaluation as a whole, that is post hoc.

D.3 Breakdowns by scenario and venue labels

The materials cross eight scenario names with eight venues, generating distinct observation values and origin IDs for each of 64 groups. This is not a design with eight repeated measurements of each of eight parent datasets. Shared labels and templates may still have effects; these tables do not establish independence. No additional test or bootstrap was performed to claim robustness with eight clusters.

Scenario names

Label	N	Both succeed	R only	S only	Both fail	Unresolved
Book exchange	8	5	2	1	0	0
Community gardening class	8	7	0	1	0	0
Stargazing event	8	7	1	0	0	0
Cooking workshop	8	7	0	1	0	0
Historical walking tour	8	7	0	0	0	1
Science craft exhibition	8	7	1	0	0	0
Disaster-preparedness class	8	7	0	1	0	0
Music appreciation event	8	7	1	0	0	0

The book-exchange label had three resolved discordances—two R-only and one S-only—while six other scenarios had one each. The historical walking tour had no resolved discordance and one unresolved group. This concentration is descriptive: the arrangement with eight scenario names cannot distinguish scenario effects from chance or other material attributes.

Venues

Label	N	Both succeed	R only	S only	Both fail	Unresolved
Hill venue	8	7	1	0	0	0
Central venue	8	7	0	1	0	0
North venue	8	7	1	0	0	0
South venue	8	7	0	0	0	1
Riverside venue	8	7	1	0	0	0
East venue	8	5	1	2	0	0
West venue	8	6	1	1	0	0
Station-front venue	8	8	0	0	0	0

D.4 Descriptive intervals for E2

Condition	Completion	Completion CP95%	Evidence-appropriate handling	Handling CP95%
ABSENT/DETAILS	8/8	63.06%–100.00%	8/8	63.06%–100.00%
ABSENT/ID_ONLY	0/8	0.00%–36.94%	8/8	63.06%–100.00%
CORRECT/DETAILS	8/8	63.06%–100.00%	8/8	63.06%–100.00%
CORRECT/ID_ONLY	8/8	63.06%–100.00%	8/8	63.06%–100.00%
OLD/DETAILS	8/8	63.06%–100.00%	8/8	63.06%–100.00%
OLD/ID_ONLY	0/8	0.00%–36.94%	8/8	63.06%–100.00%

These nominal 95% intervals were added post hoc for individual cells; they are not simultaneous intervals. They are reference intervals under a model treating the eight fixed cases as exchangeable, independent Bernoulli trials. The six conditions share the same eight cases and do not create 48 independent cases. Zero completion under an incorrect ID plus ID_ONLY follows from a condition made unresolvable under the scoring contract; its 0/8 interval is not an estimate of an unknown target-resolution ability. The 8/8 evidence-appropriate result records preservation and deferral without inventing targets on these inputs.

D.5 Verification of the pre-run power calculations

The following scenarios appeared in the September 20, 2026 estimates and September 21 pre-run plan. They were independently checked by summing multinomial probabilities.

N	True difference	Discordance rate	Power
64	10 percentage points	0.30	23.59%
64	20 percentage points	0.30	82.69%
64	20 percentage points	0.40	66.76%

Assumptions were fixed N, no missingness, independent and identically distributed groups, and a two-sided exact McNemar test at α=.05. The 20-point scenario was a compromise between cost and detectable effect size, not an empirical estimate of the expected effect. Section 4.2 states this design rationale. The formal data contain nine resolved discordances and one unresolved group, yielding a total discordance range of 9–10/64 (14.06–15.63%). No post hoc power calculation uses these observed values.

Discordance rate alone does not determine whether sensitivity increases or decreases. Under the working independent-group model, let D = SELECT success − REWRITE success, Δ = E[D], and q = P(D≠0). Then Var(mean(D)) = (q−Δ^2)/N. Holding Δ constant, a decrease in q reduces variance; holding the imbalance among discordances constant also changes Δ. The difference between the planned q=.30 and the descriptive observed range therefore does not, by itself, establish lower realized power.

D.6 Recalculation

Verification code (see the evidence index). Appendix B gives instructions for the reproduction package.

The checked environment was Python 3.13.5 / SciPy 1.17.0. Input files, hashes, unrounded values, and verification results are recorded in `numerical-audit.json` alongside the code. The script regenerates only its derived JSON and audit report in that directory.

D.7 Output lengths of original responses

This post hoc description was added in Japanese manuscript v0.7 using all 512 saved original responses. It counts the original bytes, including the one malformed JSON response, without correction or exclusion.

Method	Stage	Responses	Mean UTF-8 bytes	Median	Minimum	Maximum
REWRITE	MERGE	128	587.3125	587.0	579	607
SELECT	MERGE	128	487.6172	489.0	481	493
REWRITE	PROBE	128	225.0234	226.0	224	227
SELECT	PROBE	128	224.7188	224.5	207	226

These are lengths of response text returned by the model, not internal reasoning, token counts, input lengths, total transmitted data, or notes after controller-side copying. At MERGE, the difference also reflects the schemas: SELECT specifies locations, whereas REWRITE emits complete rows. Length differences do not establish a causal explanation for outcome differences. Neither 128 responses per cell nor 512 total responses replaces the number of independent materials.

Unrounded output-length and shared-observation interval results (see the evidence index). The portable package's `replay.py` also recomputes these supplements from original responses and fixed scoring.

Appendix E. Post Hoc Reference Analysis after a One-Brace Correction

After collection ended, a derived copy of E1's unresolved response was made by removing only the extra final }—one byte. The text changed from 227 to 226 bytes, with no other modification. Content, provenance, and current references were then evaluated using the original criteria. Because this was the final PROBE, no subsequent input was generated from the corrected copy. The original response, structural outcome, and formal primary aggregate were retained.

Post hoc reference result: fixed 64 groups	REWRITE	SELECT
Strict end-to-end success	60/64 (93.75%)	59/64 (92.19%)
Content-only evaluation	61/64 (95.31%)	62/64 (96.88%)

The reference primary difference was −1.5625 percentage points, with a nominal 95% interval of −17.36 to 14.48 points using the original formula. There were five REWRITE-only and four SELECT-only successful groups, giving a two-sided exact McNemar **reference p=1.000**. No second p-value was assigned to the content-only result. This is a reference value for a correction selected after observing the result, not the formal primary p-value. The interval likewise does not account for post hoc selection of the correction. A p-value of 1 neither proves equivalence nor means a 100% probability of no difference. Post hoc sensitivity analysis and derived materials.

Scope of Update Control in Private and Shared LLM Records

Field-Level Comparisons and Post-Update Rollback

Kentaro Sato
Independent Researcher
Contact: info@metaclan.jp

English version v1.0

Repository and research materials | Apache License 2.0

Abstract

When an LLM continues to use private information and shared records, the correctness of a joint answer and the freshness of each record are distinct evaluation targets. We ran the same Qwen3.5-27B model as two roles in separate contexts on a synthetic task: retaining each assigned customer's dog-or-cat preference while combining both roles' observations to choose paper or digital guidance. We explored which record fields and update stages were supported by dividing responsibilities between the model and deterministic processing. Model proposals, actual saved notes, subsequent inputs, and later responses were linked to distinguish record correctness from readout fidelity.

A comparison applied different storage controls to the same 36 update proposals, generated from 18 previously used materials with two repetitions each. Updating only shared observations produced correct joint guidance in 36/36 proposals, while leaving four stale own-observation records. Repairing only those own observations fixed all four, but end-to-end success was unchanged because shared-field defects overlapped with them. Applying control to both observation fields supported two cycles of updates, exchanges, and reuse of actual saved notes. A separate diagnostic, using eight new cases with two repetitions each, allowed customer information outside deterministic latest-revision selection to change. All new preferences were initially adopted, but old notices subsequently rolled back the preference, origin ID, and revision in 14/16 trajectories. Later responses faithfully followed those stale records.

Within this fixed task and single model, the comparisons exposed both record defects hidden by correct answers and local repairs hidden by end-to-end outcomes. Retaining fixed information, adopting new information, and maintaining it after an update were evaluated as distinct requirements. The scope of a division of responsibilities should therefore be specified by record field and update stage, with saved artifacts linked to subsequent use.

Keywords: LLM agents; private memory; shared memory; state updates; provenance; deterministic processing; exploratory evaluation

1. Introduction

Consider two reception staff members serving different customers. Role A's customer wants a dog design, while role B's customer wants a cat design. To choose guidance for the venue as a whole, the two roles must combine their observations of paper and digital accessibility in their respective areas. They need to agree on joint guidance while giving each customer the requested design. Continuing both tasks through the same record pathway is the problem studied here.

Correct joint guidance can coexist with stale observations in a private note. Conversely, repairing those observations may leave end-to-end success unchanged if another field remains defective. **Final-answer correctness does not capture the correctness of the entire record, and end-to-end success does not capture every local repair.** Evaluating success requires relating the fields reached by a control to the defects that remain.

The first report traced the storage and subsequent use of assigned facts and shared observations, identifying provenance defects alongside correct answers. This report synthesizes an adaptive exploration with a different, open-weight model around the question: **Which record fields and update stages did the division of responsibilities support?** The central evidence is a comparison that varies the fields controlled for the same update proposal, together with a diagnostic tracing rollback after a legitimate change to customer information. Readout comparisons and two cycles of continued use establish how the saved artifacts could be used. Appendix A describes the connection to the first report and the initial transfer results. First report, English version.

Table 1. Three findings and their supporting observations. These are perspectives used to synthesize the exploratory results, not hypotheses fixed before the entire series. Section 4 specifies materials and units of analysis.

Finding	Supporting observation	Section
Record freshness, readout fidelity, and answer correctness can be observed separately	Accurate extraction from stale saved notes did not ensure agreement with the latest state; the customer-update diagnostic also faithfully used records that had rolled back	5.1, 5.4
Evaluation by record field reveals both remaining defects and local repairs	Correct joint guidance coexisted with stale own observations; repairing the own-observation field left end-to-end outcomes unchanged when shared-field defects overlapped	5.2
Retention, adoption of new information, and post-update maintenance impose different requirements	Fixed customer information was retained through two cycles and used for its intended task; a separate diagnostic found rollback after adoption of new preferences	5.3, 5.4

Private information here consists of explicitly assigned customer facts and observations of each role's own area. Shared state is a structured record combining both roles' observations. This operationalization makes it possible to evaluate separate retention, necessary sharing, and use for the respective tasks.

2. Related Work

2.1 Memory updates and time

LLM systems with external memory have been evaluated on adapting stored content to new information. LongMemEval includes knowledge updates and temporal reasoning, while MemoryAgentBench addresses conflict resolution under incremental inputs. Supersede compares maintenance of bounded notes without re-presenting the original history against access to the full history. STALE separately evaluates current-state resolution, resistance to stale premises, and behavior appropriate to the current state. These studies provide a context for our distinction among retention, updating, and post-update maintenance. LongMemEval; MemoryAgentBench; Supersede; STALE.

For temporal representation, Zep distinguishes event time from ingestion time and represents fact validity intervals. MemConflict distinguishes genuine updates, later misinformation, and facts that coexist under different

conditions. Our revision numbers define a contract for selecting the current state of an observation for the same person. We evaluate information age separately from arrival order, including notices that arrive later but contain an older revision. Zep; MemConflict.

2.2 Evaluating records separately from subsequent use

Prior stage-level evaluations include HaluMem, which separates extraction, updating, and question answering, and A-TMA, which separates state appropriateness in the memory bank, retrieved evidence, and answers. MemAudit evaluates recovery of user state from the resulting memory artifact separately from assistance-task success, comparing access to the entire store with the normal retrieval interface. These studies directly connect to our separate evaluation of final answers and record quality. HaluMem; A-TMA; MemAudit.

Intermediate-process tracing also has precedents. MemTrace by Deng et al. analyzes where errors arise using execution graphs of memory operations and intermediate variables. A different paper, MemTrace by Long et al., probes the same knowledge point with current-state, historical-state, and change-trajectory questions, diagnosing evidence reachability and use. Our study belongs to this line of stage-level evaluation and links controls applied to different fields of the same update proposal with subsequent use of the actual saved artifact. MemTrace: Deng et al.; MemTrace: Long et al..

2.3 Attribution and the division of model and code responsibilities

MemGPT addresses hierarchical context management, while Collaborative Memory provides private and shared memory areas with provenance-based access control. GroupMemBench evaluates speaker-attributed information and answers conditioned on the querying user in multi-party conversations. We separate customer information, own observations, and shared observations, measuring consistency when information about the same person appears in multiple record fields. Provenance representation also connects to PROV-DM's distinction among entities, activities, and agents. MemGPT; Collaborative Memory; GroupMemBench; PROV-DM.

Reddy and Challaram divide responsibilities by assigning candidate extraction to an LLM and freshness selection to code. StateAuditor likewise uses code to verify quoted evidence and temporal ordering for proposed state transitions. CaMeL's separation of control and data flow provides a related design principle in a security setting. Our comparison varies the scope of latest-revision selection by person and whole-row copying: whether these operations apply to own observations, shared observations, or both. Don't Ask the LLM to Track Freshness; StateAuditor; CaMeL.

Table 2. Units of comparison in closely related work and this study. Existing stage-level evaluations and divisions of responsibility are related to our local comparisons and observations along actual execution paths.

Related work	Main unit of comparison or diagnosis	Correspondence examined here
HaluMem; A-TMA	Memory operations; records, evidence, and answers appropriate to the state	Fidelity to saved artifacts versus agreement with the latest state
MemAudit	Assistance tasks versus state recovery from stored artifacts	Defects in own observations alongside correct joint answers
MemTrace (Deng et al.)	Memory operations and intermediate variables in execution graphs	Repairs and remaining defects across four branches of the same update proposal
Freshness-selection decomposition; StateAuditor	Extraction or transition proposals and rule-based selection or verification	Fields covered by control and fields outside its scope
STALE; MemTrace (Long et al.)	State, premises, and behavior; the time and use of the same fact	An adopted new state rolling back under an old notice and the resulting saved state being used

Our focus is to use field-level comparisons of the same proposal to connect record defects hidden by correct answers with local repairs hidden by end-to-end outcomes, and to trace post-update rollback from actual saved

notes to subsequent responses. We position this as an exploratory report with a fixed task and a single model, specifying the evaluation denominators and the fields covered by control.

3. Task and System

3.1 Private preferences and joint guidance

Each group comprises two roles, A and B. Both use the same model, with a fresh context for each call. Information across roles and across time is connected through notes and notices explicitly supplied by the controller.

Each role holds its assigned customer's dog-or-cat preference and observations of paper and digital accessibility in its own area. The two areas are disjoint and together cover the venue. Observation values are binary: ALL means that everyone can use the channel, and NOT_ALL means that not everyone can. If exactly one channel is ALL in both areas, that channel is selected for joint guidance. If neither or both qualify, the guidance decision is deferred as UNKNOWN.

For example, if everyone in A's area can use both channels but everyone in B's area can use only digital guidance, the joint channel is digital. The assigned customer's dog-or-cat preference is determined from information separate from that joint decision.

Each role's shared note is a copy of the A/B observation rows. Immediately after a new notice, its own new revision can coexist with its partner's old revision. During exchange, each role receives the observation row that its partner actually saved for itself. Customer preferences are not included in the exchanged observation row.

3.2 Three record fields and their responsibilities

A *record field* here is a logical area within a note targeted by an update operation. Customer information and own observations occupy separate fields of the private note, while shared observations occupy a field of the shared note. The distinction does not refer to the number of physical files or memory stores.

Table 3. Record fields and the division of responsibilities in continued-use trials. A role's own observations are distinct from information about its assigned customer.

Record field	Content	Responsibility during updates	Subsequent use
Customer information	Assigned role, dog/cat preference, and origin ID; a separate customer revision is added in the customer-update diagnostic	Save the content written by the LLM	The LLM extracts the assigned role and preference; the preference determines the design
Own observations	The role's paper/digital observations, observer, origin ID, and revision	Code selects that person's latest candidate row from the input and copies the entire row	Check retention and updating of own observations; supply the row to the partner during exchange
Shared observations	Observation rows for A and B	Code selects and copies the latest row separately for each person	The LLM extracts four values; code computes the joint guidance channel

The field-level comparison independently enables or disables the controls for own and shared observations in Table 3. Customer information remains model-written in every condition. Thus, control of the own-observation field does not replace the entire private note with ground truth.

Latest-revision selection first restricts candidates to the relevant observer and then compares revisions within that person's records. It does not compare A's revision numbers directly with B's. The selected row's person, two values, origin ID, and revision are copied together, and the source and before/after states are recorded.

Candidates are limited to rows actually present in the current input. The storage controller never receives the expected state used for scoring.

3.3 Separating proposals, saved artifacts, and subsequent inputs

The model's raw response is retained, and the storage procedure is applied to structurally acceptable update proposals. Subsequent inputs are constructed by reading back the saved artifacts. Structurally acceptable but incorrect content is passed onward unchanged when it lies outside the designated storage control.

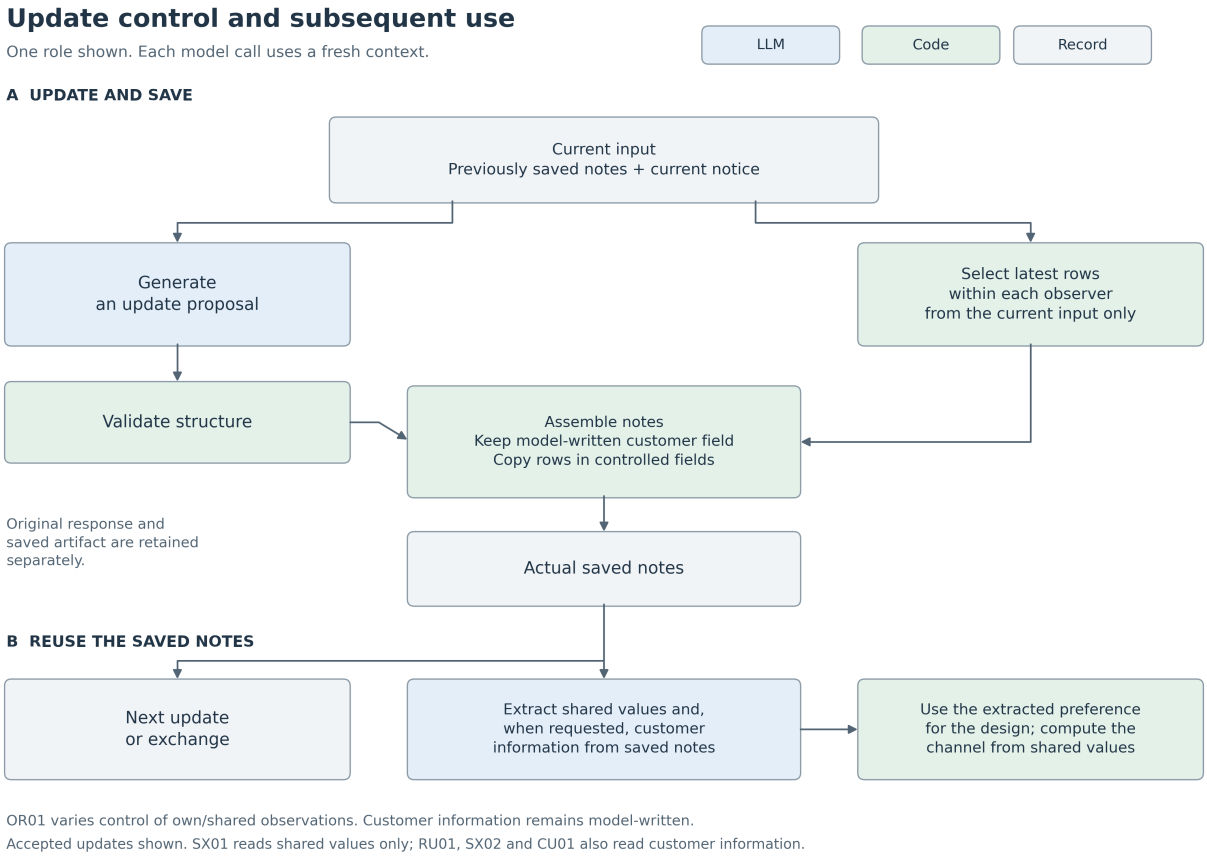


Figure 1. Division of responsibilities in continued-use trials. Observation-row copying is switched on or off in the field-level comparison. The final readout in the first cycle requests only the four shared values; the selective-use, second-cycle, and customer-update trials also request customer information. Raw responses and actual saved artifacts are retained separately at every stage.

3.4 Worked example: a correctly updated preference rolls back

In the first group in the execution order of the customer-update diagnostic, A's customer changed their preference from cat to dog. Table 4 follows the initial state, notices, actual saved states, and subsequent response. The example is CU01-C06-R1/A.

Table 4. How old and new information passed through the same trajectory. Origin IDs are reproduced exactly from the records.

Stage	Preference	Revision	Origin ID
Initial note	Cat	76	`K5E1ED87EFA479493`
New notice	Dog	77	`K0D9B80C5A23D1363`
Actual saved state after the new notice	**Dog**	**77**	**`K0D9B80C5A23D1363`**

Continued on the next page

Stage	Preference	Revision	Origin ID
Old notice arriving later	Cat	76	`K5E1ED87EFA479493`
Proposal and actual saved state after the old notice	**Cat**	**76**	**`K5E1ED87EFA479493`**
Final design answer	**Cat**	Re-citation not requested	Re-citation not requested

The input receiving the old notice included the correctly saved dog preference at revision 77. The model reverted to the cat row at revision 76. Because customer information received no corrective storage control, that row passed into the next call. The relevant part of the subsequent response was:

```
{"customer":{"subject":"A","animal":"CAT"}}
```

B's customer still preferred a dog, and the joint channel remained paper for both roles. The correct final outcome was therefore for both A and B to provide a dog design. **The criterion is agreement with each role's legitimate latest information, not whether the two answers are identical or different.** Section 5.4 reports all cases; Supplement S1 links the example to the raw responses.

4. Evaluation Methods

4.1 Adaptive exploration and relationships among trials

This exploration separated readout, storage, selective use, and continued updating in response to defects found during transfer. Materials, instructions, scoring rules, execution order, and stopping conditions were fixed locally before generation for each small trial. The series was not a confirmatory study fixed in full at its outset. The three findings in Table 1 synthesize the exploration at the writing stage; they do not change the pre-run criteria or formal results of individual trials.

Table 5. Trials and evaluation units. IDs are included for tracing source records. Repetitions use different generation seeds with the same materials.

Trial	ID	Materials and repetitions	Comparison or trajectory
Readout decomposition	RS01	12 main materials and 4 boundary inputs, each repeated twice	Direct decision; extraction plus computation; model decision from correct four-value inputs
Saved-note readout	SVR01	16 current and 10 stale saved inputs, each repeated twice	Direct decision versus extraction plus computation
Field-level storage comparison	OR01	18 previously used materials, each repeated twice	The same 36 proposals branched into four own/shared-control conditions
First cycle	SX01	8 new groups, each repeated twice	Own-observation update, exchange, old notice, shared readout
Selective use	RU01	32 final role-specific note sets from the first cycle	Unchanged; preference changed; observations changed; both changed
Second cycle	SX02	32 final role-specific note sets from the first cycle	Further update, exchange, old notice, combined private/shared readout
Customer-update diagnostic	CU01	A separate set of 8 new groups, each repeated twice	New customer preference, old notice, combined private/shared readout

Readout decomposition reused materials from an earlier output-load diagnostic. Saved-note readout selected notes from the transfer trial that could be connected to subsequent calls, including all 10 qualifying stale inputs. The field-level comparison reused 18 materials from the subsequent update diagnostic. Supplement S1 records selection rules and links to the original inputs.

The first cycle, selective-use diagnostic, and second cycle all depend on the same eight groups. Selective use manipulates the saved-note inputs rather than updating them through actual notices. The second cycle begins with the notes actually saved in the first cycle, not the manipulated notes. The customer-update diagnostic uses new initial notes with a dedicated customer revision and is separate from the continuation into the second cycle.

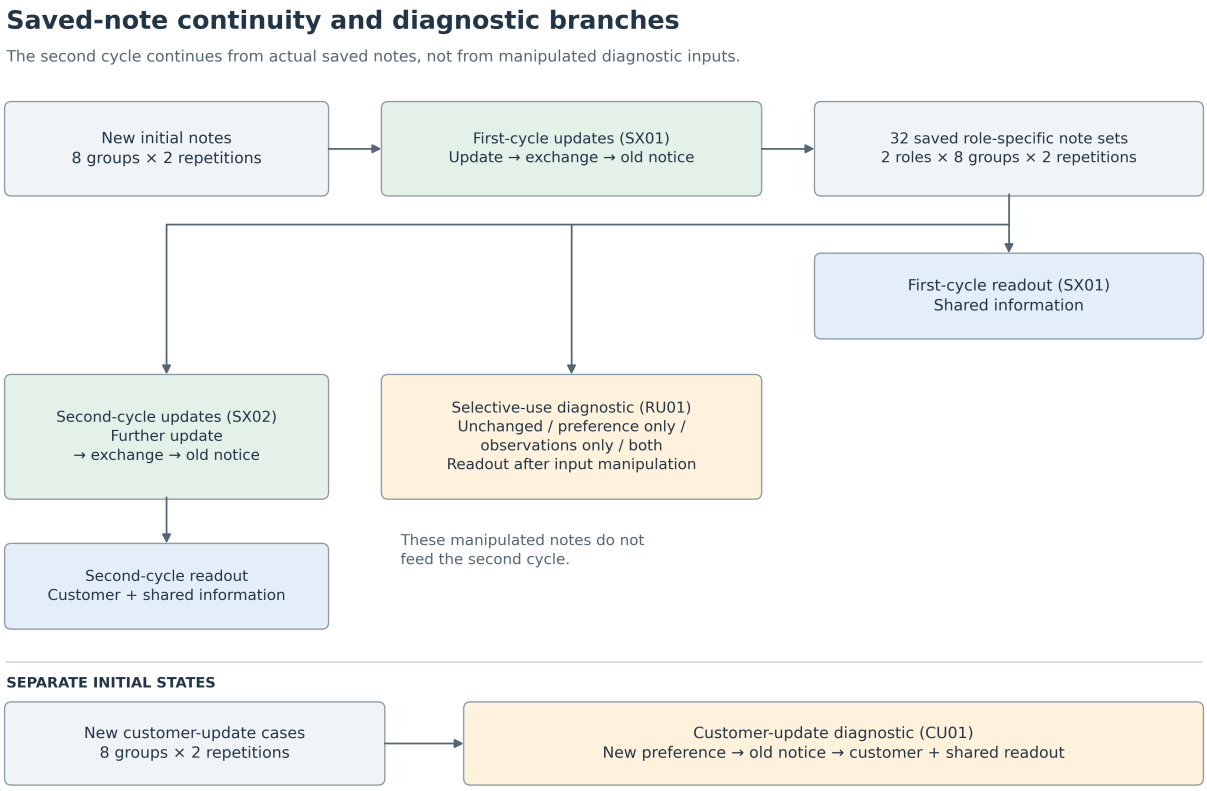


Figure 2. Continued use of saved artifacts and diagnostic branches. Manipulated notes from the selective-use diagnostic are not passed to the second cycle. The customer-update diagnostic starts from separate initial states.

4.2 Readout and storage comparisons

The main readout comparison supplied identical private and shared notes to direct decision and extraction plus computation. Direct decision asked the model to choose paper, digital, or deferral. Extraction plus computation asked it to transcribe the four A/B-by-paper/digital values from the shared note; code then derived the channel from that actual response. A third condition supplied only the correct four values from the original material and asked the model to choose a channel. This was not a second model receiving the extraction condition's output.

In saved-note readout, both conditions were instructed to use the shared note. Scoring separately evaluated agreement with the supplied note and with the latest state represented by the original notices.

The storage comparison branched each LLM update proposal into four controls: neither field, shared observations only, own observations only, and both. Own observations were already current in the initial input, so their protection was evaluated. The 18 materials comprised 10 in which a new notice should update shared observations and eight in which the state should be retained despite an old notice.

Shared information was read from each branch's saved notes using the same extraction-plus-computation method. Before generation, readout calls were shared only when they belonged to the same update proposal and matched exactly in instructions, materials, schema, model, and seed. Thus, four conditions across 36 proposals represented 144 readout instances but required 55 actual responses. Generations were not shared across different materials or repetitions.

4.3 Continued use and changes to customer information

In the first cycle, both roles received new observations and exchanged their actual saved observation rows. They then received old notices and used the saved notes for joint guidance. Latest-revision selection by person was applied to both own and shared observation fields, as in Table 3. Customer preferences remained constant and were carried forward from the preceding private note without repeated external reinjection of ground truth.

Selective use produced four conditions from the final notes: change only the customer's preference, change only the paper/digital observations, change both, or leave them unchanged. When observations changed, their private and shared copies were kept consistent. The diagnostic tested whether manipulating customer information changed only the design and manipulating observations changed only joint guidance.

The second cycle supplied further observations to each role's saved notes and again proceeded through exchange, old notices, and readout. Both roles' observations changed so that the correct joint channel reversed relative to the preceding cycle. Customer information was retained, and customer and shared information were used together in the final response.

The customer-update diagnostic used eight cases combining initially matching or different preferences, target role A or B, and an initial dog or cat preference for the target. Both roles were told about one customer's new preference and later received that customer's old preference. The target role should update its own customer information, while the other role should retain information about its own customer. Customer revisions and observation revisions were managed separately. No deterministic latest-revision selection was applied to customer information; the existing storage controls maintained observation fields.

4.4 Scoring, aggregation, and stopping rules

Fixed Python code for each trial evaluated persons, customer preferences, observation values, origin IDs, revisions, and subsequent responses. Semantic scoring did not use a separate LLM judge. Alongside re-aggregation of raw responses, a separate implementation checked the correspondence among inputs, selected source rows, saved artifacts, and subsequent inputs.

Storage success required every field to match the expected state given the information available at that stage. Immediately after an own-observation update, the role was not required to possess its partner's new information before receiving it. Readout was evaluated for both fidelity to actual notes and correctness under the latest state. Origin IDs and revisions were evaluated during updating and storage; later simplified readouts did not request their re-citation.

End-to-end success in continued-use trials required all necessary storage and readout steps for both roles in one group–repetition trajectory to succeed. Selective use required success for both roles under all four conditions. The descriptive criterion for each of the first-cycle, selective-use, second-cycle, and customer-update trials was at least 14 of 16 trajectories, with at least seven of eight in each repetition. The storage comparison fixed criteria of at least 18/20 for new notices and 14/16 for old notices, alongside repetition-specific and customer-retention conditions. These were local criteria for small trials, not thresholds for statistical significance.

Structurally acceptable semantic errors were logged and passed onward. When an artifact could not be saved, dependent responses were treated as missing, with known outcomes distinguished from unresolved ones. All seven main trials reached the planned semantic evaluation, with no unresolved outcomes. Appendix A retains the original treatment of structural missingness and post hoc reference analyses in the transfer and earlier update diagnostics.

Analyses emphasize paired counts within fixed materials, component outcomes, and repetition-specific results. As planned for the individual trials, no p-values or population confidence intervals are calculated. Four branches of a common proposal, repetitions of the same material, reuse of a note, roles, and stages are not added as

independent samples. Before execution, the customer-update diagnostic was designated to close after one fixed run regardless of outcome, followed by inspection of the raw records and preparation of this report.

4.5 Execution environment and structural validation

The seven main trials ran on Google Colab on September 25–26, 2026. The open-weight Qwen/Qwen3.5-27B model was loaded at a fixed revision and used with BF16, SDPA, and non-thinking mode. Generation settings were temperature 0.7, top-p 0.8, top-k 20, repetition penalty 1.0, a maximum input of 8,192 tokens, and a maximum output of 2,048 tokens. Seeds were recorded for materials and repetitions. The GPU was an NVIDIA RTX PRO 6000 Blackwell Server Edition. Appendix B lists the model revision and library versions. Official model card.

Readout decomposition and saved-note readout accepted bare JSON. Starting with the field-level storage comparison, a rule for removing only complete JSON code fences observed in the preceding diagnostic was incorporated before generation. The outer wrapper, JSON structure, and semantic correctness were treated as separate stages, retaining both original and normalized text. Selection and copying of observation rows were storage operations separate from wrapper normalization.

5. Results

5.1 Fidelity to records and agreement with the latest state diverged

Separating observation extraction from channel computation supported faithful use of both researcher-prepared notes and notes actually saved by the model. Because the same readout path also used stale notes accurately, **readout fidelity could be observed separately from the freshness of the stored state.**

In **readout decomposition (RS01)**, extraction plus computation produced the correct channel in all 24 main responses and all eight boundary responses. The four shared values also matched the input in all 32 responses: there were no cases in which an extraction error happened to yield the correct channel.

Table 6. Readout comparisons within the same materials. Entries are correct channel decisions divided by responses. Main materials have a unique paper or digital answer.

Trial and materials	Direct model decision	Extraction plus code computation	Model decision from correct four values
Readout decomposition: 12 main materials × 2 repetitions	11/24	**24/24**	15/24
Same trial: neither channel qualifies	4/4	4/4	4/4
Same trial: both channels qualify	2/4	4/4	4/4
Saved-note readout: 16 current inputs × 2 repetitions	11/32	**32/32**	Not run

In the main paired comparison, both extraction plus computation and direct decision were correct in 11 cases, only extraction plus computation was correct in 13, and only direct decision was correct in zero. Model decisions supplied with the correct four values scored 15/24. On materials whose correct answer was paper, this condition produced fewer correct responses than direct decision and did not meet its fixed improvement criterion.

In **saved-note readout (SVR01)**, extraction matched all four shared values in all 52 responses: 32 from current notes and 20 from stale notes. For the current-note comparison, both conditions were correct in 11 cases, only extraction plus computation was correct in 21, and only direct decision was correct in zero.

For stale notes, the channel derived by extraction plus computation scored 20/20 against the record but 4/20 against the latest information. The materials and rules established this discrepancy before execution: faithful

computation from stale values would produce it. What the trial established empirically was that the values were actually extracted correctly from every supplied note. This distinction motivated separate evaluation of updating/storage and readout in subsequent diagnostics.

5.2 Correct joint answers coexisted with defects in private records

Controlling only shared observations produced correct joint guidance in every case while leaving stale own observations. Controlling only own observations repaired those stale records but left the number of end-to-end successes unchanged because shared-field defects overlapped. The **field-level storage comparison (OR01)** branched the same 36 proposals into four conditions, exposing both defects behind correct answers and repairs invisible in end-to-end outcomes.

Table 7. Control by record field and remaining defects. The denominator is the same 36 proposals from 18 previously used materials with two repetitions each. “Own only” refers to the own-observation field.

Storage control	Own observations correct	Shared observations correct	Joint guidance agrees with latest state	End-to-end success
Neither field	32/36	25/36	28/36	25/36
Shared only	32/36	**36/36**	**36/36**	32/36
Own only	**36/36**	25/36	28/36	25/36
Both	**36/36**	**36/36**	**36/36**	**36/36**

All 72 shared rows in the original proposals exactly matched rows present in the input, but 11 were not the latest revision for the relevant person. These defects involved selecting existing old rows. Customer information was correct in 36/36 proposals and retained under every branch. All 55 actual readout responses were faithful to the four shared values supplied to them.

With shared-only control, four own observations remained stale despite correct guidance in every case. For example, when an old revision-1 notice arrived while B’s private note held revision 4, the proposal reverted both B’s own-observation field and the shared B row to revision 1. Controlling only shared observations restored the shared B row to revision 4 and supported correct guidance, but left revision 1 in the own-observation field. Controlling both brought the two fields into agreement with the latest revision.

Own-only control repaired all four own-observation defects, yet shared-field defects remained in the same four proposals. End-to-end success therefore stayed at 25/36, as with neither field controlled. This is a **local repair hidden by end-to-end outcomes**, complementing the defects hidden by correct answers under shared-only control. Together with success in all 36 proposals when both fields were controlled, these results motivate reporting component outcomes alongside end-to-end outcomes.

5.3 Observation controls supported two cycles and selective use

Saving both own and shared observations by selecting the latest revision for each person supported two cycles of updates and exchanges using actual saved artifacts, as well as selective use of customer and shared information. Even along these successful trajectories, model proposals contained defects that storage control replaced. Table 8 links **proposals before storage to the records actually passed onward**.

Table 8. Model proposals and actual saved states along continued-use trajectories. Saved notes after observation control become subsequent model inputs. Both cycles involve the same eight groups with two repetitions each.

Series	Updates with defective proposals	Mismatched observation rows	All fields match after storage	End-to-end success for both roles
First cycle	17/96	24 rows	96/96	16/16
Second cycle	23/96	32 rows	96/96	16/16

Note: A mismatched observation row is counted at each record field in each update, treating observer, paper/digital values, origin ID, and revision as one record. Multiple mismatched items in the same row count as one row; copies in own and shared fields are counted separately.

The first cycle's 24 mismatched rows comprised 19 that exactly matched old rows and five with an incorrect paper value despite correct revisions, origin IDs, and other items. All 32 mismatched rows in the second cycle were substitutions with old rows for the same person: nine in the own-observation field and 23 in shared observations. In particular, rollback appeared in 15/32 proposals at the old-notice stage of the second cycle. Storage control selected the latest input rows and replaced these mismatches before they passed onward. This table compares proposals and saved states within actual controlled trajectories; it is distinct from outcomes of trajectories run without control (Section 7).

In the **first cycle (SX01) and second cycle (SX02)**, customer information was retained in 96/96 updates in each cycle. End-to-end success for both roles was 16/16 trajectories in each cycle, or 8/8 in each repetition, meeting the fixed criteria. The first cycle began with eight new groups, and all 32 final shared readouts succeeded. The second cycle began with those actual saved notes, and all 32 final responses correctly extracted customer information and the four shared values. Joint guidance reversed correctly from the previous cycle in all 16 trajectories, while customer designs were retained.

The **selective-use diagnostic (RU01)** manipulated the first cycle's saved artifacts and correctly used customer and shared information in all 128 responses across four conditions. Table 9 shows the first group in execution order.

Table 9. Input manipulations of the same saved artifacts and selective use for two tasks. One group in a woodworking consultation scenario (RU01-C07-R1). Joint guidance was computed by code from the actual extracted values.

Input condition	Design for A's customer	Design for B's customer	Joint guidance
Unchanged	Dog	Cat	Paper
Preference changed only	Cat	Dog	Paper
Observations changed only	Dog	Cat	Digital
Both changed	Cat	Dog	Digital

Selective changes were confirmed in 64 paired customer-preference comparisons and 64 paired observation comparisons. End-to-end success across both roles and all four conditions was 16/16. These sets of 64 comparisons are diagnostics constructed from the same 128 responses. Figure 2 distinguishes the manipulated notes from the notes actually supplied to the second cycle.

5.4 Adoption of new information and post-update maintenance diverged

Whereas the two successful cycles held customer preferences fixed, the **customer-update diagnostic (CU01)** evaluated maintenance after legitimate changes to that information. New preferences were adopted and non-target information was retained in every case, but rollback remained after old notices. This separated the supported scope of retaining fixed information from that of preserving a change.

The target role adopted the new preference in 16/16 cases. The non-target role retained its own customer's information in 32/32 updates across new and old notices. The target role retained the new revision after an old notice in only 2/16 cases.

Table 10. From new customer notices to subsequent use. Eight cases with two repetitions each. Exact customer-field matching during updating includes the assigned role, preference, origin ID, and customer revision.

Stage or evaluation target	Success	Meaning of denominator
Target role adopts the new preference	**16/16**	Target role; one new notice per group–repetition trajectory
Non-target role retains its customer information	**32/32**	Non-target role; new and old notices
Target role retains the new revision after an old notice	**2/16**	Target role; one old notice per group–repetition trajectory
Readout is faithful to saved notes	**32/32**	Final responses from both roles
Latest customer and shared information are used together	**18/32**	16 non-target responses plus 2 target responses retaining the new revision
End-to-end success for both roles	**2/16**	New notices, old notices, and readout for both roles

In all 14 rollbacks, the correct new revision had just been saved and was included in the next input. The proposal and actual saved state after the old notice exactly matched the initial row, including preference, origin ID, and revision. The subsequent response used that old preference. The correspondence illustrated in Section 3.4 was verified for all 14 cases.

Shared observations were saved correctly in 64/64 updates, and final joint guidance was correct in all 32 responses. Two proposals at the new-notice stage contained four incorrect paper-value entries across own and shared observations; the observation storage control copied the correct input rows. That same control was not applied to the 14 customer-information rollbacks.

End-to-end success was 1/8 in each repetition, below the fixed criterion of 14/16 overall and 7/8 per repetition. The two successful trajectories were repetitions of the same case. There was no missingness or structural failure: all failures involved semantic mismatches in the post-update record and its use.

6. Discussion

6.1 Faithful use of a record and correctness of that record

Readout decomposition supported a configuration that extracted four values from actual records and applied an explicit rule. Saved-note readout was also faithful to stale records, making the difference between record-based and latest-information scoring observable. In the customer-update diagnostic, this difference appeared along a generation–storage–use chain: a new preference was adopted, the saved content later rolled back, and readout correctly inherited that state.

A-TMA and STALE also separate state from its use. Our customer-update diagnostic characterizes the path in which storage reverts to an old state and a subsequent answer faithfully follows it. Evaluating continued use therefore benefits from inspecting both whether the saved state was appropriate and whether that state was used faithfully, alongside final-answer correctness. Establishing readout fidelity alone can help localize whether an error arose at or before storage or during readout.

6.2 Specify the record fields covered by each control

The possibility that local repair does not change end-to-end success follows from the metric's requirement that all conditions hold. What this comparison established empirically was that the four own-field defects in actual proposals overlapped with shared-field defects, so repairing the own field of those same proposals left end-

to-end outcomes unchanged. Conversely, shared-only control produced correct guidance alongside stale own information. Linking field-specific states and subsequent use located the concrete defects and repairs hidden by the two aggregate measures.

The division of candidate processing between model and code has precedents, including Reddy and Challaram's freshness-selection method. Our comparison varied its application to own and shared observation fields. Latest-revision selection by person supports selecting an appropriate input row when the target person and revision are explicit, and saving that row unchanged. Under a contract with correct candidates and comparable revisions, matching the copied fields is supported by the properties of the code. The experiments examined outcomes when this operation was connected to model proposals, customer information outside its scope, and subsequent use of actual saved artifacts.

The label "latest-revision selection" alone does not specify whether a control applies to own observations, shared observations, or customer information. Mapping model-written fields, copied fields, and fields used downstream, as in Table 3, makes it possible to describe the supported scope and defects outside it.

6.3 Retention, change, and post-update maintenance are distinct requirements

The two continued-use cycles established a path that retained fixed customer preferences while incorporating new observations into joint guidance. Selective use showed that manipulations of customer preferences and shared observations affected their respective tasks. The customer-update diagnostic added a different requirement—a legitimate change to customer information itself—and found a separation between adopting new information or retaining non-target information and maintaining the updated state after an old notice.

Success across two cycles and failure to meet the customer-update criterion are compatible outcomes for different requirements. Separating the scope of successful retention, successful adoption, and defective post-update maintenance makes the supported conditions of the division of responsibilities explicit.

In this task, preserving private information means following legitimate information attributed to each role and updating that role's information when it changes. In Section 3.4, this correctly results in both roles providing a dog design. Attribution and agreement with the latest state, rather than answer similarity or difference, are the evaluation criteria.

6.4 Applicability conditions and untested situations

The observation controls that succeeded here operate under an input contract with explicit target persons, revisions, and candidate rows. Assessing the effectiveness of applying person/revision checks to customer information would require a comparison with and without control for that same field and the same materials. Deferral and re-confirmation for unidentified targets, missing revisions, or conflicting same-revision records also lie outside the current evaluation. Extending the task to historical states or conditional coexistence, as addressed by Long et al.'s MemTrace and MemConflict, would require specifying the time and conditions requested by a query rather than selecting only the maximum revision. The observed boundary is that a configuration supporting observation fields sustained two cycles of use, while rollback remained in the maintenance of changed customer information.

7. Scope and Limitations

Model and task. The main results concern a single Qwen3.5-27B revision, non-thinking mode, fixed generation settings, and short synthetic tasks in Japanese. Transfer from the first report changed both the model and the inference configuration and runtime. Dog/cat preferences are explicitly assigned facts, and shared state consists

of structured observations from two roles. We did not measure personality formation, spontaneous consensus, or long-term coexistence through free conversation.

Materials and repetitions. The small trials combine diagnostics on previously used materials with tests using new materials in an adaptive exploration. Two repetitions partially characterize variation in execution but do not increase the number of materials. The first cycle, selective-use diagnostic, and second cycle depend on the same eight groups; the customer-update diagnostic also contains only eight groups. Success in all observed cases does not establish reliability on unknown tasks. Instructions, schemas, and output requirements differ across conditions, so the readout comparisons concern complete methods.

Interventions and mechanisms. The storage comparison branches the same proposal by controlled field. In contrast, the two continued-use cycles employ one configuration controlling both observation fields; proposal-defect rates are not outcomes of a multi-stage series run with control disabled. Customer and observation fields in the customer-update diagnostic also differ in content and contract. Their difference cannot be treated as a causal effect estimate without comparing control on and off for the same field. We did not experimentally decompose old-notice adoption into mechanisms such as presentation position, revision comparison, or internal attention.

Input contract. Latest-revision selection depends on rows actually present in the input and on comparable revisions. The implementation detects conditions such as conflicting candidates for the same person and revision, but the model trials used consistent candidates. Whether the maximum revision is correct in the real world, whether the sender is authorized to change the information, and how to seek clarification for an unidentified target require separate evaluation.

Scoring and verification. Semantic scoring used fixed code, and raw responses and saved artifacts were checked with a separate implementation. Design, implementation, scoring, verification, and writing occurred within the same project, rather than through independent blinded rescoring. Simplified readout evaluates the assigned role, dog/cat preference, and four shared values; it is not the same measure as the first report's strict evaluation, which also requires re-citation of origin IDs and current evidence references.

8. Conclusion

This study evaluated private and shared LLM records by **linking the fields and update stages supported by control to actual saved artifacts and subsequent use**. Comparisons that varied storage control for the same update proposal revealed both private-record defects accompanying correct joint answers and local repairs invisible in end-to-end outcomes. Separating readout fidelity, record freshness, and task correctness located the respective defects.

A configuration saving own and shared observations at the latest revision for each person supported retention of fixed customer information and use of new shared observations through a second cycle. In a separate diagnostic that allowed customer preferences to change, old notices caused rollback after the new preference had been adopted, and subsequent responses were faithful to the resulting record. Retention, adoption of new information, and post-update maintenance had different supported scopes in this configuration.

The result within this fixed task is an evaluation that specifies success by controlled field, update stage, and downstream use. Reporting local storage outcomes alongside system-wide outcomes helps characterize the applicability of designs that continue to use private information and shared state.

9. Data Availability and Declarations

This English edition is based on the finalized Japanese version. The materials consist of fictional customer preferences and venue observations; this trial series uses no real customer information or personal social-media

posts. Raw responses, inputs, actual saved artifacts, scoring code, environment records, and hash lists are retained locally. Supplement S1 maps the main results to their evidence. These research materials are retained without a deletion deadline under the trials' storage policies.

The MRSI repository distributes the manuscript and evidence under Apache 2.0, with current reproduction instructions. A permanent archive DOI for this paper has not been assigned. Links in the PDF point to versioned GitHub resources; the Markdown edition uses relative repository paths.

The author determined the research direction, authorized execution, ran the model in Colab, and reviewed the manuscript. LLM assistance was used for design documents, implementation, deterministic scorers, offline verification, and writing. Funding, conflicts of interest, and author-contribution declarations for public release remain subject to the author's final confirmation.

References

1. Sato, K. (2026). *Maintaining Role-Specific Information and Shared Records in LLM Memory Updates: An Exploratory Evaluation of Content, Provenance, and Continued Use*. English manuscript v0.2, corresponding to Japanese v0.11. Unpublished. Local manuscript.
2. Packer, C., Wooders, S., Lin, K., Fang, V., Patil, S. G., Stoica, I., & Gonzalez, J. E. (2024). *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560v2 (first version, 2023). Primary source.
3. Rezazadeh, A., Li, Z., Lou, A., Zhao, Y., Wei, W., & Bao, Y. (2025). *Collaborative Memory: Multi-User Memory Sharing in LLM Agents with Dynamic Access Control*. arXiv:2505.18279v1. Primary source.
4. Moreau, L., & Missier, P. (Eds.). (2013). *PROV-DM: The PROV Data Model*. W3C Recommendation. Primary source.
5. Debenedetti, E., Shumailov, I., Fan, T., Hayes, J., Carlini, N., Fabian, D., Kern, C., Shi, C., Terzis, A., & Tramèr, F. (2025). *Defeating Prompt Injections by Design*. arXiv:2503.18813v2. Primary source.
6. Qwen Team. (2026). *Qwen3.5-27B model card*. The model revision used in the experiments is listed in Appendix B. Official source.
7. Wu, D., et al. (2025). *LongMemEval: Benchmarking Chat Assistants on Long-Term Interactive Memory*. arXiv:2410.10813v2 (first version, 2024). Primary source.
8. Hu, Y., Wang, Y., & McAuley, J. (2025). *Evaluating Memory in LLM Agents via Incremental Multi-Turn Interactions*. arXiv:2507.05257v1. Primary source.
9. Patel, V. (2026). *Supersede: Diagnosing and Training the Memory-Update Gap in LLM Agents*. arXiv:2606.27472v1. Primary source.
10. Chao, H., et al. (2026). *STALE: Can LLM Agents Know When Their Memories Are No Longer Valid?*. arXiv:2605.06527v1. Primary source.
11. Rasmussen, P., et al. (2025). *Zep: A Temporal Knowledge Graph Architecture for Agent Memory*. arXiv:2501.13956v1. Primary source.
12. Tao, Z., et al. (2026). *MemConflict: Evaluating Long-Term Memory Systems Under Memory Conflicts*. arXiv:2605.20926v1. Primary source.
13. Chen, D., et al. (2025). *HaluMem: Evaluating Hallucinations in Memory Systems of Agents*. arXiv:2511.03506v1. Primary source.
14. Shi, Z., Tang, Y., & Tung, A. K. H. (2026). *A-TMA: Decoupling State-Aware Memory Failures in Long-Term Agent Memory*. arXiv:2607.01935v1. Primary source.
15. Ma, E., et al. (2026). *MemAudit: Auditing Long-Term Agent Memory via Hidden User-State Recovery*. arXiv:2606.24595v2 (v1 was titled MemProbe). Primary source.
16. Deng, X., et al. (2026). *MemTrace: Tracing and Attributing Errors in Large Language Model Memory Systems*. arXiv:2605.28732v1. Primary source.
17. Long, X., et al. (2026). *MemTrace: Probing What Final Accuracy Misses in Long-Term Memory*. arXiv:2606.17328v1. Primary source.
18. Yang, J., et al. (2026). *GroupMemBench: Benchmarking LLM Agent Memory in Multi-Party Conversations*. arXiv:2605.14498v2. Primary source.
19. Reddy, V., & Challaram, S. (2026). *Don't Ask the LLM to Track Freshness: A Deterministic Recipe for Memory Conflict Resolution*. arXiv:2606.01435v1. Primary source.
20. Sun, H., & He, L. (2026). *When Memory Updates but Behavior Does Not: Repairing Implicit Stale Dependencies in Personalized Agent Responses*. arXiv:2608.01619v1. Primary source.

Appendix A. Entry Points and Supplementary Analyses Preserving Formal Outcomes

A.1 From model transfer to readout

The transfer trial (XM01) moved the first report's transcription-method comparison to Qwen using the same 64 materials, each repeated three times. Of 1,536 planned responses across two methods, two roles, and note creation/subsequent decision stages, 1,008 were obtained. The remaining 528 were dependent missing responses because parent notes did not pass the storage contract.

Across 192 group–repetition units per method, rewriting had zero successes, 185 failures, and seven unresolved outcomes; selection plus copying had zero successes, 187 failures, and five unresolved outcomes. Among the 240 subsequent responses that could be connected, all dog/cat decisions were correct, but only 30 guidance decisions were correct, and none passed strict evaluation including provenance and current evidence references. Of 230 responses whose shared notes matched the latest state, 171 deferred guidance. Storage defects alone therefore could not explain readout defects. Transfer results, Japanese.

An output-load diagnostic (RL01) then held correct notes fixed. Across 12 main materials with two repetitions each, a condition requesting dog/cat, guidance, and evidence scored 3/24; removing evidence requests scored 7/24; requesting only joint guidance also scored 7/24. Reducing output requirements did not meet the fixed criterion, leading to the readout decomposition in the main text. Output-load results, Japanese.

A.2 Separating JSON wrappers from update content

An earlier update diagnostic (UR01), using extraction plus computation for readout, obtained 72 update responses from 18 materials, two repetitions, and two methods. Of these, 37 wrapped JSON in code fences and were rejected under the format rules then in force; their 37 subsequent responses were not generated. The 35 accepted notes were read faithfully in 35/35 cases, but shared observations matched the latest state in 24/35, and guidance matched the latest state in 25/35. Formal results, Japanese.

A separate post hoc analysis removed only complete outer wrappers. Of those 37 responses, 27 then matched the required update content. Combined with the originally accepted responses, update content matched in 51/72; the remaining 21 contained stale shared rows. Three own-observation defects were also present, including one that placed the partner's observation in the own-observation field. This analysis did not fill in unperformed subsequent responses or change formal end-to-end outcomes. Post hoc reference analysis, Japanese.

Starting with the subsequent field-level storage comparison, this restricted wrapper handling was fixed before generation and applied consistently. A readable wrapper, correct attribution of an observation, and selection of the appropriate revision were evaluated separately.

Appendix B. Implementation and Aggregation Details

B.1 Execution settings

The model revision was `fc05daec18b0a78c049392ed2e771dde82bdf654`. The seven main trials used Python 3.13.15, PyTorch 2.11.0+cu128, Transformers 5.17.0, and Accelerate 1.15.0. Only the first half of the transfer trial used an A100-SXM4-80GB; its second half and the subsequent main trials used an RTX PRO 6000 Blackwell Server Edition. Generation seeds, actual prompts, finish reasons, token counts, and environments were recorded with responses.

Each call was text generation without tool operations. Python handled note delivery, storage, exchange, structural validation, and scoring. Normalization handles only a complete pair of `json` code fences around a response that

ends with EOS. It does not repair internal keys, values, IDs, or types. Structural defects such as duplicate keys, and conflicting candidates for the same person and revision, are checked separately.

B.2 Response accounting for the main trials

Trial	Update responses	Readout or other responses	Total	Outer-fence removals
Readout decomposition	0	96	96	0
Saved-note readout	0	104	104	0
Field-level storage comparison	36	55 (shared across 144 condition instances)	91	20
First cycle	96	32	128	54
Selective use	0	128	128	125
Second cycle	96	32	128	79
Customer-update diagnostic	64	32	96	75

All actual responses in this table ended with EOS and passed structural validation. This establishes completion of the format-processing path, which is counted separately from semantic or end-to-end success. Materials and contracts differ among trials, so no pooled success rate is calculated from this table.

B.3 Evidence access

Supplement S1 links manuscript tables and the worked example to result reports, fixed re-aggregations, original ZIP archives, raw-response correspondences, and storage/readout code. Readout decomposition, field-level comparison, continued use, and customer-update outcomes can be traced at their original aggregation units. Supplement S1: Evidence index.

The continuation in Figure 2 passes the first cycle's actual saved artifacts to the second cycle. Selective use creates four branches from those saved artifacts; the customer-update diagnostic starts from separate new initial states. Expected states used for scoring and the actual artifacts supplied to subsequent calls are retained in separate files.

Separating the Stability of Role-Specific Judgments from the Quality of Joint Agreements

Timing of Computational Support and Fixed-Input Repetitions in a Two-Role LLM Task

Kentaro Sato
Independent Researcher
Contact: info@metaclan.jp (email)

English version v1.0

Repository and research materials | Apache License 2.0

Abstract

For LLM roles to collaborate while maintaining judgments grounded in their own experience, role-specific recommendations, joint agreement quality, and faithful execution must be evaluated separately. We ran the same Qwen3.5-27B model as two roles in separate contexts. Each role recommended a guidance medium for its own area, while the joint task allocated limited equipment between the roles. Generated role-specific and joint records were passed to subsequent inputs to trace intermediate judgments and final use.

We compared supplying a computed summary of each role's experience only at final use with supplying it from judgment formation onward. In two trials, using previously used and previously unused numerical materials, respectively, role-specific correctness throughout the pathway increased from 15/24 to 24/24 pathways. Final use met its criteria in all cases under both conditions, concealing the intermediate difference. In one pathway with new materials, a lower-total joint allocation was accepted despite correct role-specific judgments and supplied candidate totals. The resulting agreement was then used faithfully.

We subsequently repeated the actual inputs from the same eight materials three times in separate judgment-formation and joint-confirmation panels. All 16 formation inputs passed in every repetition with summaries; without summaries, 13, 13, and 11 passed. The same incorrect joint acceptance also recurred in all three fresh repetitions. In another case, the defective field changed across repetitions even though the combined outcome failed each time. These small comparisons locate the stages supported by computational summaries and distinguish remaining defects in joint approval from fidelity in subsequent use.

Keywords: LLM agents; collaboration; role-specific judgments; joint agreements; computational support; stability across repetitions

1. Introduction

Consider two guidance staff members who have tried paper and digital guidance in their respective areas. For their own area alone, each can recommend the medium that performed better in past observations. At a joint event, however, only one paper guidance station and one digital terminal are available. Assigning one role a medium different from its own recommendation may produce the best combined outcome.

An agreement to use paper in the joint task does not imply that paper worked better in that role's own experience. Both judgments can be maintained in accordance with their respective purposes and evidence. Likewise, both roles following the same allocation does not establish that accepting that allocation was appropriate.

Along actual execution paths, role-specific correctness, joint agreement quality, and faithful use of the agreement came apart. In fixed-input repetitions, role-specific judgments and joint responses differed in their persistence and variability; even when the combined outcome remained the same, the location of the defect could change. We measured these distinctions by storing experience-based recommendations and joint assignments in separate fields and passing the actual records to subsequent inputs.

The broader research program concerns design principles for forming and updating shared state while preserving what is specific to each agent. Here, we operationalize one part of that question: whether a role can maintain a recommendation grounded in its own experience without conflating it with its joint assignment. The exploratory series began with both retaining a judgment for unchanged experience and appropriately revising it when experience changed. Identical recommendations from the two roles are appropriate when each follows its own evidence.

Table 1. Three findings examined in this report.

Finding	Main evidence	Evaluation target
A correct final recommendation can coexist with incorrect judgments saved earlier	Two comparisons of summary-supply timing	Role-specific judgments and references from formation through use
Consistently correct role-specific judgments can coexist with repeated incorrect joint acceptance	An actual pathway using new materials and three fixed-input repetitions	Separate correctness of role-specific judgments and joint responses
The same combined outcome can conceal different defect locations and downstream consequences	Repetitions with changing defective components; faithful use of an incorrect agreement	Fields within each response and correspondence between storage and use

These findings organize an adaptive exploration; they were not hypotheses fixed before the entire series. Before each main trial, we fixed its materials, conditions, scoring, number of responses, and stopping rules in local documents. This report organizes those fixed results around the three questions above.

2. Related Work

2.1 Evaluating individual objectives and joint outcomes

SOTOPIA evaluates interactions with assigned roles and social goals along multiple dimensions, including goal achievement and relationships [1]. NegotiationArena measures proposals, acceptance, and negotiation outcomes through resource exchange, allocation, and bargaining tasks [2]. These studies evaluate role-specific objectives and relationships without reducing interaction performance to the correctness of a single answer.

Our task elicits a numerically determined recommendation from each role's experience separately from its joint assignment. The joint task explicitly specifies a shared objective of maximizing the combined total. Maintaining

one's own recommendation is therefore distinct from prioritizing self-interest over collective benefit. Short response sequences restricted to acceptance, counterproposal, and deferral expose performance on the two tasks.

2.2 Stage-level evaluation and reliability across repetitions

MultiAgentBench evaluates milestone contributions, communication, and planning alongside final task performance [3]. AgentBoard's progress rate captures intermediate achievement that final success rates may not distinguish [4]. MAST classifies failures in multi-agent execution traces involving system design, inter-agent misalignment, and verification [5]. Stage-level evaluation and analysis of failure locations thus have established precedents.

Similarly, τ-bench evaluates consistent success across repeated runs of the same task using pass^k [6]. Our repetitions hold saved inputs fixed and examine how correctness of role-specific judgments and joint responses combines across three fresh responses. We focus on cases in which the combined outcome remains unsuccessful while the incorrect component switches between the role-specific and joint fields.

2.3 Dividing computation and judgment

PAL delegates execution of LLM-generated programs to an interpreter, separating language processing from computation [7]. In a negotiation setting, OG-Narrator uses a deterministic Offer Generator to set offer prices and an LLM to express them in natural language [8]. Moving computation or candidate construction into code also has precedents.

In our task, code produces own-experience summaries and totals for joint candidates. The LLM still selects a role-specific recommendation and responds to the joint proposal. Within this division of responsibilities, we measure intermediate records under different supply timings and approval errors that remain despite access to computed evidence.

Table 2. Units of evaluation in related work and in this study.

Related work	Main evaluation unit	Correspondence examined here
SOTOPIA; NegotiationArena	Social interaction; negotiation objectives and outcomes	Experience-based recommendations versus joint assignments
MultiAgentBench; AgentBoard; MAST	Stages of achievement; collaboration processes; execution failures	Field-level correctness, saved judgments, and subsequent use
τ-bench	Consistent success across repetitions of a task	Whether defect locations also remain stable across fixed-input repetitions
PAL; OG-Narrator	Division of computation or candidate construction and generation	Timing of computed evidence and approval decisions retained by the LLM

Our study combines **the timing of computational support, subsequent use of actual records, and fixed-input repetitions in a task that elicits role-specific judgments and joint responses together**. The literature review memo records the primary-source passages checked and the scope adopted in this manuscript; the memo is in Japanese.

3. Task: Separating a Role's Recommendation from Its Joint Assignment

3.1 Two judgments derived from experience

We execute the same model as guidance roles A and B. Each generation starts in a fresh context with the role, observations, and required saved records. Each role has observation rows from its own area for paper and digital

guidance. Each row records the observer, session, number of people trying each medium, number successfully guided, and experience ID.

For the role-specific task, both media are available. The role recommends the medium with the higher cumulative success rate across all observations, including earlier sessions. For the joint task, one paper station and one digital terminal must be allocated to maximize the combined projected number successfully guided, assuming 40 people in each area. The projected value for role i and medium m is:

$$q_i(m) = 40\frac{\sum_t s_{i,t,m}}{\sum_t n_{i,t,m}}$$

Here, s is the number of successes and n is the number of trials. A role-specific judgment compares only that role's q values. The joint judgment compares the sums for the two feasible allocations: A paper/B digital and A digital/B paper. These are projections defined within the task, not estimated intervention effects on real users.

The `local` response field contains the role's own recommendation and experience IDs. During confirmation, the `reply` field contains the action on the joint proposal, the A/B allocation, the record being answered, and both roles' experience IDs. During subsequent use, the `team` field contains either an allocation following the saved joint decision or a deferral, together with references to the relevant records. Fixed rules check observation ownership and references.

3.2 Worked example: Digital for one's own task, paper for the joint task

Table 3. Projected values for one previously unused material (ST02, N05).

Role	Paper	Digital	Correct role-specific recommendation
A	17	27	Digital
B	18	20	Digital

The joint total is 45 for A digital/B paper and 37 for A paper/B digital. An appropriate response from B therefore combines "I recommend digital for my own area" with "I will take paper in the joint allocation." Keeping these separate avoids rewriting the role's experience-based judgment to match its joint assignment.

In the observed responses for this material, the model sometimes accepted the 37-total allocation while correctly retaining its own recommendation. Section 5.2 traces that response through storage and use; Section 5.3 examines repeated responses to the same confirmation input.

4. Methods

4.1 Two execution-path comparisons and a fixed-input repetition study

The main text covers three trials: **summary-supply timing with previously used materials (ST01)**, **the same comparison with previously unused numerical materials (ST02)**, and **three fresh repetitions of actual saved inputs (JR01)**. Appendix A summarizes the preceding exploration.

In each timing comparison, both roles formed their judgments once per condition for each of eight materials. Each condition's 16 role-specific responses were reused across three types of initial joint proposal: optimal, feasible but suboptimal, and resource-violating. This produced 24 pathways per condition. Only if the confirming role counterproposed did the initiating role perform one recheck. Both roles subsequently used the resulting actual records.

ST01 reused all eight materials from an earlier trial. ST02 retained the eight strata defined by initiating role A/B, matching/differing role-specific recommendations, and a joint-total gap of 2/8, while replacing experience

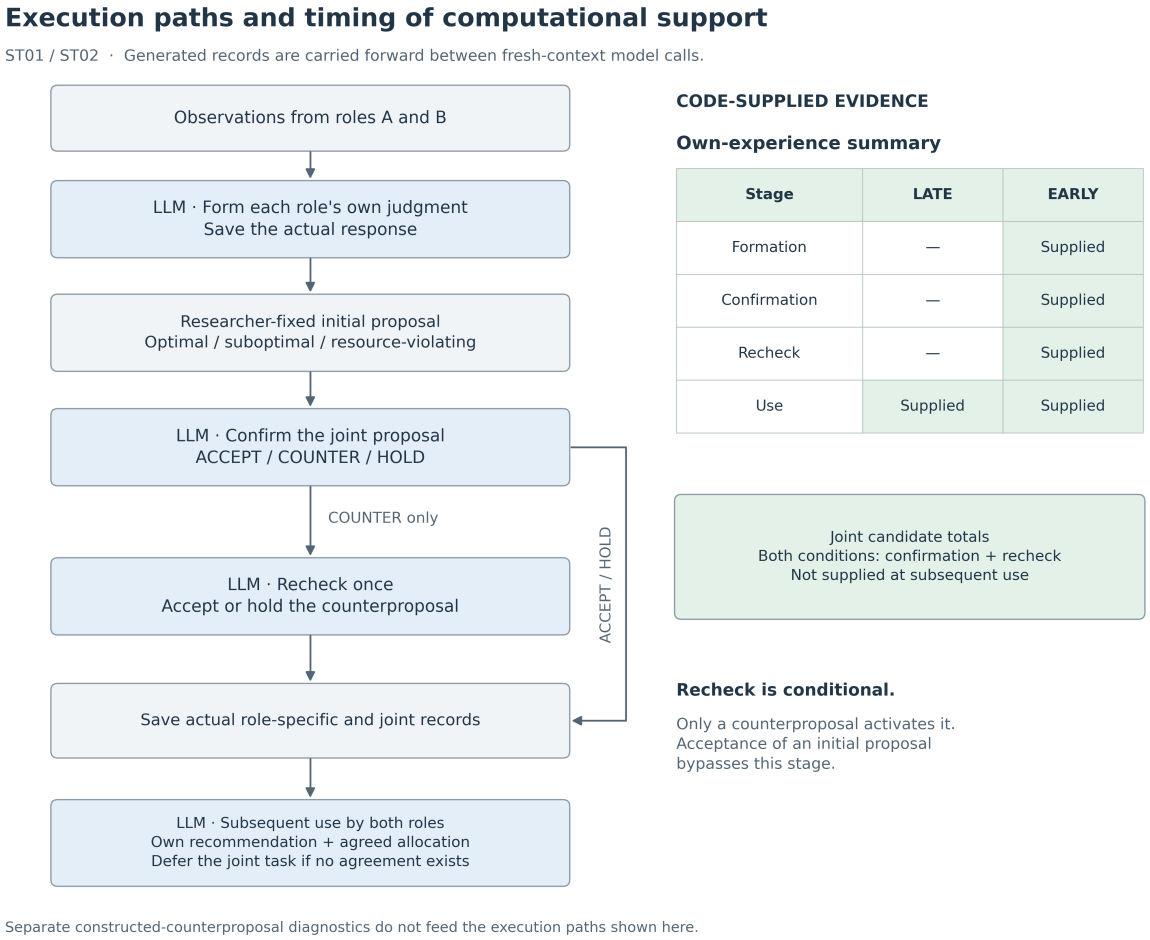


Figure 1. Execution paths in the supply-timing comparisons. Solid arrows show the progression of the task and the handoff of generated records. The schedule distinguishes computed support from model-generated responses. Both conditions receive an own-experience summary at subsequent use. Joint candidate totals are supplied at confirmation and recheck, not at use. The separate constructed-counterproposal diagnostic does not feed this pathway. Vector version.

values and IDs. Each role had observations from two sessions, and the initiating role had one additional session, giving five experience rows per material. Before generation, we checked experience arrays and the two roles' cumulative projections against saved inputs from the preceding 11 trials, treating role and medium permutations as equivalent. No duplicates were found. "Previously unused" refers to these numerical materials; the task format remained the same.

JR01 included all 32 formation inputs and 48 confirmation inputs from ST02, each repeated with three fresh generation seeds. **The two panels were not connected: confirmation always received the role-specific notes actually saved in ST02.** Incorrect old notes were retained. This differs from repeating the entire pathway with newly formed judgments passed to new confirmation calls.

4.2 Conditions for supplying computed evidence

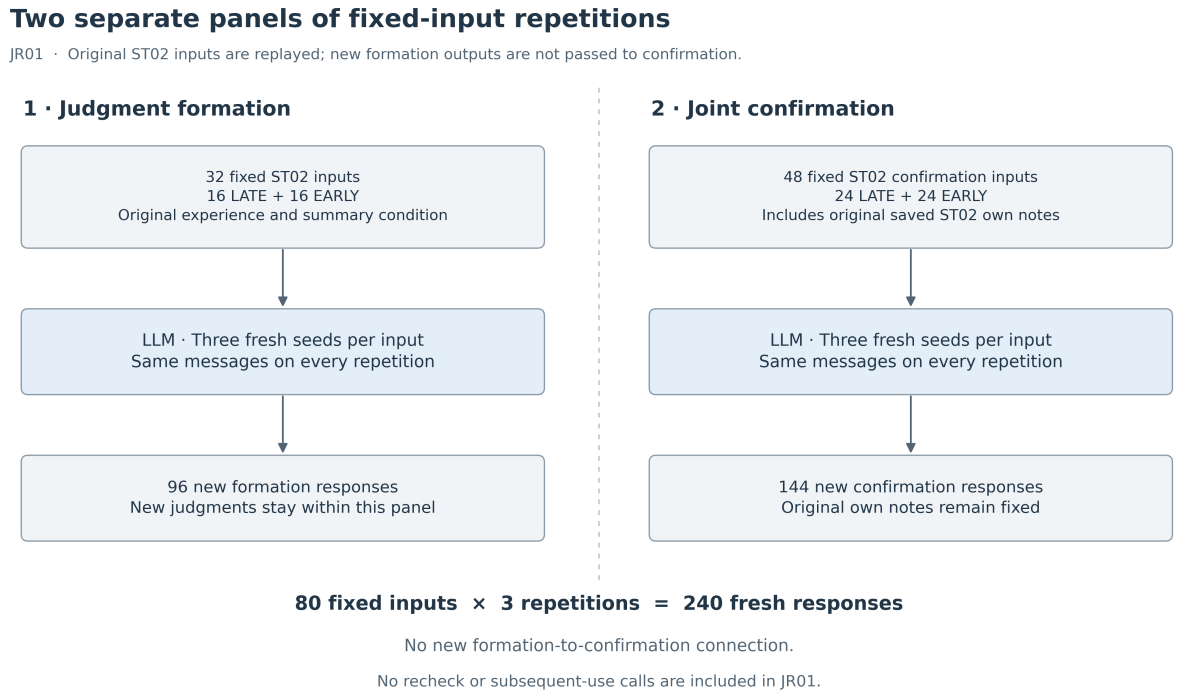


Figure 2. Separate fixed-input repetition panels in JR01. The formation and confirmation panels comprise 80 fixed inputs and 240 fresh responses in total. Newly generated role-specific judgments do not enter the confirmation panel. Confirmation repeatedly receives the original input, including the role-specific note saved in ST02. Recheck and subsequent use are outside this repetition study. Vector version.

Table 4. Timing of own-experience summary supply.

Stage	Supplied only at final use (LATE)	Supplied from formation onward (EARLY)
Judgment formation	All own experience	Same experience + own-experience summary
Joint confirmation and recheck	All experience + actual role-specific record + joint candidates and totals	Same types of evidence + own-experience summary
Subsequent use by both roles	All experience + actual role-specific record + actual joint records + own-experience summary	Same types of evidence

Code computes each own-experience summary from only the role's observations in the input. It contains projections for each medium, total trial counts, and experience IDs. It occupies a separate field from joint candidate totals and does not label a recommended medium. At formation, the conditions differ only in the presence of this summary field; instructions and output format are shared. Later stages carry forward the actual responses generated under each condition.

JR01 uses the same summary-field contrast for formation. At confirmation, both the summary and the old role-specific note can differ between conditions. We therefore treat confirmation as a conditional comparison that includes the original saved state. Repetition numbers, condition labels, and responses from other repetitions are not supplied to the model.

4.3 Defining appropriate acceptance, deferral, and use

At joint confirmation, a feasible initial proposal with the highest total should be accepted unchanged. For a suboptimal or resource-violating proposal, the role should either counterpropose the optimal feasible allocation or defer only the joint task. At recheck, an optimal counterproposal with appropriate allocation and references

should be accepted; otherwise it should be deferred. Further counterproposals are disallowed, closing the exchange after at most one counterproposal and recheck.

To separate agreement quality from fidelity of use, subsequent use was defined as carrying out an approved feasible agreement, or deferring only the joint task when no agreement existed. Execution is measured by a JSON answer specifying the allocation for the next task; no real equipment or users are operated. Re-optimization of the joint allocation is not requested at this stage. Role-specific judgments are evaluated against the experience available at each stage, so faithfully retaining an incorrect saved judgment does not count as correct.

Table 5. Evaluation units used in the main text.

Metric	Unit	Success criterion
Judgment formation	One role's formation response	Correct recommendation and appropriate references to its own experience
End-to-end role-specific correctness (I)	One material–initial-proposal pathway	Role-specific judgments and references are appropriate in both formation responses, confirmation, recheck when applicable, and both use responses
End-to-end correctness (J)	Same pathway	Both role-specific and joint responses, references, and use are appropriate at every applicable stage
Appropriate agreement (G)	Joint records at the end of the exchange	Agreement is reached on a feasible allocation with the highest total and appropriate required references
Subsequent use	One role's use response	Correct role-specific judgment and references, faithful execution of agreement or appropriate deferral without agreement, and appropriate joint references
Repeated joint confirmation	One response to one fixed input	Role-specific judgment and references (B), joint response and references (C), and both together (V), evaluated separately

Appropriate deferral can pass the joint-response criterion but does not count as agreement G. Because J evaluates the response contract at every stage, a pathway ending in appropriate deferral can also satisfy J. We therefore report J alongside agreement, deferral, and incorrect acceptance. A counterproposal likewise remains unapproved until the other role responds; we do not infer a final agreement rate from JR01's initial confirmation responses alone.

4.4 Generation, scoring, and record verification

We used Qwen/Qwen3.5-27B at fixed revision `fc05daec18b0a78c049392ed2e771dde82bdf654`, running on an RTX PRO 6000 Blackwell Server Edition in Colab. Settings were BF16, SDPA, non-thinking mode, batch size 1, temperature 0.7, top_p 0.8, top_k 20, and repetition penalty 1. Software versions were Transformers 5.17.0 and Accelerate 1.15.0. Input and output limits were 8,192 and 1,024 tokens, respectively. Each scheduled response was generated once.

JR01 assigned three fresh seeds to each fixed input. Corresponding conditions and the three initial-proposal types within the same material shared seeds. Conditions were executed adjacently within each repetition, with their order reversed in the second repetition. All repetitions ran within one GPU session.

Code fixed before execution performed formal scoring of structure, role identity, experience IDs, numerical role-specific judgments, joint allocations, response targets, and action contracts. A separate implementation, without importing the scoring module, checked rational arithmetic, references, record handoffs, and aggregates against raw responses and actual inputs. Selected original texts were examined through non-blinded, LLM-assisted reading within the same project. No new LLM-judge votes or open-ended semantic ratings entered the main-trial counts.

We retained raw responses, saved JSON, subsequent inputs, and hashes of parent records. Structurally readable but inappropriate responses were passed onward unchanged. Format defects or missing required parents were treated as unresolved, separately from substantive failures. In the three main trials, missing responses, format defects, normalization, and regeneration were all zero. The 235 responses in ST01 and 231 in ST02 each include 24 separate constructed-counterproposal rechecks. When no counterproposal was made, recheck was omitted by protocol and was not counted as missing. JR01 produced 240 responses.

4.5 Denominators and stopping rules

For each condition, the timing comparisons comprise 24 pathways branching from eight materials, and the repetition study comprises 16 formation inputs or 24 confirmation inputs. One formation response is reused across three pathways, so an increase in successful pathways is not an equal number of independent improvements at formation. Likewise, totals of 48 or 72 repeated responses describe the same inputs measured three times.

Before execution, ST01/ST02 fixed a directional prediction for the condition difference in I, and JR01 predicted a positive formation difference in all three repetitions. The missing-outcome rule was to report bounds obtained by assigning unresolved outcomes 0 or 1; no main-trial outcome was unresolved. Differences are descriptive results for fixed materials. We do not calculate population confidence intervals or p-values. The engineering progression criteria for ST01/ST02 combined several role-specific and joint requirements, listed in Appendix B. Meeting those criteria is reported separately from the direction of the comparison. Repetitions were not added in response to the results.

5. Results

5.1 Early summaries supported intermediate role-specific judgments, with the same direction across three repetitions

Table 6. Summary-supply timing and correctness from formation through use.

Metric	Used materials, ST01: LATE	ST01: EARLY	New materials, ST02: LATE	ST02: EARLY
Judgment formation	13/16	16/16	13/16	16/16
End-to-end role-specific correctness I	15/24	**24/24**	15/24	**24/24**
End-to-end role-specific and joint correctness J	14/24	23/24	15/24	23/24
Appropriate agreement G	22/24	23/24	24/24	23/24
Incorrect agreement	0/24	0/24	0/24	**1/24**
Subsequent use by both roles	48/48	48/48	48/48	48/48

Note: Subsequent use evaluates role-specific judgments and references, fidelity to an approved feasible agreement (or appropriate deferral without agreement), and joint references. Maximizing the joint total is evaluated separately by G.

In each trial, 15 pathways satisfied I under both conditions, while nine additional pathways satisfied I only with summaries supplied from formation onward. The difference was 9/24, or 37.5 percentage points. In each trial, the difference originated in three formation responses, each reused across three initial-proposal types. The role–material combinations producing these errors differed between the trials.

Under late supply, incorrect role-specific judgments sometimes remained in records passed to later stages. At final use, both conditions received own-experience summaries. Correct judgments were recovered in nine ST01

responses and four ST02 responses whose immediate input still contained an incorrect saved judgment. Thus, identical final-use scores of 48/48 concealed different intermediate records.

Repeating the fixed formation inputs from the new-material trial produced the following results.

Table 7. Three fresh-seed repetitions of 16 fixed formation inputs per condition (JR01).

Fresh repetition	Without summary	With summary	Paired difference in passes
1	13/16	**16/16**	+3
2	13/16	**16/16**	+3
3	11/16	**16/16**	+5

The predicted direction held in all three repetitions. The mean difference was 3.67/16, with a range of 3–5/16. Without summaries, 10 of the 16 inputs passed all three times, four had mixed outcomes, and two failed all three times. With summaries, all 16 passed all three times. Own-experience references were correct in every response; the difference concerned the selected medium.

For example, one input without a summary yielded paper in all three repetitions despite projections of 15 for paper and 18 for digital. Another input, with paper 20 and digital 19, yielded paper, digital, and digital. The summary comparison used all inputs, including both persistent and variable errors.

5.2 Correct role-specific judgments coexisted with a suboptimal joint agreement

Under early supply in the new-material trial, the material in Table 3 produced the only incorrect agreement. The actual input included joint candidate totals of 45 and 37, a correct role-specific note, and an own-experience summary. Nevertheless, B accepted the initial proposal with a total of 37.

Table 8. Actual saved records and subsequent responses (ST02, EARLY, N05, suboptimal initial proposal).

Stage	Role-specific field	Joint field	Evaluation
Confirmation by B	Digital	Accept A paper/B digital	Correct role-specific judgment; inappropriate joint response
Storage of joint records	Role-specific field retained	Feasible agreement totaling 37	Agreement reached; total not maximized
Subsequent use by A	Digital	Execute A paper/B digital	Correct role-specific judgment; faithful use of agreement
Subsequent use by B	Digital	Execute A paper/B digital	Correct role-specific judgment; faithful use of agreement

The following excerpt contains only judgment fields from the actual saved confirmation JSON. Reference fields are omitted for display; Supplement S1 links the complete response.

```
{
  "local": {"subject": "B", "choice": "DIGITAL"},
  "reply": {"action": "ACCEPT", "allocation": {"A": "PAPER", "B": "DIGITAL"}}
}
```

Because no counterproposal was made, recheck was not activated. Both roles subsequently used the 37-total agreement as recorded. Role-specific and joint reference IDs were also appropriate. Appropriate agreement at 23/24 can coexist with subsequent use at 48/48 because the latter scores fidelity to an approved agreement.

In contrast, the used-material trial included one inappropriate joint response in each condition that counterproposed a total of 37 in place of the optimal initial total of 39. The other role deferred at recheck, preventing an incorrect agreement. These examples show that a defective joint response can lead to different downstream paths depending on whether it is passed to recheck as a counterproposal or accepted immediately.

5.3 Repeating fixed inputs did not align role-specific and joint stability

Table 9. Three repetitions of 24 fixed confirmation inputs per condition (JR01).

Metric	Without summary: repetitions 1, 2, 3	With summary: repetitions 1, 2, 3
Role-specific judgment and references B	22/24, 23/24, 21/24	**24/24, 24/24, 24/24**
Joint response and references C	22/24, 23/24, 24/24	22/24, 23/24, 23/24
Both role-specific and joint criteria V	20/24, 22/24, 21/24	22/24, 23/24, 23/24

The old role-specific notes were held fixed in the confirmation inputs. Among the 24 inputs with summaries, 22 passed the joint criterion in all three repetitions, one had mixed outcomes, and one failed all three times. Without summaries, 22 passed all three times and two had mixed outcomes. Joint references were appropriate in all 144 responses.

Persistent incorrect acceptance. For the fixed input corresponding to Table 8, all three fresh responses correctly retained the digital recommendation but accepted the 37-total allocation. The JSON contents of the three responses were identical. These repetitions were part of the full set of 24 confirmation inputs included regardless of their earlier success or failure; they were not additional runs selected for this one input.

A response that varied across repetitions. For an optimal initial proposal in another material, only repetition 1 in each condition counterproposed a total of 46 instead of the optimal 48. Repetitions 2 and 3 accepted 48. The role-specific digital recommendation was correct all three times. Because corresponding conditions shared seeds, the matching pattern across conditions is not counted as two independent replications.

A failure whose components changed. For the same material as Table 3, without a summary and starting from a resource-violating proposal, the components changed as follows.

Table 10. Combined criterion V failed three times, but the defective component changed (JR01, LATE, N05).

Repetition	Role-specific judgment B	Joint response C	Both V
1	Recover digital: correct	Accept digital allocated to both roles: incorrect	Fail
2	Recover digital: correct	Accept the same resource-violating allocation: incorrect	Fail
3	Retain paper from the old note: incorrect	Counterpropose the feasible optimal allocation totaling 45: correct	Fail

Counting only three failures obscures that role-specific recovery and joint correction did not occur together in the same response. Relating the combined outcome to the fields and to the accept/counterpropose actions exposed this variation.

6. Discussion

6.1 Support timing affects the quality of intermediate saved judgments

The two execution-path comparisons and the three formation repetitions showed the same direction: supplying own-experience summaries early supported more correct role-specific judgments. In the pathway comparisons, both conditions gave correct final-use responses, yet the intermediate saved judgments differed. When one stage uses another stage's role-specific record, those intermediate states are meaningful evaluation targets alongside the final answer.

This effect was observed with explicit summaries computed separately for each experience owner. The LLM still selected the answer and generated its references from those summaries. The timing comparison evaluates reconstructing and saving role-specific judgments from evidence available at each stage, rather than maintaining information internally over long periods.

6.2 Approving an agreement and following it are different tasks

A correctly supplied table of joint totals did not guarantee an appropriate acceptance decision. In Table 8, storage and use were correctly connected after the incorrect acceptance. Improving downstream fidelity alone would therefore not necessarily recover agreement quality along this pathway.

Judging appropriately at recheck and activating that stage are also separate evaluation targets. The protocol invokes recheck only after a counterproposal; acceptance of the initial proposal bypasses it. The suboptimal acceptance in Table 8 proceeded to storage and use without recheck, exactly as the protocol specified. Locating a check for incorrect acceptance requires examining the branches that reach the checking stage as well as the checking role's responses.

6.3 Specify what remains stable

Three failures establish that the combined outcome was unchanged. An AND-defined criterion can conceal defect locations by construction. Beyond that logical property, we observed the defective component switch between the role-specific and joint fields across three responses to the same saved input.

Conversely, three correct role-specific judgments did not imply three correct joint responses. Combining fixed-stage repetitions with pathways that connect actual saved artifacts distinguishes which input–response relationships varied from what a response enabled downstream.

7. Scope and Limitations

Materials and repetitions. The main results involve one model, eight previously used materials, eight new numerical materials with the same task format, and a short exchange with at most one counterproposal and recheck. JR01 reused the latter materials and ran three seeds in one GPU session. Inputs, branches, and repetitions are dependent; three successes or failures do not establish population probabilities. An earlier own-summary comparison had 32/32 correct formation responses in both conditions, so summary supply did not always add a benefit (Appendix A).

Scope of intervention. ST01/ST02 supplied summaries early at formation, confirmation, and recheck together, leaving their individual contributions unresolved. JR01's formation comparison varies only the summary field, but its confirmation comparison also includes condition-specific old notes. The evidence does not identify a single cause of incorrect joint acceptance, such as the summary, an old judgment, or self-prioritization. We collected neither free-form rationales nor internal reasoning; the analysis concerns input–output correspondence.

Operationalization. Role-specific judgment is a recommendation derived from explicitly supplied synthetic observations, not a spontaneously formed personality or value system. Experience is presented at each stage, and initial joint proposals are researcher-constructed. Long-term continuation, asymmetric information, larger groups, spontaneous negotiation, and memory retention without renewed evidence supply remain untested. Joint "quality" is the task's specified total-maximization objective, rather than a general account of fairness or diverse real-world preferences.

Evaluation contract. Subsequent use scores fidelity to a feasible approved agreement without requesting re-optimization. Under this contract, faithful use of a low-quality agreement passes the use criterion. A system that rechecks quality immediately before execution would require a different task contract and comparison. Success

in constructed-counterproposal diagnostics was not pooled as evidence that the same checking opportunity arose naturally.

Design, scoring, and availability. The series was an adaptive exploration in which earlier results informed subsequent comparisons, not a publicly preregistered confirmatory study. Fixed scoring was checked with separate code and original responses, but development, verification, and writing remained within the same project. No blinded third-party rescoring was performed. Current supplementary materials index local research records rather than constitute a public reproducibility package.

8. Conclusion

Separately storing and evaluating experience-based recommendations and joint assignments distinguished the stages supported by role-specific summaries from remaining defects in joint approval. Early summary supply improved role-specific correctness along the pathways in both material sets, and the formation comparison retained the same direction across three fixed-input repetitions.

At the same time, suboptimal acceptance recurred from an input containing a correct role-specific judgment and computed joint totals. Along the actual pathway, the resulting agreement was subsequently used faithfully. A combined outcome could also remain unsuccessful while the incorrect component changed between the role-specific and joint fields.

Maintaining one's own judgment, agreeing on a good joint allocation, and executing the agreement faithfully should be checked separately against saved artifacts and the inputs to each stage. This report provides concrete examples of how that evaluation decomposition reveals both the reach of computational support and the defects that remain.

9. Ethics, Author Contributions, and Data Availability

All task materials are synthetic, and the responses are LLM-generated. The study involves no human-participant experiment or personal data from real customers. The author determined the research direction and execution scope, ran the Colab experiments, assessed results, and reviewed the manuscript. LLM assistance was used for design, implementation, scoring code, verification, literature research, and writing. Model-assisted reading and feedback are distinguished from independent peer review or third-party rescoring.

Research materials are retained without a deletion deadline and distributed in the MRSI repository. Supplement S1 links fixed plans, original ZIP archives, aggregates, response diagnostics, and scoring and verification code. The repository specifies Apache 2.0 licensing, distribution scope, and current reproduction instructions. A permanent archive DOI for this paper has not been assigned. Formal funding and conflict-of-interest declarations require the author's confirmation for the public version.

References

1. Xuhui Zhou, Hao Zhu, Leena Mathur, Ruohong Zhang, Haofei Yu, Zhengyang Qi, Louis-Philippe Morency, Yonatan Bisk, Daniel Fried, Graham Neubig, and Maarten Sap. 2024. *SOTOPIA: Interactive Evaluation for Social Intelligence in Language Agents.* ICLR 2024. arXiv:2310.11667v2.
2. Federico Bianchi, Patrick John Chia, Mert Yuksekgonul, Jacopo Tagliabue, Dan Jurafsky, and James Zou. 2024. *How Well Can LLMs Negotiate? NegotiationArena Platform and Analysis.* arXiv:2402.05863v1.
3. Kunlun Zhu, Hongyi Du, Zhaochen Hong, Xiaocheng Yang, Shuyi Guo, Zhe Wang, Zhenhailong Wang, Cheng Qian, Xiangru Tang, Heng Ji, and Jiaxuan You. 2025. *MultiAgentBench: Evaluating the Collaboration and Competition of LLM agents.* ACL 2025, 8580–8622. DOI:10.18653/v1/2025.acl-long.421.
4. Chang Ma, Junlei Zhang, Zhihao Zhu, Cheng Yang, Yujiu Yang, Yaohui Jin, Zhenzhong Lan, Lingpeng Kong, and Junxian He. 2024. *AgentBoard: An Analytical Evaluation Board of Multi-turn LLM Agents.* NeurIPS 2024. arXiv:2401.13178v2.
5. Mert Cemri et al. 2025. *Why Do Multi-Agent LLM Systems Fail?* arXiv:2503.13657v3.
6. Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. 2024. *τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.* arXiv:2406.12045v1.
7. Luyu Gao, Aman Madaan, Shuyan Zhou, Uri Alon, Pengfei Liu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. *PAL: Program-aided Language Models.* ICML 2023, PMLR 202:10764–10799. PMLR.
8. Tian Xia, Zhiwei He, Tong Ren, Yibo Miao, Zhuosheng Zhang, Yang Yang, and Rui Wang. 2024. *Measuring Bargaining Abilities of LLMs: A Benchmark and A Buyer-Enhancement Method.* Findings of ACL 2024, 3579–3602. DOI:10.18653/v1/2024.findings-acl.213.

Appendix A. Task and Evaluation Scope Established by the Preceding Exploration

This table preserves an entry point to the exploratory series. Results from trials with different methods or materials are not pooled into a single effect estimate. Supplement S1 provides the formal results, outcomes unresolved because of format defects, and post hoc reference analyses for each trial.

Trial	Evaluation target	Relation to the main text
ES01: Own experience and joint records	Reversed experience; a diagnostic reversing only the joint allocation; subsequent use	Basic task separating one's recommendation from a joint assignment
AC01: Agreement on tied candidates	Responses and agreement for two tied allocations	Distinguishing an appropriate proposal from an agreed proposal
EU01: Experience updates	Legitimate own updates, preservation of the other role's judgment, and joint allocation	A correct own update can coexist with a resource violation
FC01/FC02: Feasible candidates	Directly writing versus selecting a joint allocation; transfer to unused experience	Separating feasibility from optimality
CT01: Candidate totals	Supplying computed totals	Optimal selection increased from 20/32 to 32/32 within the comparison; defects remained in the role-specific field and confirmation
OS01: Own-experience summaries	Summary supply at proposal and use	Formation was 32/32 in both conditions; 56 responses with extra keys remain formally unresolved
BJ01: Boundary-judge evaluation	Judge calibration separating structure and content	Of 64 judgments, 53 agreed, 4 disagreed, and 7 were unresolved; this did not justify replacing main-trial scoring with an LLM judge
CS01: Support at confirmation	Supplying joint candidates and totals to the confirming role	Appropriate confirmation increased from 28/32 to 32/32; appropriate response to constructed incorrect proposals increased from 1/8 to 7/8
CR01: Counterproposal recheck	One recheck and subsequent use by both roles	Subsequent-use correctness can conceal defects in earlier role-specific formation

The absence of a formation difference in OS01 and the formal failure to meet progression criteria in ST02, including its incorrect agreement, remain part of the evidence on computational support. Earlier format defects are not repaired and counted as successes in the main results of this report.

Appendix B. Progression Criteria, Constructed Diagnostics, and Implementation

The ST01/ST02 progression criteria required end-to-end J ≥ 22/24; J ≥ 7/8 for each initial-proposal type; I = 24/24; zero incorrect agreements; G ≥ 7/8 for optimal initial proposals and ≥ 6/8 for each of the suboptimal and resource-violating types; and 12/12 in the constructed diagnostic. These were engineering criteria for progression within fixed materials, not statistical-significance thresholds.

Table B1. Constructed diagnostics and overall progression, kept separate from the main evaluation.

Trial	LATE: constructed diagnostic	EARLY: constructed diagnostic	LATE: overall criteria	EARLY: overall criteria
ST01	9/12	12/12	Not met	Met within the fixed sample
ST02	12/12	12/12	Not met	Not met: 1 incorrect agreement

The constructed diagnostic used four materials and three counterproposals per material to check acceptance of optimal proposals and deferral of suboptimal or resource-violating proposals. It was counted separately from pathways where actual confirmation generated a counterproposal. JR01 added no recheck, subsequent use, or constructed diagnostic.

The implementation separated input evidence, model generation, structural validation, storage, and substantive scoring. Storable responses were not filtered by correctness labels, allowing incorrect judgments to travel downstream. References in role-specific and joint fields were checked separately. Copying an input-only summary into an extra output field counted as a structural defect. Saved record contents were passed as data rather than executed as new instructions.

Supplement S1 retains hashes linking scores, raw generation, and subsequent inputs, together with the reading records, including successful cases. The extracted tables and examples were derived from existing aggregates. Scoring rules and formal experimental results were not changed for manuscript preparation.

Boundaries Between Role-Specific Judgment, Individual Approval, and Group Agreement in Three LLM Roles

Carrying Actual Votes Forward and Comparing Matched Approval States

Kentaro Sato
Independent Researcher
Contact: info@metaclan.jp (email)

October 13, 2026 | English version v1.0

Repository and research materials | Apache License 2.0

Abstract

For LLM roles to cooperate while preserving their own judgments, they must distinguish the quality of a joint proposal from who has approved it. We ran the same Qwen3.5-27B model as three roles in separate contexts and evaluated recommendations based on each role's experiences, votes on joint proposals, and agreement and execution decisions derived from received votes. The task allocated paper and digital guidance equipment among three roles. Code supplied summaries of each role's experiences and totals for two joint candidates; the LLM produced recommendations, votes, and interpretations of the approval state.

In a connected sequence using three cases with two technical replicates each, one role correctly retained its own judgment but withheld approval of the higher-total proposal. All three subsequent readers correctly preserved the absence of agreement based on that actual vote. A matched comparison then held own notes and the proposal fixed while changing one role's vote among missing, explicit hold, and approval. Across three cases, two technical replicates, and three readers, all 54 responses correctly preserved role-specific judgments and agreement and execution decisions, but one response recorded a missing vote as an explicit hold. We also compared a condition that required the model to state the ranking of the candidates before voting. It produced 54/54 correct votes versus 52/54 in the direct condition on previously used cases; both conditions achieved 54/54 on numerical cases not previously used in this study.

These results show that voting appropriately, faithfully handling actual votes, and correctly recording each role's approval state are distinct evaluation targets. Combining downstream use of actual votes with matched interventions on approval states made it possible to trace cooperation that preserved role-specific judgments, as well as state errors hidden by correct final decisions.

Keywords: LLM agents, three-role cooperation, role-specific judgment, individual approval, unanimous agreement, state readout, fixed-input repetition

1. Problem Setting and Contributions

Suppose three guides have each tried paper and digital guidance in their own area. Working alone, each could choose the medium that worked better in their own experience. In a joint task, however, only one digital device is available. Even if digital guidance works better in a particular guide's area, allocating the device to another area and assigning paper to that guide may produce a higher total across the three areas.

Two judgments can therefore hold together: "Digital works better in my own experience" and "I will use paper in the joint task." Proposal quality and approval are also distinct. If two roles have approved but the third has not replied, a task requiring everyone's explicit approval cannot yet begin.

This paper **evaluates the appropriateness of votes separately from fidelity to the records after voting. It also shows that a correct agreement and execution decision can coexist with an incorrect record of who expressed what.** We examine a path that passes LLM-generated own notes and actual votes into subsequent inputs, together with a comparison that changes only one role's approval state while retaining the same saved material.

Table 1. Three findings and the comparisons supporting them

Finding	Observed relationship	Main evidence
Vote appropriateness and fidelity to actual votes can diverge	Three subsequent roles treated an incorrect HOLD on the higher-total proposal as the recorded hold	A connected path from own-note formation through voting to readout
Incorrect individual states can remain behind correct final decisions	A missing vote was recorded as an explicit hold, while pending agreement and deferred execution were correct	Three approval states with the same own notes and proposal
Differences associated with a comparison field varied across cases and replicates	Two additional votes were correct on previously used cases; both conditions were entirely correct on new cases	Matched comparisons of fixed voting inputs and a check on new cases

These findings organize the exploratory results; they were not a single hypothesis specified before the entire series. For each comparison, inputs, allocation, scoring, counts, and stopping conditions were fixed in local documents before execution. The final three-state comparison was likewise designed in light of earlier findings, with a commitment to close the 54-response run regardless of the direction of its results.

The broader research program concerns forming shared states while preserving the distinctiveness of individual agents. Here, part of that aim is operationalized as **reporting recommendations and evidence grounded in each role's experiences without confusing them with joint assignments or approval states**. The task does not measure personality or autonomous values by requiring the roles to keep giving different answers.

2. Related Work

2.1 Reaching Joint Answers Through Debate and Voting

Kaesberg et al. compare consensus mechanisms, including unanimity, with several voting mechanisms to examine relationships between decision protocols and tasks [1]. ReConcile improves answers through iterative discussion and aggregation in which diverse LLMs consult one another's answers, explanations, and confidence [2]. Choi et al. separate discussion from voting over initial answers to evaluate the contributions of discussion and aggregation [3].

Against this background, using three roles or a unanimity rule is not itself this paper's contribution. We focus on the relationship between votes on proposals that can be evaluated from numerical evidence and **the subsequent**

interpretation of each role's actual received vote. Voting refers to candidate totals, whereas readout refers to received votes. Consequently, the same HOLD has different evaluative meanings at different stages.

2.2 Individual Objectives and Verifiable Cooperative States

SOTOPIA evaluates interactions involving roles and social goals along multiple dimensions [4]. Negotiation-Arena examines proposals, acceptance, and negotiation outcomes in resource exchange, allocation, and buying and selling [5]. In our task, an own recommendation is determined by cumulative success rates in a role's experiences. The joint task has a common objective of maximizing the total. Preserving an own recommendation is therefore distinct from pursuing a self-interested negotiation strategy.

MASBench studies collaboration under partial observability. In scheduling tasks, it distinguishes failures in candidate search from failures in verification and uses memory of candidate and rejected options together with phase-specific JSON voting [6]. Albayaydh and Zhao present a framework that checks negotiation-message schemas and protocol invariants at runtime [7]. These studies provide closely related designs that distribute candidate search, structured voting, state management, and verification across components.

We fix candidate search by supplying all three roles' numerical evidence and a precomputed candidate table at joint-decision stages. Holding own notes and the proposal constant, we compare inputs in which the same role's vote is absent, explicitly on hold, or approving. The focus is to inspect each reader's own-information fields, per-role approval states, agreement, and execution assignment together.

2.3 Stage-Specific Evaluation and Repeated Reliability

MultiAgentBench evaluates contributions and collaboration processes as well as task performance [8]. MAST categorizes failures spanning system design, inter-agent misalignment, and verification from execution histories [9]. The τ-bench benchmark evaluates consistent success over repeated attempts at the same task [10]. Sharing these concerns, we describe how the correctness of votes and approval states persists or varies when saved identical inputs are repeated.

Table 2. Related evaluation targets and the present comparisons

Research area	Targets in prior work	What this paper fixes or varies
Consensus and voting for answer aggregation [1–3]	Decision protocols, debate, task performance after aggregation	Evaluation of two candidates and subsequent readout of generated votes
Social interaction and negotiation [4,5]	Role-specific goals, proposals and acceptance, joint outcomes	Experience-based own recommendations and joint assignments
Cooperative states and protocol verification [6,7]	Candidate search, state management, voting, constraint checks	Three approval states with fixed own notes and a fixed proposal
Stage-specific failures and repetition [8–10]	Intermediate achievement, failure location, consistency of success	Saved records, individual output fields, and correctness variation on identical inputs

The contribution is **to connect a path that carries actual votes into subsequent judgments with a matched comparison varying only approval states, so that role-specific judgments, votes, state records, and execution decisions can be evaluated against the same evidence**.

3. Task and Evaluated Boundaries

3.1 Own Recommendations and Joint Assignments

The same model is run as roles A, B, and C. Every generation uses a fresh context with the role name, experience records, and required saved records explicitly supplied. Each role has two experience rows. A row records its

owner, session, trial and success counts for paper and digital guidance, and an experience ID. Each medium has 20 trials per session, or 40 across the two sessions.

For its own task, a role recommends the medium with the higher cumulative success rate over all its own experiences. For role i and medium m, the predicted number of successes per 40 people is

$$q_i(m) = 40\frac{\sum_t s_{i,t,m}}{\sum_t n_{i,t,m}}.$$

Here, s denotes successes and n denotes trials. In the joint task, one medium is available for one role and the other for two roles. For the two feasible candidates c supplied by code, the model compares the total under assignment $a_c(i)$:

$$Q(c) = \sum_{i\in\{A,B,C\}} q_i(a_c(i))$$

Throughout this paper, the "higher-total proposal" means **the proposal with the higher total among the two presented candidates**. The model does not search all possible assignments. Neither own recommendations nor candidate totals contain ties.

Own-note formation receives the role's own experiences and summary. Voting and readout receive all three roles' experiences, the reader's or voter's own summary, and the per-role predictions and total for each candidate. The purpose is to examine which evidence is used for different judgment targets, rather than whether the model can correctly add the totals.

3.2 From Individual Votes to Agreement

An own note stores its holder, recommended medium, and own experience IDs. When voting, the role must output ACCEPT for the higher-total proposal and HOLD for the lower-total proposal. A proposal ID identifies the proposal being voted on, separately from candidate IDs.

Code carries forward the sender, proposal ID, and ACCEPT/HOLD decision from each generated vote as a received vote record. Each reader reports the individual approval states, overall agreement, and its execution assignment. Agreement requires everyone's explicit approval: only three ACCEPT votes on the same proposal produce AGREED; otherwise the state is PENDING. For AGREED, the reader outputs its assigned medium under the proposal. For PENDING, it outputs HOLD to defer execution.

Table 3. Evidence and expected answers for each judgment target

Judgment target	Output field	Evidence used to determine the expected answer
Role-specific judgment	`own`: holder, recommendation, experience IDs	The role's original experiences and the role addressed by the call
Vote on a joint proposal	`vote`: proposal ID, ACCEPT/HOLD	Totals of the two presented candidates and the current proposal
Each role's approval state	`approval_state`: separate states for A/B/C	Vote records actually present in the current readout input
Group agreement	`agreement`: AGREED/PENDING	Explicit approval by all three roles for the same proposal
Execution decision	`joint_action`: PAPER/DIGITAL/HOLD	Agreement and the reader's assignment under the proposal

A HOLD vote records an explicit decision to withhold approval. MISSING means that the role's vote is absent from the input. Both prevent group agreement, but they differ in what the role has expressed. Readout is not a new vote: even if HOLD is recorded for the higher-total proposal, the reader must not correct it to ACCEPT.

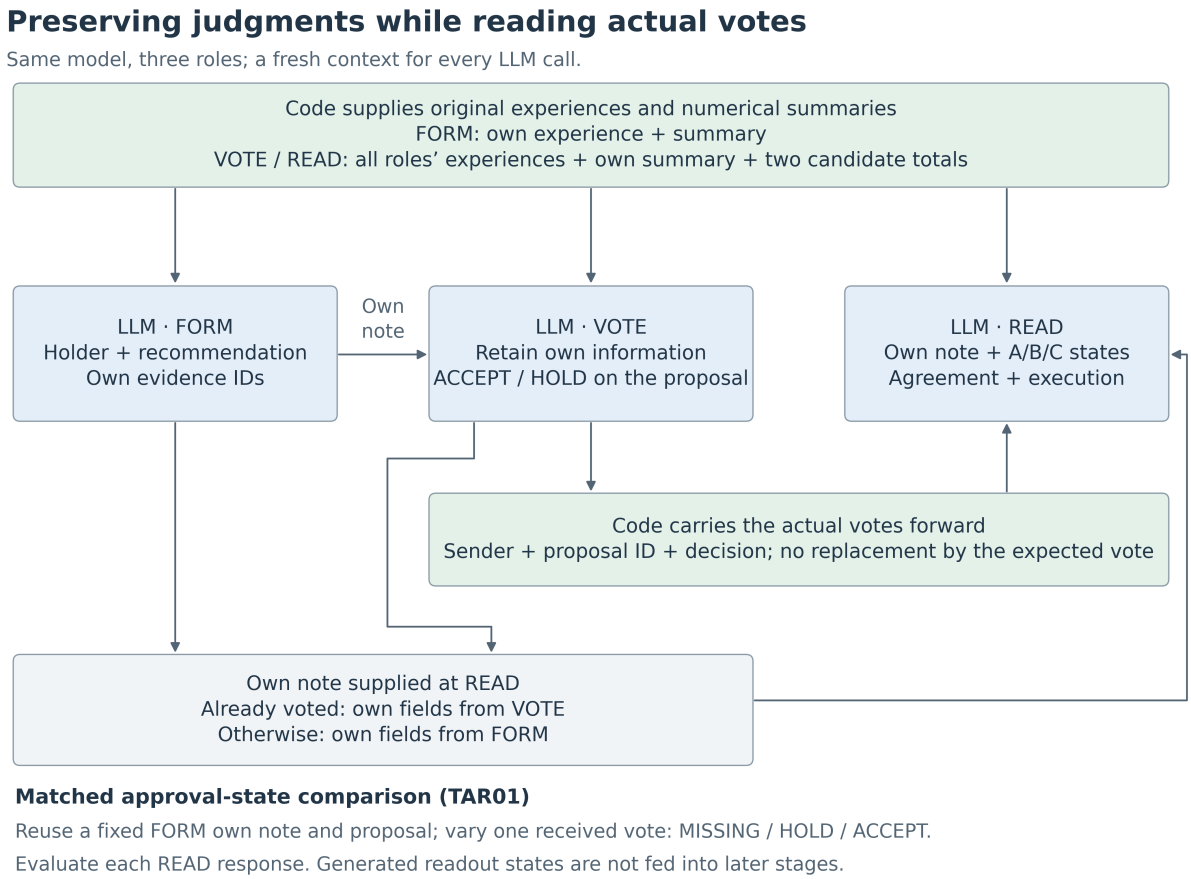


Figure 1. Information flow under evaluation. Every LLM operation uses a fresh context. Code supplies evidence and actual records without substituting the expected vote or agreement decision. In the connected path, a reader's own note comes from its voting response if it has already voted, or from its formation response otherwise. In the matched approval-state comparison, the same formation-stage own note is used across all states.

3.3 Worked Example: A Role's Preferred Medium and the Joint Allocation

Table 4 shows predicted successes for one case. Role A recommends digital guidance for itself. In the joint task, where only one role can receive the digital device, the candidate assigning it to B has the higher total of the two.

Table 4. Own and joint judgments derived from the same experiences (case N01)

Role	Paper prediction	Digital prediction	Own recommendation	Assignment in the higher-total proposal
A	24	29	DIGITAL	PAPER
B	24	35	DIGITAL	DIGITAL
C	32	16	PAPER	PAPER
Two candidate totals	A DIGITAL, B PAPER, C PAPER: 85	A PAPER, B DIGITAL, C PAPER: 91	—	Proposal totaling 91

A's appropriate response is to retain DIGITAL as its own recommendation and approve the proposal totaling 91. If all three roles approve, A's joint assignment becomes PAPER, without requiring A to change its own recommendation to PAPER.

Now suppose the input contains only B's and C's ACCEPT votes for that proposal; A's vote has not been supplied. The correct states are A MISSING, B ACCEPT, and C ACCEPT. Agreement remains pending and execution is

deferred. Section 5.2 presents an actual response that misidentified only A's state in this example.

4. Methods

4.1 Connecting Actual Records and Repeating Fixed Voting Inputs

The connected-path trial, TB01, used three cases with two technical replicates each. Each replicate formed own notes for all three roles and reused them across paths for the higher- and lower-total proposals. In each path, the three roles voted individually. The received records after two votes and after three votes were then read by each of the three roles. The trial produced 18 formation, 36 voting, and 72 readout responses, for a total of 126. Voters did not receive earlier votes from other roles. Readout responses after two votes were not fed back into subsequent voting or readout.

The fixed-input voting trial, TVR01, retained all 18 voting inputs from TB01's first replicate, including the saved own notes. Each input was used for three new generations with new seeds, producing 54 votes. This comparison examined variation on identical inputs; it did not regenerate formation or readout responses.

4.2 Two Conditions Separating Own Information From Candidate Comparison

The voting comparison held own notes, original experiences, and candidate tables constant between two conditions:

- **DIRECT:** Output the own-information fields followed by the vote.
- **COMPARE:** Between the own-information fields and the vote, output the target candidate ID, the other candidate ID, and their HIGHER/LOWER relationship. The instructions also specify how the comparison maps to the vote.

TCV01 used the same 18 previously used inputs, with three technical replicates per condition: 108 responses, or 54 matched pairs. Each pair used the same seed. Within the 18 pairs in each replicate, nine ran each condition first. TCV02 retained the instructions and scoring while using three numerical cases not previously used in this study, with changed experience values and candidate assignments. Nine newly formed own notes were reused across the subsequent 108 votes.

The common evaluation targets were the three own-information dimensions and the vote. COMPARE additionally evaluated candidate binding, the ranking, and consistency between the ranking and vote. This is a comparison of conditions that jointly change instructions, schema, and output amount, rather than an estimate of the isolated causal effect of a single comparison field.

4.3 Three Approval States With Fixed Own Notes and Proposals

The approval-state comparison, TAR01, used records saved in the new-case comparison. It fixed the nine formation-stage own notes and the nine actual votes from COMPARE's first replicate on the higher-total proposals. The selection rule was defined in this follow-up design after all records had been inspected. It did not select favorable votes from among the three replicates.

For each case, the focal role was the one whose assignment under the higher-total proposal differed from its own recommendation: A in N01, B in N02, and C in N03. Keeping the other two roles' approvals fixed, we changed only the focal role's received vote as shown in Table 5.

Table 5. Input interventions and expected readout for three approval states

Condition	Construction of the focal role's vote	Focal role's state	Group agreement	Execution decision
Missing vote (MISSING)	Remove the actual vote from the input	MISSING	PENDING	HOLD
Explicit hold (HOLD)	Replace only the actual vote's decision, from ACCEPT to HOLD	HOLD	PENDING	HOLD
Approval (ACCEPT)	Use all three actual votes unchanged	ACCEPT	AGREED	Reader's assignment under the proposal

MISSING and ACCEPT present a subset or the full set of actual votes. HOLD is a counterfactual vote constructed by the experimenter, not a newly generated decision by the model to withhold approval. The original votes were retained and the input transformation was recorded. Nor did the experiment induce an actual communication delay.

Three cases × three states × three readers × two technical replicates yielded 54 responses, with 18 per state. The three states for the same case, reader, and replicate formed one of **18 matched triplets**. A common seed was used within each triplet. The six possible state orders were assigned to three triplets each. There were 27 distinct inputs, each generated twice. All states used the same formation-stage own note, separating changes in approval state from changes in the supplied own note.

Inputs contained neither condition labels nor expected answers. Received vote records supplied only sender, proposal ID, and decision. The comparison fields accompanying the original votes were omitted, and removed votes were not exposed through another route. Each readout used a fresh context. Evaluation ended at the own-information fields, approval states, agreement, and execution decision. We did not evaluate the effects of saving generated readout responses as new states and feeding them into later stages.

4.4 Scoring, Repetition, and Stopping Conditions

Own information was scored along three dimensions: holder, recommendation, and evidence IDs. Votes were scored against candidate totals and the proposal ID. Readout added three dimensions: the complete A/B/C approval state, agreement, and execution decision. The approval-state dimension was correct only if all three entries matched the input votes.

Expected answers and formal scores were computed entirely by fixed-rule code. Success rates were calculated as fractions from original experiences, and evidence and proposal IDs were checked against their sources. In the connected path, a structurally valid but semantically incorrect response was passed forward as recorded, without replacement by the expected answer. A prespecified rule treated format failures as unresolved; the five completed trials covered here had no format failures, missing responses, or answer repairs.

The fixed diagnostic assigned PASS only when every planned response was correct on every designated dimension; any semantic failure produced FAIL. Differences between conditions were reported separately, including cases in which COMPARE was entirely correct but DIRECT was not and the overall diagnostic was therefore FAIL.

We distinguish cases, inputs, responses, matched pairs, and matched triplets. Dependent observations are not counted as independent sample sizes, and we do not estimate population rates or calculate p-values. Two or three technical replicates were used to describe variation. Illustrative examples selected after execution and supplementary summaries restricted to agreement and execution are identified as such.

4.5 Model and Evidence Preservation

All trials used the same Qwen3.5-27B revision with BF16 weights, SDPA, and non-thinking mode. Settings were fixed at temperature 0.7, top-p 0.8, top-k 20, a maximum of 1,024 generated tokens, and a maximum of 8,192 input tokens. Runs used an RTX PRO 6000 Blackwell Server Edition GPU in Colab. No action tools were provided to the model; execution decisions were limited to reporting a medium or HOLD in an output field.

Original responses, subsequent inputs, rendered prompts, token sequences, and generation settings were saved. Returned ZIP archives were reaggregated using fixed local analysis code. An independently implemented verifier within the same project also checked correspondences among original experiences, received votes, and outputs. Appendix C lists the model revision and dependencies, and Appendix D provides evidence entry points.

5. Results

5.1 An Incorrect Vote Can Be Followed by Faithful Readout

In the connected path, TB01, the holder, recommendation, and evidence were correct in all 126 responses. Votes were correct in 35/36 cases and readout in 72/72. The single voting failure reduced end-to-end success to 11 of 12 paths.

Table 6. Success in the connected path and the stage at which a failure occurred

Stage or aggregation unit	Correct	Observed failure
Three own-information dimensions (all responses, each dimension)	126/126	None
Own-note formation	18/18	None
Individual voting	35/36	One HOLD on the higher-total proposal
Readout after two votes	36/36	None; all PENDING and HOLD
Readout after three votes	36/36	None; decisions followed the three actual votes
End-to-end (case × replicate × higher/lower proposal)	11/12 paths	One higher-total path containing the vote above

The failure occurred in case T02, where C's predictions were 26 for paper and 29 for digital. The joint candidates were A PAPER, B DIGITAL, C PAPER, totaling 90; and A PAPER, B PAPER, C DIGITAL, totaling 84. C correctly reported its own recommendation DIGITAL and own evidence, but returned HOLD for the proposal totaling 90.

The other two roles had accepted that proposal, so the three received votes were A ACCEPT, B ACCEPT, and C HOLD. Each reader—A, B, and C—recorded C as HOLD, agreement as PENDING, and execution as HOLD. **The erroneous vote was not replaced by the expected vote downstream; the resulting absence of agreement was correctly preserved.** In the second replicate using the same voting input for C, the vote was ACCEPT. After agreement, C retained DIGITAL as its own recommendation while reporting PAPER as its joint assignment.

In the fixed voting repetition, TVR01, the own-information dimensions were again correct in 54/54 responses, while voting was correct in 52/54. Sixteen of the 18 inputs were correct in all three replicates; two mixed correct and incorrect votes. The T02/C higher-total input produced ACCEPT, ACCEPT, and HOLD. The T03/A lower-total input produced HOLD, ACCEPT, and HOLD, showing an incorrect approval of a lower-total proposal as well as an incorrect hold on a higher-total proposal. These new votes were not passed downstream, so this trial does not establish that the incorrect approval led to group agreement.

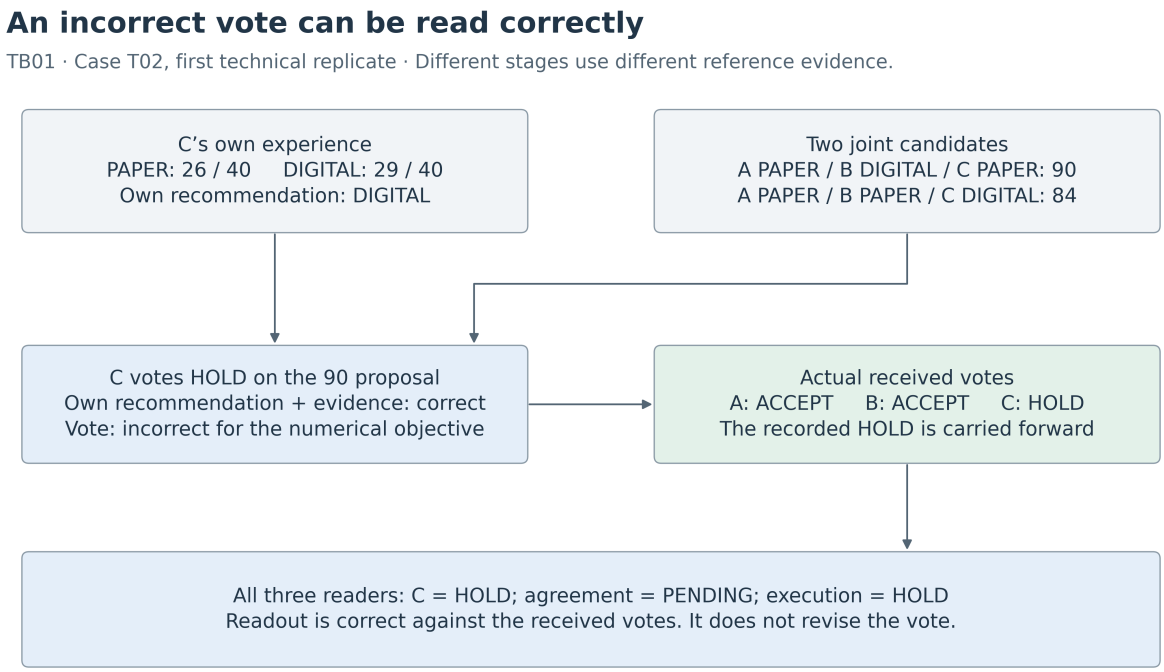


Figure 2. A connected path combining a voting failure with correct downstream readout. The figure shows the first replicate of case T02. Voting and readout use different reference evidence, so retaining C's recorded HOLD is the correct downstream response.

5.2 A Missing Vote Became an Explicit Hold Behind a Correct Deferral

In the matched approval-state comparison, TAR01, each own-information dimension, agreement, and execution decision was correct in 54/54 responses. Per-role approval states were correct in 53/54, with one failure in the missing-vote condition.

Table 7. Readout results by approval state (18 responses per condition)

Evaluation target	Missing vote	Explicit hold	Approval
Own-note holder	18/18	18/18	18/18
Own recommendation	18/18	18/18	18/18
Own evidence IDs	18/18	18/18	18/18
Approval states of all three roles	**17/18**	18/18	18/18
Group agreement	18/18	18/18	18/18
Execution decision	18/18	18/18	18/18
All six dimensions	17/18	18/18	18/18

The failure occurred when A read case N01 from Section 3.3 in the first replicate. The input contained B's and C's ACCEPT votes for the proposal totaling 91, but no vote from A. The model correctly preserved A's recommendation DIGITAL and its two experience IDs, yet recorded A's approval state as HOLD rather than MISSING. The following reproduces the original JSON values, with whitespace adjusted only for presentation.

```
{
  "own": {
    "holder": "A",
    "recommendation": "DIGITAL",
    "evidence_ids": ["E79c8f7f51e", "Ee4703603d4"]
  },
  "approval_state": {"A": "HOLD", "B": "ACCEPT", "C": "ACCEPT"},
  "agreement": "PENDING",
  "joint_action": "HOLD"
}
```


The error was to treat an absent vote from A as an explicit HOLD. Keeping agreement pending and deferring execution were appropriate. For the same case, reader, and replicate in the explicit-hold condition, the model produced the same JSON values. In the approval condition, it instead recorded all three roles as ACCEPT and switched to AGREED with PAPER as its joint assignment.

Table 8. A's actual responses with the same own note (case N01)

Replicate	A's vote in the input	Reported state of A	Agreement / execution	Own recommendation
1	Absent	**HOLD (expected: MISSING)**	PENDING / HOLD	DIGITAL
1	HOLD	HOLD	PENDING / HOLD	DIGITAL
1	ACCEPT	ACCEPT	AGREED / PAPER	DIGITAL
2	Absent	MISSING	PENDING / HOLD	DIGITAL
2	HOLD	HOLD	PENDING / HOLD	DIGITAL
2	ACCEPT	ACCEPT	AGREED / PAPER	DIGITAL

The missing-vote prompt and token sequence were identical in the first and second replicates; the second correctly reported MISSING. Of the 27 inputs repeated twice, this was the only one for which the structured response values and correctness changed.

Across the 18 matched triplets of approval states, own information was correct and identical throughout in 18/18. The distinction between a missing vote and an explicit hold was correct in 17/18, and all six dimensions were correct in all three states in 17/18 triplets. A post hoc supplementary summary restricted to agreement and execution yielded 18/18. No response declared agreement or execution without everyone's approval, or deferred execution when all three had approved.

This case shows that an appropriate final deferral does not ensure that each role's underlying state has been recorded correctly. Missing votes and explicit holds map to the same overall decision, so correctness at the endpoint alone could not detect the confusion.

5.3 The Difference With a Comparison Field Did Not Appear on New Cases

In TCV01, which separated own information from the ranking of the candidates, DIRECT produced 52/54 correct votes and COMPARE produced 54/54. Own information was correct in 54/54 for both conditions. Both differences occurred in the first replicate; all votes were correct in both conditions in the remaining two replicates.

In TCV02's new cases, all nine formation responses were correct, followed by 54/54 correct own-information and voting responses in each condition. In both trials, COMPARE's candidate IDs, ranking, and mapping from ranking to vote were correct in all 54 responses.

Table 9. Matched voting comparison and breakdown by replicate

Cases	Replicate	DIRECT	COMPARE	Matched pairs correct only in COMPARE
Three previously used cases (TCV01)	1	16/18	18/18	2
Same	2	18/18	18/18	0
Same	3	18/18	18/18	0
Same	Total	**52/54**	**54/54**	**2/54**
Three new cases (TCV02)	1	18/18	18/18	0
Same	2	18/18	18/18	0
Same	3	18/18	18/18	0
Same	Total	**54/54**	**54/54**	**0/54**

Neither trial contained a pair correct only in DIRECT or a pair incorrect in both conditions. Comparison fields preceded the vote in every response. The two TCV01 differences corresponded to the two inputs that varied in the fixed repetitions in Section 5.1. For T02/C's proposal totaling 90, DIRECT returned HOLD, whereas COMPARE returned HIGHER and ACCEPT. For T03/A's proposal totaling 76, DIRECT returned ACCEPT, whereas COMPARE returned LOWER and HOLD. Own recommendations remained correctly DIGITAL and PAPER, respectively.

The condition with comparison fields showed a favorable difference on previously used inputs and correctly combined own information, comparison, and voting on new cases. DIRECT was also entirely correct on the new cases, however, so the difference did not persist uniformly across the materials. A zero observed difference is not evidence of equivalence.

Table 10. Output amounts and generation times recorded in the same voting comparisons

Cases	DIRECT output tokens	COMPARE output tokens	DIRECT generation seconds	COMPARE generation seconds
Three previously used cases	4,799	7,522	264.159	408.596
Three new cases	4,573	7,526	252.141	408.186

Each entry is the total for 54 responses and excludes formation costs. COMPARE increased both output amount and generation time. These are cost records from the same runs, not a dedicated speed benchmark. The value of making the comparison visible should be assessed alongside its additional cost.

6. Discussion

6.1 Choosing a Good Proposal and Preserving Actual Votes

Vote appropriateness was judged against this task's specified objective: maximizing the total among the two presented candidates. This is distinct from the general legitimacy of withholding approval in real-world decision-making. Readout refers to actual votes regardless of proposal quality. In the connected example, HOLD on the better proposal was an incorrect vote, but preserving that HOLD and reporting pending agreement was appropriate readout. Making downstream vote interpretation more accurate would therefore not automatically produce more agreements on good proposals.

This distinction identifies which stage needs improvement before cooperation is reduced to a single success rate. Improving votes requires examining candidate evaluation and its mapping to the vote. Improving agreement

judgments requires examining received votes and the agreement rule. Scoring the same HOLD as an error upstream and a faithful record downstream follows this division of responsibilities.

6.2 Overall State Does Not Recover Individual States

In one three-state comparison response, A's missing vote became an explicit hold while own information, agreement, and execution were all correct. The fact that both states lead to PENDING and HOLD follows directly from the rule: correctness after aggregation cannot distinguish them. The empirical observation is that **an actual generated response lost this distinction, whereas another replicate of the same input preserved it**.

In an operational system, the distinction could matter for deciding whose response to await and whom to ask to reconsider. Here, checking each role's reported state separately detected a record of an unexpressed intention even while the overall decision appropriately remained on hold.

6.3 Making a Comparison Visible Provides Information Beyond an Accuracy Gain

COMPARE externalized the ranking between the target and other candidate and the mapping from that ranking to the vote. This allowed the correctness of the ranking and the consistency of the vote with it to be scored separately. Both were entirely correct in COMPARE, so the comparison did not identify the internal mechanism of DIRECT's errors.

The two-response difference on previously used cases is a useful local observation, as is the tie on new cases. An evaluation faithful to these results should present diagnostic information, performance by case set, and generation costs together, rather than equating the value of an additional visible field with a universal gain in accuracy.

6.4 Retaining an Own Judgment While Accepting a Joint Assignment

When everyone approved, a role could report a joint assignment different from its own recommendation while retaining that recommendation and its evidence. Following the same joint proposal did not require rewriting each role's evaluation of its own experiences to match the joint assignment.

The experimental conditions supporting this coexistence included original own experiences and their summary, joint candidate totals, explicit own-information fields, received votes, and an agreement rule. With these supplied, the study identified which fields were correct and where failures remained. Further work on role distinctiveness and shared states can preserve these separations while evaluating changes in experiences or votes.

7. Scope and Limitations

Cases and model. This is a small diagnostic using one model revision and generation configuration, with six synthetic numerical cases. New cases also came from the same task template. Free negotiation, real customer interactions, and transfer to other models were not evaluated. Role-specific judgment is defined as experience-based recommendation and evidence, rather than personality or enduring individuality.

Information and interventions. Voting and readout received all three roles' experiences and precomputed totals. The study did not examine negotiation with withheld information or difficult candidate search. COMPARE also changed instructions, schema, and output length. The approval-state comparison was a follow-up using fixed existing own notes and votes, not independent regeneration of complete paths from FORM onward.

Dependence and variation. Roles, proposals, and replicates derived from the same case are dependent; readouts also share the same set of votes. The 18 matched triplets are not 18 independent cases. Limited repetitions do not establish population error frequencies, superiority or equivalence between conditions, or the internal

reasoning responsible for variation. Nor do these results estimate a causal effect of increasing the number of roles, because no two-role control was run under matched conditions.

Untested states. The three-state comparison used higher-total proposals and constructed HOLD votes. Crossing these conditions with unanimous incorrect approval of a lower-total proposal, votes for other proposals, late arrival, revocation, mixed versions, and renegotiation remains future work. Senders were bound by the host, so resistance to impersonation was not tested. Execution was an answer in the `joint_action` field; no real-world task was executed.

8. Conclusion

Tracing actual records from three LLM roles separated the correctness of role-specific judgments, votes on joint proposals, individual approval records, and agreement and execution decisions. A role correctly retained its own judgment yet withheld approval of the higher-total proposal; that vote was faithfully handled downstream. In the matched approval-state comparison, one missing vote was recorded as an explicit hold even though every agreement and execution decision was appropriate.

The condition with comparison fields produced two more correct responses than DIRECT on previously used cases, while both conditions were entirely correct on new cases. Within this setting, roles preserved their own judgments while following the joint criterion and, once approval was complete, accepting assignments that differed from their own recommendations.

Evaluating a joint proposal appropriately and accurately carrying forward who approved what are separate requirements for cooperation. Connecting actual votes to their readout and comparing matched approval states provides a concrete way to inspect those requirements without hiding them inside final-answer correctness.

9. Data Availability and Research Disclosures

Original inputs, responses, saved notes, received votes, fixed designs, code, settings, hash lists, and reaggregated results are retained locally without a deletion deadline. The evidence index links the paper's tables to their sources; paper data provide machine-readable summaries; and evidence extracts contain the illustrated original inputs and responses. The MRSI repository distributes these materials with current reproduction instructions. A permanent archive DOI for this fourth paper has not been assigned. PDF links point to versioned GitHub resources; Markdown links refer to repository files.

The author is responsible for research design, implementation and Colab execution, review of results, and the manuscript. LLM assistance was used in design, implementation, research, analysis, and writing. Formal scoring used fixed code; LLM reading feedback is not treated as third-party rescoring. Verification was conducted within the same project.

The materials comprise fictional roles and numerical experiences. No human participants were recruited and no personal responses were collected. No external services were operated and no real equipment was allocated. The repository provides Apache 2.0 license information and citation metadata. Final funding and competing-interest declarations remain subject to the author's confirmation.

Appendix A. Cases, Replicates, and Response Counts

Table A1. Trial roles and counting units

ID	Descriptive name	Responses collected	Relationship between cases and replicates	Fixed diagnostic
TB01	Connected path carrying actual votes forward	18 formation + 36 voting + 72 readout = 126	Three cases × two replicates; the two higher/lower paths share formation	FAIL: one vote
TVR01	Repetition of fixed voting inputs	54 votes	18 inputs from TB01's first replicate × three new replicates	FAIL: two votes
TCV01	Comparison fields on previously used cases	108 votes	Same 18 inputs × two conditions × three replicates = 54 matched pairs	FAIL: two DIRECT votes
TCV02	Comparison fields on new cases	9 formation + 108 voting = 117	Three new cases; nine formation responses reused for 18 voting inputs across two conditions and three replicates	PASS_IN_FIXED_COMPARISON
TAR01	Three individual approval states	54 readouts	Three TCV02 cases; 27 inputs × two replicates = 18 matched triplets	FAIL: one approval-state response

The total of 459 responses is an inventory of saved generations, not a performance aggregate over 459 independent samples. All five trials completed their planned counts and every response ended with EOS. There were zero unresolved responses, zero format repairs, and zero regenerations to improve scores. An initial TB01 attempt stopped before any generation because the input checker incorrectly flagged the official template's handling of trailing newlines. That original record is retained; TB01 in this paper denotes the completed run after the input check was corrected.

Table A2. Predictions and the two candidates for all six cases

Case	A (PAPER, DIGITAL)	B (PAPER, DIGITAL)	C (PAPER, DIGITAL)	Higher-total candidate (A/B/C); total	Lower-total candidate (A/B/C); total
T01	34,22	27,23	18,30	PAPER / DIGITAL / DIGITAL: 87	DIGITAL / PAPER / DIGITAL: 79
T02	31,17	24,33	26,29	PAPER / DIGITAL / PAPER: 90	PAPER / PAPER / DIGITAL: 84
T03	26,22	19,29	32,21	DIGITAL / DIGITAL / PAPER: 83	PAPER / DIGITAL / DIGITAL: 76
N01	24,29	24,35	32,16	PAPER / DIGITAL / PAPER: 91	DIGITAL / PAPER / PAPER: 85
N02	18,33	28,23	31,20	DIGITAL / DIGITAL / PAPER: 87	DIGITAL / PAPER / DIGITAL: 81
N03	20,33	28,17	24,30	DIGITAL / PAPER / PAPER: 85	PAPER / PAPER / DIGITAL: 78

Values are predictions per 40 people, aggregated from each role's two original experience sessions. The role whose own recommendation differs from its assignment in the higher-total proposal is B in T01, C in T02, A in T03, A in N01, B in N02, and C in N03. Old and new cases share the same template; role names and numerical materials were not fully crossed.

Appendix B. Output Contracts and Metric Definitions

B.1 Stored Fields

All schemas reject unknown additional fields. The own-information structure is common across the three stages: `holder` is A/B/C, `recommendation` is PAPER/DIGITAL, and `evidence_ids` is an array of experience IDs. The evidence is scored against the complete set of the role's own experience IDs, irrespective of array order.

Table B1. Output structure by stage

Stage	Required top-level fields	Additional scoring targets
FORM (own-note formation)	`own`	Holder, recommendation, own evidence
VOTE (DIRECT)	`own`, `vote`	`vote.proposal_id` and `decision`
VOTE (COMPARE)	`own`, `comparison`, `vote`	`candidate_id`, `other_candidate_id`, `relation`, and their consistency with the vote
READ (readout)	`own`, `approval_state`, `agreement`, `joint_action`	Three roles' vote states, agreement, own assignment or hold

`approval_state` is an object with fixed keys A/B/C. Its value schema allows ACCEPT, HOLD, MISSING, and WRONG_PROPOSAL. WRONG_PROPOSAL denotes a vote on a different proposal, but that condition was not included in this paper's experimental cases.

B.2 Deriving Expected Answers From Received Votes

The following pseudocode summarizes the scoring rule. Expected own-information fields are computed separately from original experiences.

```
for role in [A, B, C]:
    receipt = that role's receipt in the current input
    if receipt is absent:
        expected_state[role] = MISSING
    elif receipt.proposal_id != current_proposal_id:
        expected_state[role] = WRONG_PROPOSAL
    else:
        expected_state[role] = receipt.decision

agreed = all(expected_state[role] == ACCEPT for role in [A, B, C])
expected_agreement = AGREED if agreed else PENDING
expected_action = current_assignment[reader] if agreed else HOLD
```

This rule is deterministic. In the experiment, its answers were used to verify the model's state, agreement, and action fields, rather than substituted into the readout responses.

B.3 Success for Responses, Matched Triplets, and Complete Cases

A READ response is fully correct when all six dimensions—three own-information dimensions, approval state, agreement, and execution—are correct. A TAR01 matched triplet contains the three states for the same case, reader, and replicate; all six dimensions must be correct in each of its three responses. Aggregating further over all three readers for each case × replicate yields full correctness in 5/6.

Despite its name, `agreement_execution_all_states` in TAR01's fixed summary JSON includes **three dimensions: approval state, agreement, and execution**. Its value is 17/18. The paper's "agreement and execution only: 18/18" is a different, post hoc supplementary metric restricted to two dimensions. Neither the formal key nor the scoring rule was changed.

Appendix C. Execution Settings and Reproducibility Scope

Table C1. Common settings

Item	Value
Model	`Qwen/Qwen3.5-27B`
Fixed revision	`fc05daec18b0a78c049392ed2e771dde82bdf654`
Weights and attention	BF16, SDPA
Thinking	`enable_thinking=false`
Sampling	`do_sample=true`, temperature 0.7, top-p 0.8, top-k 20
Other generation settings	Repetition penalty 1.0, `use_cache=true`
Length limits	8,192 input tokens; 1,024 generated tokens
Dependencies	Transformers 5.17.0, Accelerate 1.15.0
GPU	RTX PRO 6000 Blackwell Server Edition
Execution date	October 13, 2026

Seeds and execution orders were fixed before each trial and retained in the design files and response records. For the illustrated TAR01 case N01/A, the first-replicate seed was 230410795 and the second was 2039158309. The same seed was used for all three states within a replicate. A common seed does not guarantee identical responses to different inputs, nor does fixing seeds guarantee bitwise reproducibility across GPUs or libraries.

Outputs underwent strict JSON structure validation before scoring. The parser includes handling for known outer wrappers, but every response in the five completed trials could be scored without changing either its content or its outer form. No extra attempts were added to improve response scores.

Appendix D. Evidence Entry Points

The task materials, scoring rules, and counting units needed to interpret the results are described in Sections 3–4 and Appendices A–C. The following resources support detailed checks.

Item to check	Resource and contents
Table values and costs	Paper data: extracts from fixed reaggregations and original response times and token counts
Examples in Sections 5.1 and 5.2	Evidence extracts: actual inputs, original responses, scores, and archive locations
All responses and designs fixed before execution	Evidence index: original ZIP archives, fixed designs, and analysis code for the five trials
Scope of related-work claims	Research memo (Japanese): checked primary-source passages and claims used in this paper

The paper-data file supports reproducing the extraction of table values. It does not replace the original ZIP archives containing responses, inputs, and execution code. Verification from original archives and extraction into manuscript tables remain separate entry points.

References

[1]Lars Benedikt Kaesberg, Jonas Becker, Jan Philip Wahle, Terry Ruas, Bela Gipp. 2025. *Voting or Consensus? Decision-Making in Multi-Agent Debate.* Findings of ACL 2025, 11640–11671. ACL Anthology / DOI:10.18653/v1/2025.findings-acl.606.

[2]Justin Chen, Swarnadeep Saha, Mohit Bansal. 2024. *ReConcile: Round-Table Conference Improves Reasoning via Consensus among Diverse LLMs.* ACL 2024, 7066–7085. ACL Anthology / DOI:10.18653/v1/2024.acl-long.381.

[3]Hyeong Kyu Choi, Xiaojin Zhu, Yixuan Li. 2025. *Debate or Vote: Which Yields Better Decisions in Multi-Agent Large Language Models?* NeurIPS 2025. Published version; version checked: arXiv:2508.17536v1.

[4]Xuhui Zhou et al. 2024. *SOTOPIA: Interactive Evaluation for Social Intelligence in Language Agents.* ICLR 2024. Version checked: arXiv:2310.11667v2.

[5]Federico Bianchi, Patrick John Chia, Mert Yuksekgonul, Jacopo Tagliabue, Dan Jurafsky, James Zou. 2024. *How Well Can LLMs Negotiate? NegotiationArena Platform and Analysis.* arXiv:2402.05863v1.

[6]Qizhi Chu, Zekai Yu, Sijie Wen, Yang Liu, Chen Qian, Cheng Yang, Chuan Shi, Zhiyuan Liu. 2026. *MASBench: Benchmarking LLM-based Multi-Agent Collaboration under Partial Observability.* arXiv:2610.04672v1.

[7]Wael Albayaydh, Rui Zhao. 2026. *Do LLM Agents Negotiate Rationally? A Mechanism-Design Framework for Verifiable Multi-Agent Interaction over A2A/MCP.* arXiv:2608.14613v1.

[8]Kunlun Zhu et al. 2025. *MultiAgentBench: Evaluating the Collaboration and Competition of LLM agents.* ACL 2025, 8580–8622. ACL Anthology / DOI:10.18653/v1/2025.acl-long.421.

[9]Mert Cemri et al. 2025. *Why Do Multi-Agent LLM Systems Fail?* Version checked: arXiv:2503.13657v3.

[10]Shunyu Yao, Noah Shinn, Pedram Razavi, Karthik Narasimhan. 2024. *τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains.* arXiv:2406.12045v1.

Failure Analysis and Design Principles for Multi-Role LLM Systems

Boundaries of Records, Approval, and Joint Use in Two- and Three-Role Tasks

Kentaro Sato
Independent Researcher
Contact: info@metaclan.jp (email)

October 13, 2026 | English version v0.3

Repository and research materials | Apache License 2.0

Abstract

For multiple LLM roles to work together, they must connect shared records, proposal approvals, and assignments for execution while preserving each role's information and judgment. This report organizes failure observations and design principles from synthetic tasks that ran the same model as two or three roles in separate contexts. The tasks involved saving and updating customer preferences and venue observations, or deriving own recommendations and joint equipment allocations from each role's experience. We examined paths that passed generated notes or actual votes into subsequent inputs, together with matched comparisons that changed only part of the supplied material.

In two-role experiments, we observed incorrect provenance carried forward with correct answers, rollback after adoption of a newer state, and acceptance of a suboptimal proposal despite a correct own recommendation. In three-role experiments, subsequent responses faithfully used an actual vote that incorrectly deferred an appropriate proposal; in another case, a response correctly deferred group action while describing a missing vote as an explicit hold. Correct joint answers also concealed defects in private records, while end-to-end outcomes concealed successful local repairs.

We organize these findings into six observations and seven design principles. Central requirements are to preserve content, attribution, and provenance together; update according to target and purpose; identify the fields and stages covered by a control; separate own judgments from joint assignments; and record proposal assessment, actual votes, group agreement, and execution authorization separately. We present a reference architecture incorporating reasons for deferral and component-level evaluation, alongside standalone executable examples. Supporting both role-specific and joint activity can be framed as preserving, across stages, whose information or expressed position justifies which next action, rather than making all roles give the same answer.

Keywords: LLM agents; multi-role systems; failure analysis; shared memory; state updates; individual approval; group agreement; design principles

1. Goal: Preserve Each Role's Judgment While Working Together

Consider several reception staff members who remember their respective customers' preferences and jointly decide how to provide guidance at a venue. The customers may have different preferences. Assigning responsibilities for the joint task need not rewrite each role's past experience to match its assignment. At the same time, relevant information and judgments must remain open to revision when new experience or a legitimate customer change arrives.

This goal combines two pairs of requirements: retention with change, and private with shared activity. Simply fixing everything that should be retained prevents adaptation. Simply sharing more information leaves unclear whose information is being used and for what purpose. The required structure preserves attribution and purpose, updates the parts for which a change is justified, and establishes a policy for joint action.

The central proposition of this report is that **local correctness is specific to its object and criterion**. A correctly selected answer can carry an incorrect origin ID. A process that reads a record accurately can read a stale record accurately. Faithful execution of an agreement can faithfully implement a poor agreement. Group-level deferral can be correct even when the account of who expressed a hold is wrong. Connecting locally successful stages requires examining the records passed between them and the requirements assumed by the receiving stage.

For joint judgment in particular, we distinguish **whether a proposal meets the objective, whether each role actually approved it, whether the agreement rule is satisfied, and whether execution is authorized**. Here, "multi-role systems" refers to the connections between roles examined in the two- and three-role experiments.

This report makes three contributions. First, it organizes experimental cases by the location of a defect and the path along which it is carried forward. Second, it develops seven implementable design principles from these observations. Third, it presents a reference architecture and an evaluation approach that distinguish local improvements, remaining defects, and excessive deferral.

2. What the Roles Were Asked to Do

2.1 Running the Same Model as Multiple Roles

The two-role experiments invoked the same LLM separately as reception staff members A and B; the three-role experiments used A, B, and C. Each call began in a fresh input context containing the role, the current materials, and the required saved notes. Memory therefore refers to external records explicitly passed into later calls, rather than conversation history retained internally by the model. The staff members are model roles supplied with role labels and materials, not human participants.

The processing responsibilities were distinct. The **researcher** specified materials, instructions, and scoring criteria; the **LLM** generated update proposals and judgments; and the **experimental program** checked, saved, and passed those outputs onward. The term *storage component* refers to the part of that program responsible for saving records. In conditions where it copied input rows, the model proposal and the artifact produced by copying were saved separately. Fixed-rule code scored correctness, and structurally usable but substantively incorrect responses were still passed to subsequent stages. This made it possible to examine where errors arose and what was carried forward unchanged.

The report covers two tasks: record management with two roles, and joint judgment with two or three roles. Both use paper and digital guidance, but their correctness rules differ. The first selects **a medium accessible to everyone**; the second selects **an equipment allocation that maximizes the predicted total number of successful outcomes**.

2.2 Record Management: Customer Preferences and Venue-Wide Guidance

Roles A and B each serve a different customer and cover a different part of the venue. Each role records whether its customer wants a dog or cat design in a private note. It also receives observations indicating whether everyone in its area can use paper guidance and whether everyone can use digital guidance. All materials describe fictional situations constructed by the researcher.

Table 1. Three information fields in the record-management task. "Own observation" belongs to the staff role, which is distinct from the customer assigned to that role.

Information field	Recorded information	Purpose
Assigned-customer information	The role's customer's dog-or-cat preference and provenance; a revision number is also included in preference-update trials	Select the design to prepare for that customer
Own observation	Paper/digital accessibility in the role's area, observer, revision, and provenance	Retain and update the role's observation and pass it to the other role
Shared observations	Saved observation rows for both A's and B's areas	Select guidance for the venue as a whole

The private note contains separate fields for assigned-customer information and the role's own observation. The shared note contains copies of both roles' observations. A shared note is a record received and used by each role, not a single internal memory automatically kept identical across the two roles.

Under the task rule, if exactly one medium is accessible to everyone in both areas, that medium is the correct joint choice. For example, if both media are accessible to everyone in area A but only digital guidance reaches everyone in area B, the correct choice is digital. If zero or two media meet the criterion, the task requires deferral. The design prepared for each customer is determined independently by that customer's preference.

The researcher supplied new observations, observations saved by the other role, and delayed old notices. The LLM proposed note updates, and later calls answered questions about the customer design and joint guidance using the saved notes. In the field-level comparison described below, the program calculated joint guidance from four values read out by the LLM: A/B × paper/digital. Each case specifies this division of responsibilities.

2.3 Joint Judgment: Own Recommendations and Equipment Allocation

In the second task, each role receives several experiences stating how many people were guided using each medium and how many were successful. For its own task, the role can freely use either medium and should recommend **the medium with the higher success rate pooled over all of its experiences**. Earlier experiences remain part of the calculation; this is not a task of selecting only the latest row.

For the two-role joint task, the available equipment is limited to one paper guidance station and one digital terminal. Even if both roles recommend digital for their own task, one must use paper in the joint assignment. Each role's success rate is used to calculate the predicted number of successes when guiding 40 people in its area. The totals for A-paper/B-digital and A-digital/B-paper are compared. The joint adoption criterion is to maximize this total while respecting the equipment constraint.

In the two-role experiments, after each role recorded its own recommendation, the researcher supplied a fixed initial joint proposal. The other role responded with acceptance, a counterproposal, or deferral. **Only a counterproposal triggered one recheck by the initiating role.** The experimental program constructed a joint record from these responses and finally asked both roles to report their own recommendation and joint assignment. Execution here means reporting an allocation, not operating actual equipment or serving actual customers.

Joint confirmation and subsequent use had separate scoring criteria. At confirmation, a role should accept an optimal feasible proposal and respond to a suboptimal or resource-violating proposal with an optimal counterproposal or deferral. At subsequent use, a role should follow an approved agreement that respects the equipment

constraint. Reoptimization was not required at the use stage. Thus, accepting a suboptimal proposal could be incorrect while subsequently following that agreement could satisfy the use-stage criterion.

In the three-role task, one medium is assigned to one role and the other to two roles. The experimental program supplies two feasible candidates and their totals calculated from the roles' success rates. The LLM should accept (ACCEPT) the higher-total candidate and defer (HOLD) the lower-total candidate. The task does not require searching all possible allocations. At the vote-readout stage, three ACCEPT votes for the same proposal yield agreement (AGREED); otherwise, the state is pending (PENDING). A role reports its joint assignment only when agreement has been reached.

Three-role readout distinguishes MISSING, where no vote is supplied, from HOLD, where an explicit hold vote is supplied. Both prevent unanimous agreement, but they represent different expressed facts. A favorable proposal total does not justify filling in an absent vote or changing an actual HOLD to ACCEPT.

Vote correctness in this report is evaluated against the specified numerical objective. Whether deferral is justified in a real meeting is a separate question involving the participant's circumstances and the decision-making institution.

2.4 How the Evidence Is Organized

The provenance, copying, and revocation trials in the record-management task used `gpt-5.6-luna`. The field-level control, update, and two- and three-role joint-judgment trials used Qwen3.5-27B. Within each trial, all roles used the same model. All tasks were short synthetic tasks in Japanese. Each case specifies what was held fixed and what was varied.

This technical report organizes the experimental records of four reports by the author [1–3,6] around failure analysis and design. The six observations and seven principles emerged from this synthesis; they were not hypotheses fixed before the entire experimental series. Each trial retains its model, instructions, scoring scope, and denominators. Tasks, manipulations, and results are explained within this report. Appendix C maps them to original trial identifiers, and Supplement S1 links to the evidence.

3. Units of Failure Analysis: Record Fields and Stage Boundaries

3.1 Distinguish Private Records, Actual Votes, Agreement, and Execution Authorization

A *contract* in this report is an explicit requirement for a field or processing stage, not a legal contract. Whether an observation value is correct, whether its origin ID is correct, whether an old notice may be adopted, and whether a proposal may be used as an agreement are different contracts.

Four terms are used throughout. An **origin ID** identifies the source material in which a fact or observation was first recorded. A **revision** is a researcher-assigned number used to compare versions for the same target. A **current evidence reference** identifies the location of an input used directly in the present call. If an observation arrives through a shared note, its origin ID still identifies the original observation, while its current evidence reference points to the corresponding row in that note. A **record field** is a logical information field within a note, not a separate physical file.

Table 2. Objects and requirements to specify before connecting stages.

Object	Relationship to preserve	Grounds for an update
Role-specific assigned information	Correspondence among the target person, content, provenance, and valid revision	A legitimate change concerning that target
Own and shared observations	Correspondence among observer, values, origin ID, and revision	Comparable new information about the same observation target
Own judgment	Recommendation consistent with the role's experience and its intended use	New experience or evidence that the earlier judgment was incorrect
Joint proposal	Correspondence among the task, candidates, constraints, and evaluation criterion	New information, changed conditions, or a counterproposal
Individual vote	Who expressed what about which proposal	A new response from the same role whose target can be identified
Group agreement	Correspondence between received votes and the specified agreement rule	Receipt of the required roles' votes or an update to existing votes
Execution authorization	Correspondence among the target proposal, agreement, task constraints, and adoption criterion	Satisfaction of the conditions required for execution
Subsequent use	Correspondence between the supplied records, valid agreement, and use outcome	An applicability judgment based on the state and purpose at use time

Assigned information and own observations remain separate fields even within the same note. Sharing the same observations does not require the roles to make identical own judgments. Conversely, identical answers are appropriate when each role's legitimate information supports the same conclusion.

The reference design records the fact of unanimous approval separately from the controller's authorization to execute a proposal. If all roles approve a suboptimal proposal, their approval remains recorded while execution is deferred on the basis of proposal assessment. This proposed additional check is distinct from the original two-role use-stage rule, which scored faithful use of an agreement as correct.

3.2 Separate Defect Origin, Propagation, and Evaluation

Figure 1 summarizes how the researcher traced defects. Although checking and transformation differed between trials, each distinguished the model proposal, the artifact actually saved, the next input, and the subsequent response. When votes were passed onward, generated votes were matched to the receipts actually included in the readout input.

Failure descriptions have three layers: a mismatch in information or judgment; the path carrying it through storage, exchange, and use; and the aggregation of evaluations that can obscure the mismatch. The third layer also includes cases in which an aggregate metric obscures a successful local repair.

4. Failure Analysis: What Was Tested and What Each Actor Got Wrong

Each case first states the question and manipulation, then follows the LLM response, storage processing, and subsequent use. It ends with the design distinction supported by the case. The cases are organized by failure and evaluation issue, not by experimental chronology.

4.1 Incorrect Provenance Is Carried Forward Alongside Correct Answers

What was tested. The researcher provided correct initial materials and examined whether both roles retained content–provenance correspondence after integrating them into private and shared notes. Two methods trans-

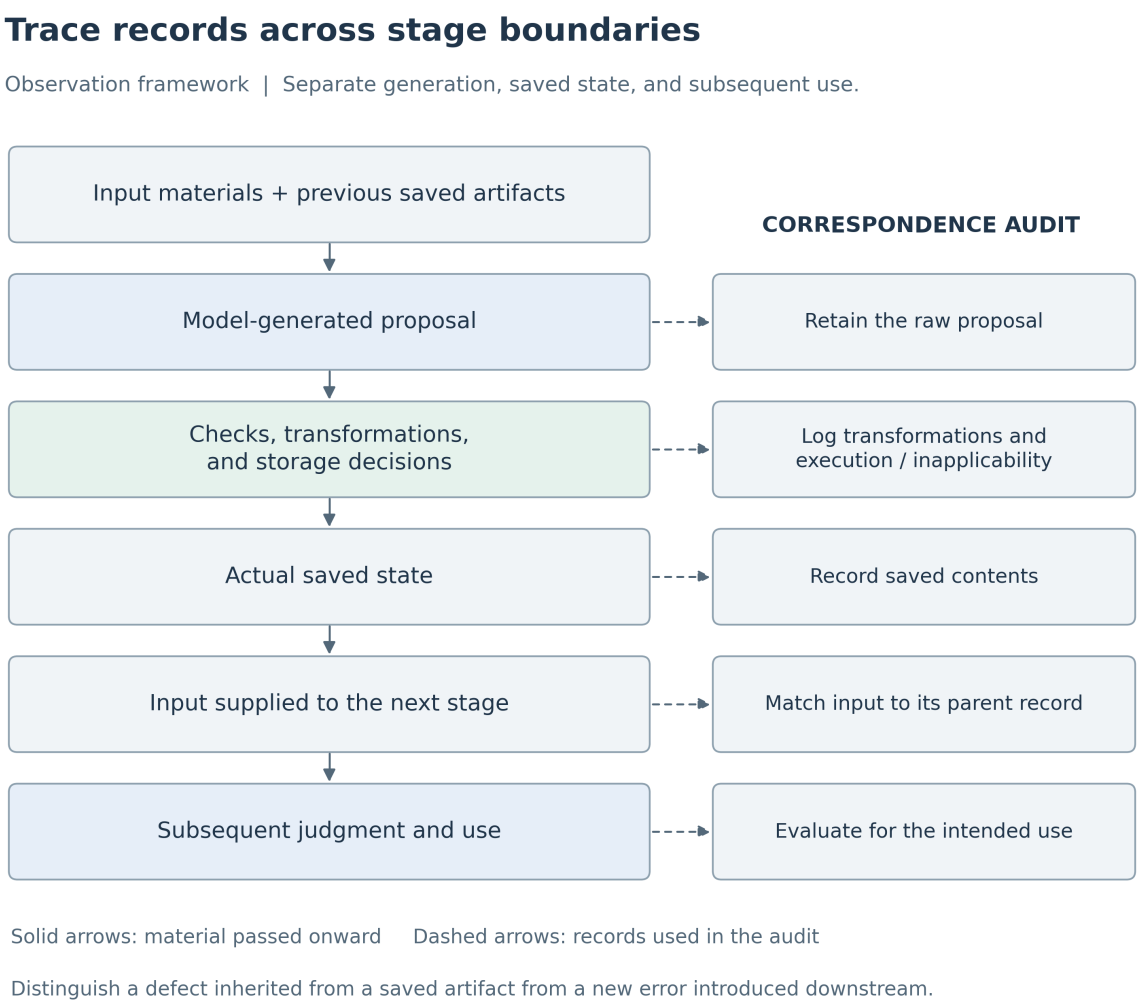


Figure 1. An observation framework for locating defects and tracing what is carried forward. Distinguishing proposals from actual saved artifacts identifies where a defect arose and which transformations it passed through. The same correspondence also distinguishes errors newly introduced downstream.

ferred shared observations. Under *rewriting*, the LLM rewrote the person, values, revision, and origin ID. Under *selection plus copying*, the LLM identified an input row and the storage component copied it verbatim. In both methods, the LLM still wrote the private note and declared current evidence references. The storage component saved the selected row even if the selection was wrong.

Both methods received the same 64 previously unused groups. A group comprised initial materials for both roles. Under each method, both roles saved notes and then answered subsequent questions from those notes. End-to-end success required all content, provenance, references, and judgments to satisfy their criteria across these four responses.

What happened. In one group, role B's customer wanted a cat design, with origin ID `RAD9D1FEC60F1FA2`. Under selection plus copying, the LLM correctly recorded role B and the cat preference in the private note, but omitted the final `2` from the origin ID, writing `RAD9D1FEC60F1FA`. Because mechanical copying covered shared observation rows, this private field was saved as written by the model. The experimental program passed that saved note into the next call.

The subsequent role B correctly chose the cat design and paper guidance. Paper was accessible to everyone in both areas, whereas digital was not, making paper the correct joint medium. However, role B again output the truncated origin ID as support for its correct answer.

Correct input: B's customer wants a cat; origin ID ends in "…FA2."
Private field written by the LLM and saved: B's customer wants a cat; origin ID ends in "…FA."
Next LLM response: cat/paper is correct; the origin ID still ends in "…FA."

This is not an inference that the later call independently dropped the same character. The saved incorrect ID was present in the actual next input, and the response retained a matching ID. Under rewriting with the same initial materials, this customer's origin ID was correct at both storage and subsequent use.

Across all 64 groups, selection plus copying met the provenance-inclusive criteria for 256/256 shared observation rows. Both methods achieved end-to-end success in 59/64 groups: rewriting had four failures and one unresolved outcome, while selection plus copying had five failures. The unresolved outcome arose from a response-format defect and remained unresolved in the formal results. Failures under selection plus copying occurred in private-information writing or subsequent readout, outside the mechanically copied fields. Defining the method difference as the selection-plus-copying rate minus the rewriting rate, the envelope of conservative 95% intervals across all completions of the unresolved outcome was −17.36 to 16.60 percentage points. End-to-end superiority was not established. Under the prespecified missingness rule, the primary p-value was not calculated.

What this establishes. Producing a correct answer and associating content with correct provenance are distinct tasks. Even when shared rows are copied correctly, private fields and later responses that rewrite provenance need separate checks. The design object is the correspondence among person, content, revision, and provenance through to use, rather than values or IDs protected in isolation.

4.2 A Role Adopts a New Preference, Then Rolls It Back After an Old Notice

What was tested. The researcher examined whether the targeted role could update a customer's changed preference and retain it when an old notice arrived later. Eight new cases were generated twice each, giving 16 trajectories. In each trajectory, one role received a new customer preference. Its saved note was then passed into the next input together with an old notice about the same customer. The other role was checked for retention of its unaffected customer's information.

The storage program applied per-observer latest-revision selection to paper/digital observation fields. It saved the customer-preference field as written by the LLM. Thus, although the notes were subject to a "retain the latest revision" operation, the preference field being changed lay outside that operation's scope.

What happened. In one case, role A's note contained a cat preference at revision 76. The researcher supplied a notice changing it to dog at revision 77. Role A correctly wrote dog, revision 77, and the new origin ID, and the storage component saved that state. Next, the researcher supplied the saved dog/revision-77 note together with the delayed cat/revision-76 notice. Role A reverted its customer field to cat, revision 76, and the old origin ID. After that proposal was saved, the final role A call selected the cat design.

The final call did not misread the supplied note: the note already said cat, and the role answered accordingly. The failure occurred earlier, when the update replaced newer information present in the input with the content of the old notice.

Initial adoption of the new preference succeeded in 16/16 trajectories. The unaffected role's customer information was also retained in 32/32 evaluations across the two update points. After the old notice, however, only 2/16 trajectories retained the new revision; in 14/16, the preference, revision, and origin ID rolled back. In all 14 cases, the correct newer state had been saved immediately beforehand and was included in the input processing the old notice.

What this establishes. Adopting new information, preserving unaffected information, and maintaining an adopted update are separately measurable capabilities. More accurate readout alone would not repair this path.

The updating role needs a way to distinguish a notice arriving now from its content being valid now. The outputs do not establish how the internal revision comparison failed.

4.3 Joint Answers Can Be Correct While Own Observations Remain Stale

What was tested. The researcher examined what was repaired when storage processing was applied to different fields. Eighteen materials were generated twice each, yielding 36 model-proposed note updates. The own observation was already current in the initial input; the task was to preserve it while updating shared observations. Ten materials required updating the shared field from a new notice, and eight required retaining the state despite an old notice.

The experimental program **branched each identical update proposal into four conditions**. One saved observation fields without intervention. The others applied automatic processing to shared observations only, own observations only, or both. Processing first separated input candidates by observer A/B, selected that observer's highest-revision row, and copied the person, two values, origin ID, and revision together. All conditions retained the model-written customer-preference field.

Each branch's saved note was supplied to a subsequent LLM call, which read the four A/B × paper/digital values from the shared note. The program calculated joint guidance from that actual response. Own-observation correctness was assessed in the private-note field; shared-observation correctness was assessed in the shared-note rows, both against the latest input information. Generations were reused only when branches produced exactly identical subsequent inputs. Consequently, 144 branch conditions required 55 actual readout responses.

What happened. Table 3 reports the four branches. End-to-end success counts update proposals satisfying all requirements, including storage in the relevant fields and correct joint guidance.

Table 3. Field-level storage controls applied to the same 36 update proposals. These are 18 materials × two repetitions; the four branches are not 144 independent materials.

Storage control	Correct own observation	Correct shared observations	Joint guidance matching the latest state	End-to-end success
None	32/36	25/36	28/36	25/36
Shared only	32/36	36/36	36/36	32/36
Own only	36/36	25/36	28/36	25/36
Both	36/36	36/36	36/36	36/36

With shared-only control, the storage component correctly repaired the shared rows, and subsequent joint guidance was correct in 36/36 proposals. Yet the untouched own-observation field retained four stale records from the model proposals. Because joint guidance was calculated from the shared note, it could be correct despite these private-note defects.

Conversely, own-only control repaired all four own-observation defects, but end-to-end success remained 25/36. These four proposals also contained shared-observation defects, so fixing only the own field did not satisfy every condition. Controlling both fields yielded end-to-end success in 36/36 proposals.

What this establishes. Inferring that the entire record is correct from a correct joint answer misses defects in the own field. Inferring that a repair did not work because the aggregate score did not change misses successful own-field repairs. An all-conditions-required metric can obscure partial repair by definition. This comparison additionally located overlapping defects in actual model proposals and related the control scope applied to the same proposal to its downstream result.

4.4 Preserving Own Judgment Does Not Ensure a Correct Decision on a Joint Proposal

What was tested. In the two-role joint-judgment task, each role first formed a recommendation from its experience and then evaluated a joint proposal. One condition supplied an own-experience summary from the formation stage and provided both candidate totals at joint confirmation. The comparison examined whether roles could preserve their recommendations and accept an appropriate joint proposal when the information required for calculation was available.

What happened. In one previously unused numerical material, predicted successes for 40 people per role were as follows.

Role	Predicted successes with paper	Predicted successes with digital	Appropriate own recommendation
A	17	27	Digital
B	18	20	Digital

Both roles should recommend digital for their own task. With one unit of each medium, however, A-digital/B-paper yields 27 + 18 = 45, exceeding the 17 + 20 = 37 from A-paper/B-digital. Role B can appropriately recommend digital from its own experience while accepting paper as its joint assignment.

The researcher supplied the total-37 allocation to role B as the initial proposal. The input contained experience records for both roles, role B's own correct judgment note, B's own-experience summary, and the joint candidate table showing 45 and 37. Role B correctly reported digital as its own recommendation but accepted the total-37 proposal in its joint response. The program saved that acceptance in the joint record.

Because B did not counterpropose, the initiating role's recheck was not called. This followed the experimental rule that rechecking occurs only after a counterproposal; it was not an execution failure. At final use, A reported digital as its own recommendation but paper as its joint assignment, while B reported digital for both. The roles faithfully used the poorer total-37 agreement. Their references were also correct, and subsequent use satisfied the scoring contract described in Section 2.3.

The researcher then fixed this confirmation input and generated three new responses with fresh random seeds. In all three, B retained its digital recommendation and accepted the total-37 proposal. This was one case within a trial repeating all 48 confirmation inputs three times, rather than a rerun of a single outcome-selected case. The original own notes remained fixed; newly generated judgments were not connected to confirmation. These repetitions did not include subsequent use.

Three roles also incorrectly deferred an appropriate proposal. In a connected three-role trial, the researcher generated each of three materials twice, linking votes on higher- and lower-total proposals to readout of the actual votes. In one case, role C's own predictions were 26 for paper and 29 for digital. The joint candidates were A-paper/B-digital/C-paper with a total of 90, and A-paper/B-paper/C-digital with a total of 84. Role C correctly retained digital as its own recommendation but returned HOLD for the total-90 proposal.

The other two roles had returned ACCEPT for that proposal, giving A-accept/B-accept/C-hold. When A, B, and C subsequently read these actual votes, all three reported C as HOLD, group agreement as PENDING, and execution as HOLD. Across the trial, vote correctness was 35/36 and readout correctness was 72/72. In the second repetition with the same input for C's vote, C returned ACCEPT. Following agreement, C retained its own digital recommendation while reporting paper as its joint assignment.

What this establishes. Accepting a suboptimal proposal in the two-role task and deferring a higher-total proposal in the three-role task are different errors at the proposal-assessment stage. Faithful subsequent use of either record still counts as success under the downstream criterion. A path that rechecks only counterproposals does not inspect a directly accepted proposal through that recheck. The design needs to separate proposal assessment from recording what each role actually voted, and to apply common checks to the paths leading to execution.

4.5 Correct Group-Level Deferral Can Coexist with a Misrecorded Individual Position

What was tested. The researcher fixed the three roles' own notes and the higher-total joint proposal, varying only one role's vote. For each of three materials, the target vote was removed from the input, replaced with an explicit HOLD, or left as the original ACCEPT. The other two roles' ACCEPT votes were common to all conditions. The HOLD condition was constructed by the researcher, not newly generated by the model.

Each of A, B, and C read each state twice: three materials × three states × three reading roles × two repetitions, yielding 54 responses. These were two repetitions of 27 inputs using the same own notes and proposals. The three states were compared in 18 matched triplets sharing material, reading role, and repetition. Each response was scored for its own-judgment fields, per-role approval states, agreement, and execution judgment. The trial did not evaluate effects of saving these readout responses as new state and feeding them into further stages.

What happened. In one case, A's own predictions were 24 for paper and 29 for digital, so its own recommendation was digital. The joint candidates were A-digital/B-paper/C-paper with a total of 85 and A-paper/B-digital/C-paper with a total of 91. Role A read an input containing only B's and C's ACCEPT votes for the total-91 proposal. Because A's vote was absent, the correct approval states were A=MISSING and B/C=ACCEPT.

Role A correctly reported its own digital recommendation and references to its experience, but described its own approval state as HOLD. Reporting PENDING for the group and HOLD for execution was appropriate. In the second repetition with the same input, A correctly reported MISSING. Under unanimous approval, A retained its own digital recommendation while proceeding to its paper joint assignment.

Table 4. Readout comparison with three approval states. Each condition contains 18 responses. Approval-state success requires all three roles' states within a response to match the input.

Evaluation target	Missing vote	Explicit hold	Unanimous approval
Own-judgment holder, recommendation, and evidence (each axis)	18/18	18/18	18/18
Approval states of all three roles	17/18	18/18	18/18
Group agreement	18/18	18/18	18/18
Execution judgment	18/18	18/18	18/18
All six axes	17/18	18/18	18/18

Own judgment, agreement, and execution judgment were each correct in 54/54 responses; approval states were correct in 53/54. All six axes succeeded across all three states in 17/18 matched triplets. The observed defect was one reinterpretation of an absent vote as an explicit hold. No response marked an unapproved role as having approved.

What this establishes. A missing vote and an explicit hold both map to the group-level decision that the group cannot yet proceed. Appropriate action alone therefore does not establish faithful recording of an individual's expressed position. Even when both lead to the same stop decision, the design should preserve vote absence and an explicitly expressed hold as separate states. This allows a later stage to identify whose response is still needed and what response has actually been expressed.

4.6 Identical Overall Failures Can Hide Different Defects, and Final Answers Can Hide Earlier Errors

What was tested. The researcher examined how own and joint judgments varied across repetitions of fixed inputs. The case considered here came from a condition that did not add an own-experience summary at confirmation. Role B received all experience records and a table of joint candidate totals, together with an earlier model-written note incorrectly recommending paper for its own task. The correct recommendation from all

experience was digital. The initial proposal assigned the single digital terminal to both roles, violating the equipment constraint. The same input was generated three times.

What happened. In the first two responses, B corrected its old note and appropriately recommended digital, but accepted the proposal assigning the one digital terminal to both roles. In the third, B corrected the joint proposal to the appropriate A-digital/B-paper allocation, totaling 45, while retaining the incorrect paper recommendation in its own field.

Response to the same confirmation input	Own recommendation	Joint response	Both correct
Repetition 1	Digital: appropriate	Accepts digital for both roles: inappropriate	No
Repetition 2	Digital: appropriate	Accepts digital for both roles: inappropriate	No
Repetition 3	Retains paper: inappropriate	Counterproposes A-digital/B-paper: appropriate	No

The aggregate statement "failure in all three repetitions" is correct, but the same defect did not occur three times. Joint confirmation was inappropriate in the first two; own judgment was inappropriate in the third. The third response's counterproposal was not passed to the other role for rechecking in this repetition trial, so it was not counted as a final agreement.

The researcher also compared when own-experience summaries were supplied. One condition supplied them only at final use; the other supplied them from judgment formation onward, including confirmation and recheck. Both supplied the summary at use and joint candidate totals at confirmation and recheck. Each trial branched eight materials into three initial proposals, giving 24 paths. Both roles answered at the end of each path, yielding 48 use responses. The comparison was conducted once with previously used materials and once with unused numerical materials.

In both trials, own judgments and references were appropriate throughout 15/24 paths with use-only support and 24/24 with support from formation. Nevertheless, final use was appropriate in 48/48 responses under both conditions. With use-only support, the LLM sometimes corrected an erroneous intermediate own record using the experience and summary supplied in the final input. Scoring only the endpoint would conceal this difference along the path.

What this establishes. End-to-end success asks whether any required component failed; final-answer correctness asks whether the last response was appropriate. Both are useful, but defect locations and correction paths require separate records. Retaining component-level own fields, joint fields, and acceptance actions helps identify where a control should act.

5. Seven Design Principles for Connecting Private and Shared Activity

Three principles below include short Python examples expressing requirements as operations. Observation selection reuses the core of an existing procedure; recording joint assignments and separating approval from execution illustrate the proposed reference architecture.

In the code, `owner` and `role` identify roles, `revision` is an information revision, `origin_id` identifies provenance, and `allocation` specifies joint assignments. `deepcopy` copies a record without modifying the original. Vote states are MISSING/HOLD/ACCEPT, group agreement is PENDING/AGREED, and the controller's execution decision is DEFERRED/AUTHORIZED, each in a separate field. HOLD in Code example 1 denotes deferral of observation selection, not an individual's vote.

The caller checks input types, source identity, task scope, and the binding between contents and their IDs/versions in advance. The functions return candidate saved contents and decisions without writing to disk. Appendix D contains the complete code, including helpers, all inputs, and 18 tests. Tests of the design examples are distinct from the original LLM experimental results.

Principle 1. Treat Content, Attribution, and Provenance as a Unit

Requirement. After storage or exchange, information remains correctly associated with its target, origin, and valid revision.

Implementation. Represent observer, values, origin ID, and revision in one row and copy the selected row intact. Keep the origin ID separate from the reference to the current input. Specify the fields passed to other roles and the private fields used only by their owning role. Bind votes to their sender and target proposal, and match received original votes to the state representation derived from them. Record selection correctness separately from copying fidelity so that an incorrect source selection can be identified.

Verification. Compare saved rows with the selected sources and verify that subsequent inputs derive from the actual saved artifacts. If a later stage rewrites provenance, score that provenance separately. Selection plus copying in Section 4.1 exemplifies this division of responsibilities; defects remaining outside shared fields also delimit its coverage.

Principle 2. Identify the Update Target and Purpose Before Selecting Information

Requirement. Legitimate new information reaches the relevant field, while fields without grounds for change are preserved. Distinguish when a notice arrives from when its information is valid.

Implementation. First filter candidates by target person and information type, then compare revisions or validity intervals within that set. A task about the current state should use current information; a task about the past should use information applicable at that time. If the same target and revision have conflicting candidates, retain them and record what is needed to resolve the conflict. Do not compare A's revision number with B's.

Verification. Separately assess retention, adoption, maintenance after adoption, and preservation of unaffected fields. Per-observer latest-revision selection for observation fields was compared in Section 4.3. Extending that control to the customer field, same-revision conflicts, and notices with uncertain authority are subjects for further comparison.

Code example 1. Compare revisions within each observer and copy the selected row with its provenance. Reconstruction of an existing procedure's core.

The storage procedure in Section 4.3 selected the observer before comparing that observer's revisions. The following function illustrates selection and copying. `rows` contains observations of the same kind for the same task. Input-source matching, structural validation, and selection-source audit records from the original implementation are omitted here; the first two are preconditions. Missing candidates and same-revision conflicts, which the original implementation reported as exceptions, are represented by `HOLD` for illustration.

```
from copy import deepcopy

def select_observation(rows, owner):
    candidates = [r for r in rows if r["observer"] == owner]
    if not candidates:
        return {"status": "HOLD", "reason": "MISSING_OWNER"}
    by_version = {}
    for row in candidates:
        version = row["revision"]
        if version in by_version and by_version[version] != row:
            return {"status": "HOLD", "reason": "SAME_VERSION_CONFLICT"}
        by_version[version] = row
    latest = by_version[max(by_version)]
    return {"status": "READY", "row": deepcopy(latest)}
```

Input and output. For illustration, supply the following three rows in order. `ALL` means accessible to everyone, and `NOT_ALL` means not accessible to everyone.

Observer	Revision	Paper	Digital	Origin ID
A	2	ALL	NOT_ALL	OA2
B	99	ALL	NOT_ALL	OB99
A	3	NOT_ALL	ALL	OA3

`select_observation(rows,"A")` returns the **entire row** containing A, revision 3, paper `NOT_ALL`, digital `ALL`, and origin ID `OA3`. B's revision 99 is not an A candidate. A legitimate revision 4 for A would be adopted. Conflicting content or provenance within an A revision instead produces a hold. The original implementation also detects same-revision conflicts, but the original LLM comparison used consistent candidates; the conflicting inputs here test the illustrative code.

This function selects one observation row. It is not directly applicable to customer preferences or own judgments that aggregate all experience.

Principle 3. Map a Control's Field Coverage to Subsequent Uses

Requirement. The scope protected by a procedure can be identified by record field and receiving stage.

Implementation. For assigned information, own observations, shared observations, own judgments, per-role approval states, and joint agreements, list the generator, storage operation, and consumer. Bind feature descriptions such as "latest-revision selection" or "validation" to explicit fields. Save the model proposal, changes made by the procedure, and actual saved artifact separately.

Verification. Where conditions can share the same proposal, vary which fields receive the control and inspect local repairs and remaining defects. The four-way branch in Section 4.3 is an example. Measure defects in proposals separately from what would happen in a distinct continued run without the control.

Principle 4. Retain Own Judgments and Joint Assignments as Separate Artifacts

Requirement. Own recommendations correspond to the role's experience, while joint assignments correspond to the joint objective, constraints, and agreement. Taking a different joint assignment does not rewrite the role's experience to match it.

Implementation. Associate an own judgment with its role, purpose, and experience references. Associate a joint proposal with its task, candidates, constraints, and evaluation criterion. Supply computed own-experience summaries and joint candidate totals in separate fields. Specify the formation, confirmation, and use stages at which each is needed.

Verification. Construct inputs changing only own experience, only joint conditions, or both, and test whether the corresponding tasks change selectively. Include valid cases with matching recommendations, different recommendations, and an own recommendation different from the joint assignment. The timing comparison in Section 4.6 illustrates earlier support sustaining intermediate own judgments. In the three-role task, a digital own recommendation also coexisted with a paper joint assignment after agreement (Sections 4.4–4.5).

Code example 2. Record only an authorized joint assignment without changing the own judgment. Design example.

`state` is the role's own state, and `decision` is the controller decision from Code example 3. Whether the own judgment matches the experience is checked separately; this function adds only the joint assignment.

```
from copy import deepcopy

def record_assignment(state, decision):
    if decision["execution"] != "AUTHORIZED":
        return {"status": "DEFERRED", "reason": "NO_EXECUTION_PERMISSION"}
    proposal = decision["proposal"]
    updated = deepcopy(state)
    updated["joint_assignment"] = {
        "medium": proposal["allocation"][state["role"]],
        "proposal_id": proposal["id"],
        "proposal_version": proposal["version"],
        "evidence_version": proposal["evidence_version"],
    }
    return {"status": "READY", "state": updated}
```

Input and output. A in Section 4.5 recommends digital for itself and takes paper under the total-91 proposal. Given an authorized proposal, the function records paper and the proposal/evidence versions in `joint_assignment`, preserving digital and the experience references in `own_judgment`. The same structure applies when B takes paper under the two-role total-45 proposal. If execution is deferred, the function returns DEFERRED without constructing an updated state.

Principle 5. Separate and Connect Proposal Assessment, Actual Votes, Agreement, and Execution Authorization

Requirement. Proposal quality does not justify rewriting actual votes, and receipt of all votes does not by itself establish that task requirements are satisfied. Execution authorization combines proposal assessment and agreement for the same target.

Implementation. Separate three operations. First, assess equipment constraints and the adoption criterion within the fixed candidate set. Second, derive each role's state from received votes and apply the specified agreement rule. Third, use both results to authorize execution. Direct acceptance and acceptance following a counterproposal should reach the same execution check. Bind decisions to the proposal ID, proposal version, and experience-evidence version; another role's approval cannot fill an absent vote.

Verification. Test progress under unanimous approval of the higher-total proposal, and stopping under a missing vote or explicit hold while retaining their distinct individual states. Under unanimous approval of the lower-total proposal, retain the approval facts and defer execution only. Record separately whether each commitment path invokes its check and whether that check is correct.

Code example 3. Preserve actual votes and connect proposal assessment and unanimous approval to the execution decision. Design example.

`assess_proposal` compares candidates using predictions calculated from the supplied experience. `project_votes` records actual votes for the target proposal by role. Both helpers appear in full in Appendix D. The role keys in `scores` specify everyone whose approval is required; this illustrative rule also requires an explicit

vote from the proposer. It is not a verbatim reproduction of the original two-role procedure, which constructed agreement from a proposal and the other role's response.

```
from copy import deepcopy

def decide(proposal, receipts, scores, candidates, capacity, evidence_version):
    assessment = assess_proposal(proposal, scores, candidates, capacity, evidence_version)
    approvals = project_votes(proposal, receipts, tuple(scores))
    authorized = assessment["eligible"] and approvals["agreement"] == "AGREED"
    return {"proposal": deepcopy(proposal), "assessment": assessment,
            "approvals": approvals,
            "execution": "AUTHORIZED" if authorized else "DEFERRED"}
```

Input and output. If only B/C ACCEPT votes are supplied for the three-role total-91 proposal, `states` contains A=MISSING and B/C=ACCEPT, `agreement` is PENDING, and `execution` is DEFERRED. Adding A's HOLD sets A=HOLD while leaving the group-level deferral unchanged. Only adding A's ACCEPT yields both unanimous approval and execution authorization.

If all roles ACCEPT the two-role total-37 proposal, `agreement` remains AGREED, the assessment reports LOWER_TOTAL, and execution is DEFERRED. The code neither rewrites the total-37 proposal as the total-45 proposal nor replaces votes with HOLD. Adopting a new proposal requires new confirmation of that proposal.

The appendix fixes the candidate set and resource allocation. Predictions are integers or exact fractions, rather than totals reported by the LLM. Duplicate votes or votes for a mismatched target are retained in their original form pending resolution. The effectiveness of this complete reference architecture, which separates proposal quality from expressed individual positions, remains a subject for a future connected comparison; it is not retroactively assigned to the original experiments.

Principle 6. Carry Unresolved Outcomes and Deferral Forward with Reasons and Scope

Requirement. Unidentified information and unapproved proposals are not used as resolved records. Other established information and valid work remain available.

Implementation. Separate the processing target, applicability conditions, result state, and unresolved reason. Distinguish insufficient information, an unscorable response format, a substantive requirement failure, appropriate deferral, and a stage that is inapplicable by protocol. For individual votes, distinguish nonreceipt, explicit hold, and approval in fields separate from group-level execution deferral. Do not overwrite these reasons with one aggregate state merely because the action is the same, as in Section 4.5. Preserve raw responses separately from artifacts produced by syntax normalization or storage processing, recording the changes. If retries are allowed, fix their permitted changes and limits beforehand.

Verification. Pair inputs that can be resolved with inputs lacking the information needed for resolution. In one diagnostic using eight record-management scenarios, the researcher changed saved origin IDs to correct, old, or absent IDs and supplied each role with a notice from a source designated as trusted. The notice revoked an observation—that is, invalidated a previously recorded observation. Notices either contained only an ID or additionally specified observer, revision, and paper/digital values. The LLM was instructed to invalidate an observation when its ID matched, or, if the ID did not match, when all details uniquely matched one observation. If the target could not be identified, it should retain every row and mark the matter unresolved. When identified, all copies of the target observation were marked invalid while retaining their values, revisions, and IDs as history.

With an incorrect stored ID and an ID-only notice, the LLM did not guess a row to invalidate. It preserved customer information and observations and carried joint-decision deferral into the next response. For both the old-ID and absent-ID conditions, target-resolution completion was 0/8 and evidence-appropriate handling was 8/8. Where details uniquely identified the target, invalidation was completed. Inability to identify a target from

the ID alone follows from the materials and rules; the observed behavior was whether the LLM avoided guessing and preserved unresolved state and unaffected information. Extensions with ambiguous notices or conflicting details should measure both unintended changes and excessive deferral.

Principle 7. Retain Component-Level Evaluation and Evaluate the Validator

Requirement. Identical aggregate outcomes do not erase which fields or stages succeeded and what remained defective. Checks that obstruct correct inputs are also observable.

Implementation. Record own state, shared state, provenance, updates, proposal judgments, per-role approval states, agreement, and subsequent use as components alongside end-to-end outcomes. In fixed-input repetitions, retain each component and action. Assign checks determined by explicit rules—such as structure, arithmetic, and reference existence—to code. When an LLM assists with semantic assessment, calibrate its evidence format and unresolved outputs as well as its decisions.

Verification. A judge-model diagnostic supplied 32 existing responses and 32 constructed controls containing known errors and valid boundary cases. The judge assessed output structure, attribution/references, own judgment, and joint action against explicit rules. All four assessments agreed with the reference assessment in 53 cases; four disagreed; and seven were unscorable because the judge's own supporting explanation violated its required format.

In all four disagreements, a response contained an unauthorized extra field, but the judge reported that no extra field existed. Fixed structural-validation code detected all four. Three were constructed controls with altered field names or placement, not evidence that the model spontaneously concealed a violation. The comparison shows that an LLM capable of assisting with content assessment has not thereby completed structural validation. Record false alarms, missed defects, unresolved judgments, and blocked legitimate updates separately.

Evaluate explicit comparison fields separately from performance. In three-role voting, a condition requiring candidate IDs and a higher/lower comparison separately from the own-judgment fields achieved 54/54 correct votes, compared with 52/54 under the earlier format, across 54 matched pairs from three previously used materials. With the same instructions applied to three numerical materials not previously used in this study, both conditions achieved 54/54. In each trial, all 54 comparison-field responses correctly aligned the candidates, their ordering, and the vote. The ability to inspect this correspondence and an increase in correct answers are separate outcomes. Because the comparison changed instructions and output schema together, it does not isolate the effect of a single field.

6. A Reference Architecture Connecting the Principles

6.1 Manage Record Updates and Joint Execution at Separate Boundaries

Figure 2 connects the seven principles. Before updating a record, check its target, provenance, and revision; before joint execution, check proposal assessment and agreement grounded in actual votes.

Separate information rows pointing to original sources; judgments associated with roles and purposes; proposals with targets and versions; actual votes bound to senders; agreement derived from those votes; and execution authorization with unresolved reasons. Select the fields needed by each role and stage instead of automatically transferring the entire private note into joint inputs.

Pre-storage checks use candidates actually present in the input and prespecified rules. Proposal assessment uses experience, resources, and candidates available during operation, rather than an experimental answer key. The LLM continues to generate own judgments and votes, while deterministic copying and aggregation preserve the facts already generated.

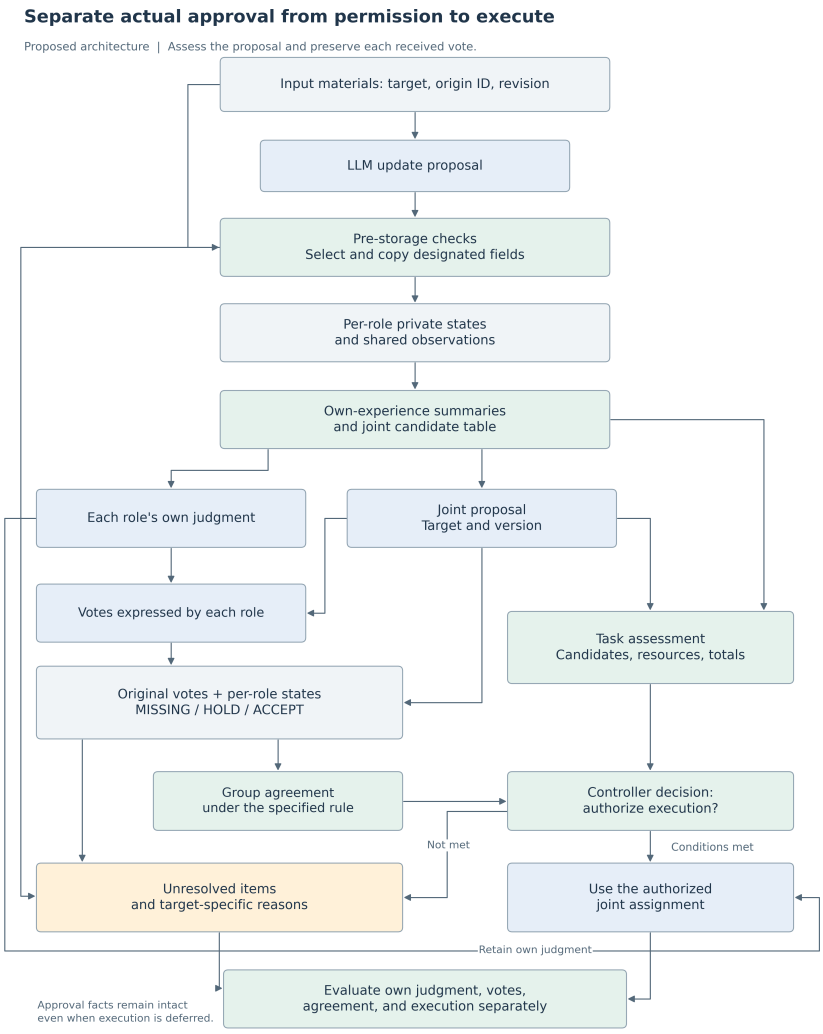


Figure 2. Proposed reference architecture connecting role-specific state and joint use. The path assessing a proposal and the path recording votes and deriving agreement meet at the controller's execution decision. Even unanimous approval leaves execution deferred if the proposal fails task requirements, while the actual approval remains unchanged. Attach the Figure 1 audit records to each stage.

6.2 Four Design-Review Cases

The following are proposed checks derived from existing cases, not additional results obtained by running the reference architecture.

An old notice arrives. If the current state is dog/revision 77 and a notice says cat/revision 76, a task requiring current information for the same target should retain revision 77 and record the older notice as not adopted. Changes should not spill into another role's observation or assigned information. An additional test should verify that applying this check to the customer field also permits a legitimate change at revision 78.

Own recommendations and joint assignments differ. Even when both A and B recommend digital for themselves, the joint allocation uses one unit of each resource. Under the total-45 proposal, B's own recommendation remains digital when B takes paper. If all roles approve the total-37 proposal, preserve that fact while deferring execution under the task's adoption criterion. Optimality here is specific to the agreed objective of maximizing the total in this task.

One role's vote is missing. Require approval from A/B/C. With only B/C ACCEPT votes, retain A=MISSING and leave the group without agreement. A's HOLD changes the individual state to an explicit hold; A's ACCEPT changes it to unanimous approval. Even when the stop decision is unchanged, retain whether a vote was not

received or a hold was expressed so that downstream stages can distinguish them.

A revocation notice has an unidentified target. If the target cannot be uniquely matched, record the unresolved notice instead of guessing a row to invalidate. Defer the joint decision that the notice may affect, while retaining well-supported private work. Explicitly delimiting the hold avoids both pretending the issue is resolved and indiscriminately stopping every task.

6.3 State What Each Component Has Established

Table 5. Evidence supporting the principles and extensions to test next. "Comparison" refers to a comparison of complete methods on the fixed materials of the source trial.

Principle	Existing evidence and established scope	Targets when extending the design
1. Preserve correspondences	Comparison of selected-row copying and subsequent use; observation of provenance defects and their transfer	Fields outside copying, wrong source selection, rewritten provenance
2. Update by target and purpose	Per-observer latest-revision selection in own/shared observations; customer-field rollback	Customer fields, same-revision conflicts, notice authority, validity intervals
3. Specify control coverage	Four branches of the same proposal and joint readout from their saved artifacts (Section 4.3)	Continued runs without the control, different tasks, longer histories
4. Separate own and joint outcomes	Separate evaluation of own recommendation and joint assignment; support-timing comparison and repetitions (Sections 4.4 and 4.6)	Freely described experiences and contributions of support at each stage
5. Separate assessment, votes, agreement, and execution	Two-role incorrect acceptance and three-role incorrect hold, followed by faithful use of actual records	Connecting separate common checks, valid progress, agreement updates
6. Preserve deferral as state	Appropriate revocation deferral; confusion between missing votes and explicit holds in three roles	State-preservation procedures, delayed/conflicting votes, subsequent use of reasons
7. Evaluate components and validators	Field-, stage-, and repetition-level diagnostics; judge calibration (Section 4 and Principle 7)	Unknown semantic boundaries, missed defects, false alarms, operating cost

7. Evaluation: Allow Appropriate Progress and Stop Inappropriate Progress

7.1 Do Not Collapse Outcomes into One Success Rate Too Early

Evaluation pairs successful progression from valid inputs with the handling of inappropriate or insufficient inputs. Unchanged own information alone does not establish that legitimate updates remain possible. Zero incorrect agreements likewise does not establish the ability to cooperate if every proposal was deferred.

Table 6. Paired tests for implementing the principles. Untested entries are examples for future designs.

Target	Inputs expected to succeed	Inputs probing the boundary	Outcomes recorded separately
Attribution and provenance	Exchange rows with matching targets and origin IDs	Correct value with wrong ID; valid ID belonging to another person	Content, attribution, provenance, propagation
Updates	Legitimate new revision for the same target	Old notice, same-revision conflict, notice for another target	Adoption, retention, unintended changes, deferral
Separation by purpose	Appropriate difference between own and joint conclusions	Change only own experience or only joint conditions	Selective effects on each use

Continued on the next page

Target	Inputs expected to succeed	Inputs probing the boundary	Outcomes recorded separately
Proposal assessment and agreement	Unanimous approval of an appropriate candidate	Unanimous approval of a suboptimal proposal; resource violations	Proposal assessment, actual agreement, execution authorization
Each role's approval state	Actual votes from all required roles for the same target	One missing vote, explicit hold, different-version or duplicate votes	Per-role states, fidelity to original votes, agreement and execution
Subsequent use	Valid agreement and appropriate saved artifacts	Invalidated evidence, no agreement, old artifacts	Fidelity to records; conformity with current requirements
Validation	Valid states and legitimate changes	Known defects and semantically ambiguous cases	Valid passage, false alarms, missed defects, unresolved outcomes

Pass actual saved artifacts onward on normal paths as well. When an earlier error is corrected at the endpoint, retain both events. Separate stages blocked by structural defects, stages inapplicable by protocol, and substantive failures. Analyses retain planned denominators and the original trials' missingness rules.

7.2 Align the Units Being Compared

Branching the same model proposal is useful for measuring a local storage operation. Evaluating the effect on continued operation requires carrying forward the artifacts actually produced under each condition. Counts of defects in proposals during a controlled run are not results from a reproduced uncontrolled run.

Repetition also has two forms. Fixed-input repetition measures response variability for that input. Repeating a connected path from formation through use measures the entire path, including upstream variability. The fixed-input repetitions in Sections 4.4 and 4.6 are of the former kind; the two forms are not interchangeable reliability measures.

Comparisons adding a check should record blocked legitimate progress, defects that pass, downstream effects, and call counts and elapsed time. Specify adoption criteria for the intended task before observing outcomes. Repairing a particular failure case and satisfying the same requirements on unused materials are separate achievements.

8. Scope and Further Development of the Design Principles

This report is an exploratory technical synthesis of four reports. Models, materials, instructions, and scoring scopes differ, so each comparison retains its denominators and evaluation targets. For example, a later response required to cite provenance and one asked only to select a customer design are evaluated on different scales. Roles, branches, and technical repetitions sharing materials are not counted as independent materials.

The observations concern explicitly assigned facts, recommendations from supplied experience, structured shared observations, and short agreement processes. They do not measure the causal effect of increasing the number of roles under matched conditions or establish performance for arbitrary group sizes. Four or more roles, delayed or withdrawn votes, votes for different versions, and judgments formed through free dialogue are candidates for extending the scope.

Per-observer latest-revision selection requires identifiable targets, candidates present in the input, and comparable revisions. Vote aggregation likewise requires an identifiable sender and target proposal. Real-world truth, authority to change information, fairness, and the appropriate agreement rule must be specified for the use case; maximum revision, maximum total, and unanimous approval do not determine them by themselves.

The four source reports' design, execution, fixed-code scoring, and verification were conducted within the same project. This report reorganizes that evidence and adds no new independent replication. The code examples and reference architecture connect previously compared components with design proposals derived from observations. Future connected evaluations should preserve the original results while jointly measuring valid progress, prevention of inappropriate execution, accuracy of role-specific state, and the cost of confirmation.

9. Conclusion

The two- and three-role experiments show the need to treat the connection between role-specific information and joint work as multiple kinds of correctness. Content can be correct while provenance is damaged; accurate readout can faithfully use stale saved information; and faithful use of an agreement can carry forward an inappropriate joint decision. In three roles, a response also correctly deferred group action while reinterpreting an absent vote as an individual's explicit hold.

The central design requirement is to **record separately whose information is protected, what evidence justifies changing it, and which conditions permit joint action**. Bind content to provenance, update according to target and purpose, and separate own judgments from joint assignments. Preserve proposal assessments and actual votes, then connect agreement based on those votes to the controller's execution authorization. Reasons for deferral and component-level evaluation make both local improvements and remaining defects visible.

Supporting role-specific and joint activity requires more than freezing each role's information or making every answer match. The seven principles and reference architecture provide a basis for implementing and testing systems that preserve each role's legitimate information and expressed position while allowing joint work to proceed.

10. Author Contributions and Data Availability

The author determined the research direction and execution scope and reviewed the results and manuscripts of the four source reports. This English edition accompanies the four experimental reports. LLM assistance was used for organizing materials, drafting, translation, and checking. Preparing this synthesis involved no new model experiments, repair of original responses, or rescoring of research outcomes. All customers, venues, and experiences are fictional.

The four reports, experimental materials, and evidence index are retained without a deletion deadline and distributed in the MRSI repository. Tasks, manipulations, and results needed to understand this report are included here. Appendix C and the references identify the evidence sources; S1 links to original results and saved-artifact checks. The repository specifies Apache 2.0 licensing, distribution scope, and current reproduction instructions. A permanent archive DOI for this report has not been assigned. Formal funding and conflict-of-interest declarations remain subject to the author's confirmation.

References

1. Kentaro Sato. 2026. Maintaining Role-Specific Information and Shared Records in LLM Memory Updates: An Exploratory Evaluation of Content, Provenance, and Continued Use. Japanese v0.11, September 23, 2026. Source report for the provenance, copying, and revocation cases. English version.

2. Kentaro Sato. 2026. Scope of Update Control in Private and Shared LLM Records: Field-Level Comparisons and Post-Update Rollback. Japanese v0.4, September 27, 2026. Source report for the field-level comparison and customer-update cases. English version.

3. Kentaro Sato. 2026. Separating the Stability of Role-Specific Judgments from the Quality of Joint Agreements: Timing of Computational Support and Fixed-Input Repetitions in a Two-Role LLM Task. Japanese v0.2, September 29, 2026. Source report for joint-judgment and repetition cases. English version.

4. Kentaro Sato. 2026. *LLM System Failure Studies*. Technical Report v1.8, in Meteon Collected Edition v1.1 (source not bundled). Consulted for organizing requirements, failure paths, and classifications.

5. Kentaro Sato. 2026. *Autonomous LLM System Design: Build the Cage Before Granting Authority*. Technical Report v1.16, September 5, 2026, in the same collected edition (source not bundled). Consulted for the distinction between evidence and proposals in Section 3, design principles in Part III, and the mapping in Appendix A.

6. Kentaro Sato. 2026. Boundaries Between Role-Specific Judgment, Individual Approval, and Group Agreement in Three LLM Roles: Carrying Actual Votes Forward and Comparing Matched Approval States. Japanese v0.2, October 13, 2026. Source report for three-role voting, state readout, and explicit comparison fields. English version.

This report organizes failure and design issues from the author's experimental records. It is neither an exhaustive literature review nor a claim that each of the seven principles is introduced here for the first time. Appendix C explains the organizational approaches drawn from References 4 and 5 within this report.

Appendix A. From Failure Observations to Design and Verification

Table A1. Connecting the same cases to controls and checks. The rows are not mutually exclusive failure classes.

Observation or evaluation issue	Main principles	What to inspect next
Correct values carry old IDs or IDs absent from the input	1, 3	Within-row correspondence, fields outside copying, later rewriting
An old notice reverts a new record	2, 3	Target-specific adoption rules and passage of legitimate updates
Correct joint answers coexist with stale own fields	3, 7	Field-level state and downstream stages using that field
Repairing own fields leaves aggregate performance unchanged	3, 7	Overlapping defects and successfully repaired components
Correct own judgments coexist with acceptance of a suboptimal proposal	4, 5	Own recommendation, joint adoption criterion, and agreement commitment
Incorrect acceptance proceeds without rechecking	5, 7	Check-trigger conditions and records of passage on every commitment path
Subsequent responses faithfully retain a hold on an appropriate proposal	4, 5	Vote appropriateness and fidelity to actual votes
Correct group-level judgment coexists with describing a missing vote as an explicit hold	1, 5, 6, 7	Correspondence among original votes, per-role states, agreement, and execution
Deferral, format defects, and inapplicability are collapsed into one outcome	6, 7	State reasons, dependencies, and the scope of valid progress
The defective field changes despite identical overall failure	7	Repetitions, components, and actions for each fixed input

Appendix B. A One-Page Record for Design Review

When transferring a principle to another system, complete the following for each procedure. This form is proposed to make evidence and validation scope explicit; its own effectiveness has not been evaluated.

Field	What to record
Object to protect	Role, information field, purpose, stage
Success conditions	What counts as correct retention, change, agreement, and use
Input assumptions	Target identity, provenance, revision, candidates, authority, objective, constraints, required approvers, agreement rule
Processing responsibility	Model judgment, deterministic commitment, human decision
Applicable paths	Normal update, direct acceptance, counterproposal, retry, and other routes
When not applicable	Scope of deferral, reason, retained information, next required confirmation
Observation record	Proposal, actual saved artifact, next input, use outcome, reasons for changes and inapplicability
Verification	Valid progress, inappropriate progress, excessive stopping, repetition variability
Evidence scope	Source versions, observations, compared procedures, untested extensions

Evaluate the scope achieved along paths connecting actual saved artifacts, rather than treating a collection of completed forms as a guarantee that private and shared activity can coexist.

Appendix C. Experimental Records and Organizational References

This appendix is an index for checking original records, not a prerequisite for understanding the report. Descriptive trial names in the main text map to identifiers below. Supplement S1 links to original responses, inputs,

and saved artifacts.

Location in this report	Original trial	Materials and generation units	Source report
Section 4.1: transfer-method comparison	E1; worked example G49/B	64 groups assigned to both methods; shared rows per method: 64 groups × two roles × two rows	Reference 1, Sections 3.4, 4.2, 5.2
Section 4.2: preference change and rollback	CU01; worked example C06-R1/A	Eight new cases × two repetitions; one role targeted per trajectory	Reference 2, Sections 3.4, 5.4
Section 4.3: field-level comparison	OR01	36 proposals from 18 previously used materials × two repetitions, branched four ways; 55 readout generations	Reference 2, Sections 4.2, 5.2
Section 4.4: incorrect acceptance and subsequent use	ST02, EARLY, N05, LOWER	One path among eight new materials × three initial proposals	Reference 3, Sections 3.2, 5.2
Sections 4.4 and 4.6: fixed-input confirmation repetitions	JR01	32 formation inputs and 48 confirmation inputs, each repeated three times; two unconnected panels	Reference 3, Sections 4.1, 5.3
Section 4.6: timing of summary supply	ST01, ST02	Each trial: eight materials × three initial proposals × two supply conditions	Reference 3, Sections 4.1–4.3, 5.1
Principle 6: revocation diagnostic	E2	Eight scenarios × three provenance conditions × two notice conditions; both roles and stages evaluated at scenario level	Reference 1, Sections 4.3, 5.3
Principle 7: judge calibration	BJ01	32 actual responses plus 32 constructed controls	Reference 3, Appendix A, and original results
Section 4.4: three-role incorrect hold and actual-vote use	TB01, T02/C, higher-total proposal	Three materials × two repetitions; 36 votes and 72 readouts	Reference 6, Sections 4.1 and 5.1
Section 4.5: matched approval-state comparison	TAR01, including N01/A	Three materials × three states × three reading roles × two repetitions; 27 inputs and 18 matched triplets	Reference 6, Sections 4.3 and 5.2
Principle 7: explicit comparison fields	TCV01, TCV02	Each trial: three materials; two conditions with 54 votes each; 54 matched pairs	Reference 6, Sections 4.2 and 5.3

Reference 1 records additional provenance defects. In a continued note-update trial, B saved A's new values and revision 3 while retaining the old origin ID `SM3A`. The next call correctly answered dog/digital and also retained that old ID (SM02, G03/B). In a separate comparison of transfer methods using the same pre-merge materials, B under rewriting changed the correct origin ID `SM2G3A` to `SM3G3A`, an ID absent from the input; the next response retained it (RC01, G03/B). Old-ID retention, rewriting to a nonexistent ID, and the one-character omission in Section 4.1 are distinct forms of broken content–provenance correspondence.

The organizational approach drawn from Meteon's failure studies [4] distinguishes what a procedure was required to do, what violated that requirement, and where the defect was passed. The design report [5] provided a structure that goes beyond turning failures into prohibitions: it specifies required inputs, processing responsibilities, conditions for valid progress, and verification. These ideas are developed in Sections 3 and 5–7, without requiring knowledge of Meteon terminology or principle numbers. Meteon cases are not added to the experimental counts, and this report is not presented as an independent replication of that system.

Appendix D. Complete Code, Inputs, and Execution Instructions

The single block below combines Code examples 1–3 with proposal assessment, actual-vote projection, synthetic inputs, and 18 tests. It uses only standard Python and needs no other research material or model.

D.1 Inputs and Execution

Save D.2 as `multi_role_design_examples.py` and run the following command. The entire block can also be pasted into one notebook cell.

```
python3 multi_role_design_examples.py
```

Table D1. Input contract for the illustrative code. This is not a verbatim reproduction of the original experimental JSON.

Input	Contents and assumptions
`rows`	Rows of the same observation type for the same task: observer, integer revision, observed values, and origin ID. Structure and source identity are checked in advance
`state`	Own role, recommendation, and experience references. Joint assignment is written separately
`proposal`	ID, version, proposer, experience-evidence version, and role-specific allocation. The caller verifies that identical IDs/versions refer to identical contents
`receipts`	Sender, target proposal ID/version, experience-evidence version, and ACCEPT/HOLD. Sender identity is checked against the source call rather than relying only on the model's self-report
`scores`	Per-role, per-medium predictions recalculated from the evidence by code, as integers or `Fraction` values. Role keys identify all required approvers
`candidates`	Fixed list of feasible allocations to compare. The maximum is taken within this list
`capacity`	Equipment counts by medium, all of which are used in this task: one of each for two roles; two paper and one digital for the three-role example
`evidence_version`	Version of the experience evidence used for the current execution

This example requires explicit approval from everyone, including the proposer. Proposing is not treated as supplying a vote. Agreement aggregates actual votes for the proposal; execution authorization combines that agreement with assessment under the current task requirements.

The code handles a snapshot containing one vote per person. Duplicate votes, even identical ones, remain unresolved. Votes for another target, unknown senders, and unsupported actions also stop execution while preserving the original receipts. This is an entry point for later resolution, not an implementation of delayed-vote, withdrawal, or revoting rules. Matching versions to contents, authentication, persistence, and connecting every relevant path remain responsibilities of the caller.

D.2 Complete Executable Code

```
"""Multi-role design examples: standard library only; no LLM calls or file writes.

select_observation reuses the earlier example. The other functions illustrate
proposed architecture components; they were not connected to the original
LLM experiments. The caller checks types, source identity, and the correspondence
between IDs/versions and contents before invoking these functions.
"""
import json
import unittest
from collections import Counter
from copy import deepcopy
from fractions import Fraction
```

```
from itertools import permutations


def select_observation(rows, owner):
    candidates = [r for r in rows if r["observer"] == owner]
    if not candidates:
        return {"status": "HOLD", "reason": "MISSING_OWNER"}
    by_version = {}
    for row in candidates:
        version = row["revision"]
        if version in by_version and by_version[version] != row:
            return {"status": "HOLD", "reason": "SAME_VERSION_CONFLICT"}
        by_version[version] = row
    latest = by_version[max(by_version)]
    return {"status": "READY", "row": deepcopy(latest)}


def record_assignment(state, decision):
    if decision["execution"] != "AUTHORIZED":
        return {"status": "DEFERRED", "reason": "NO_EXECUTION_PERMISSION"}
    proposal = decision["proposal"]
    updated = deepcopy(state)
    updated["joint_assignment"] = {
        "medium": proposal["allocation"][state["role"]],
        "proposal_id": proposal["id"],
        "proposal_version": proposal["version"],
        "evidence_version": proposal["evidence_version"],
    }
    return {"status": "READY", "state": updated}


def assess_proposal(proposal, scores, candidates, capacity, evidence_version):
    """Evaluate within a fixed candidate set; do not search for new candidates."""
    def feasible(allocation):
        return (set(allocation) == set(scores)
                and Counter(allocation.values()) == Counter(capacity))

    def total(allocation):
        return sum(scores[r][allocation[r]] for r in scores)

    if proposal["evidence_version"] != evidence_version:
        return {"eligible": False, "reason": "STALE_EVIDENCE"}
    if not scores or not candidates or not all(feasible(a) for a in candidates):
        return {"eligible": False, "reason": "INVALID_CANDIDATE_CATALOG"}
    allocation = proposal["allocation"]
    if not feasible(allocation):
        return {"eligible": False, "reason": "RESOURCE_CONFLICT"}
    if allocation not in candidates:
        return {"eligible": False, "reason": "OUTSIDE_CANDIDATES"}
    value, best = total(allocation), max(map(total, candidates))
    return {"eligible": value == best, "total": value, "best": best,
            "reason": "MEETS_CRITERION" if value == best else "LOWER_TOTAL"}


def project_votes(proposal, receipts, participants):
    """Determine unanimous approval from actual votes for one proposal; absence is MISSING."""
    if not participants or len(set(participants)) != len(participants):
        raise ValueError("Participants must be nonempty and unique")
    key = (proposal["id"], proposal["version"], proposal["evidence_version"])
    states = {r: "MISSING" for r in participants}
    issues, seen = [], set()
    for vote in receipts:
        actor = vote["author"]
        target = (vote["proposal_id"], vote["proposal_version"], vote["evidence_version"])
        if actor not in states or target != key:
```

```
            issues.append("UNBOUND_RECEIPT")
            continue
        if actor in seen:
            states[actor] = "UNRESOLVED"
            issues.append("DUPLICATE_RECEIPT")
            continue
        seen.add(actor)
        if vote["action"] not in ("ACCEPT", "HOLD"):
            states[actor] = "UNRESOLVED"
            issues.append("UNSUPPORTED_ACTION")
        else:
            states[actor] = vote["action"]
    agreed = not issues and all(v == "ACCEPT" for v in states.values())
    return {"states": states, "agreement": "AGREED" if agreed else "PENDING",
            "issues": issues, "receipts": deepcopy(receipts)}


def decide(proposal, receipts, scores, candidates, capacity, evidence_version):
    assessment = assess_proposal(proposal, scores, candidates, capacity, evidence_version)
    approvals = project_votes(proposal, receipts, tuple(scores))
    authorized = assessment["eligible"] and approvals["agreement"] == "AGREED"
    return {"proposal": deepcopy(proposal), "assessment": assessment,
            "approvals": approvals,
            "execution": "AUTHORIZED" if authorized else "DEFERRED"}


def fixture(size=3, lower=False):
    """Synthetic inputs using values from existing cases, not the original experimental JSON."""
    if size == 2:
        scores = {"A": {"PAPER": 17, "DIGITAL": 27},
                  "B": {"PAPER": 18, "DIGITAL": 20}}
        candidates = [{"A": "DIGITAL", "B": "PAPER"},
                      {"A": "PAPER", "B": "DIGITAL"}]
        capacity = {"PAPER": 1, "DIGITAL": 1}
    elif size == 3:
        scores = {"A": {"PAPER": 24, "DIGITAL": 29},
                  "B": {"PAPER": 24, "DIGITAL": 35},
                  "C": {"PAPER": 32, "DIGITAL": 16}}
        candidates = [{"A": "PAPER", "B": "DIGITAL", "C": "PAPER"},
                      {"A": "DIGITAL", "B": "PAPER", "C": "PAPER"}]
        capacity = {"PAPER": 2, "DIGITAL": 1}
    else:
        raise ValueError("This illustration has fixtures for two or three roles")
    proposal = {"id": "p-low" if lower else "p-high", "version": 1,
                "evidence_version": "exp-7", "author": "A",
                "allocation": deepcopy(candidates[int(lower)])}
    receipts = [{"author": r, "action": "ACCEPT", "proposal_id": proposal["id"],
                 "proposal_version": 1, "evidence_version": "exp-7"} for r in scores]
    return proposal, receipts, scores, candidates, capacity, "exp-7"


def observation(owner, revision, origin):
    return {"observer": owner, "revision": revision, "origin_id": origin,
            "paper": "ALL", "digital": "NOT_ALL"}


class DesignTests(unittest.TestCase):
    def test_owner_then_version_and_copy(self):
        rows = [observation("A", 2, "a2"), observation("B", 99, "b99"),
                observation("A", 3, "a3")]
        for ordered in permutations(rows):
            result = select_observation(ordered, "A")
            self.assertEqual(result["row"], rows[2])
            result["row"]["origin_id"] = "changed"
            self.assertEqual(rows[2]["origin_id"], "a3")
```

```
    def test_observation_missing_conflict_and_newer(self):
        self.assertEqual(select_observation([], "A")["reason"], "MISSING_OWNER")
        rows = [observation("A", 3, "a3"), observation("A", 3, "other")]
        self.assertEqual(select_observation(rows, "A")["reason"], "SAME_VERSION_CONFLICT")
        rows = [observation("A", 3, "a3"), observation("A", 4, "a4")]
        self.assertEqual(select_observation(rows, "A")["row"]["revision"], 4)

    def test_normal_two_and_three_role_progress(self):
        for n, best in [(2, 45), (3, 91)]:
            result = decide(*fixture(n))
            self.assertEqual(result["execution"], "AUTHORIZED")
            self.assertEqual(result["assessment"]["total"], best)

    def test_missing_is_not_explicit_hold(self):
        for n in (2, 3):
            args = fixture(n)
            args[1].pop(0)
            missing = decide(*args)
            args = fixture(n)
            args[1][0]["action"] = "HOLD"
            held = decide(*args)
            self.assertEqual(missing["approvals"]["states"]["A"], "MISSING")
            self.assertEqual(held["approvals"]["states"]["A"], "HOLD")
            for r in (missing, held):
                self.assertEqual(r["approvals"]["agreement"], "PENDING")
                self.assertEqual(r["execution"], "DEFERRED")

    def test_proposing_does_not_supply_a_vote(self):
        proposal, _, scores, *_ = fixture()
        r = project_votes(proposal, [], tuple(scores))
        self.assertEqual(set(r["states"].values()), {"MISSING"})
        self.assertEqual(r["agreement"], "PENDING")

    def test_lower_total_keeps_unanimous_approval(self):
        for n, lower in [(2, 37), (3, 85)]:
            args = fixture(n, lower=True)
            r = decide(*args)
            self.assertEqual(r["assessment"]["total"], lower)
            self.assertEqual(r["assessment"]["reason"], "LOWER_TOTAL")
            self.assertEqual(r["approvals"]["agreement"], "AGREED")
            self.assertEqual(r["execution"], "DEFERRED")
            self.assertEqual(r["approvals"]["receipts"], args[1])
            self.assertEqual(r["proposal"], args[0])

    def test_resource_conflict_keeps_votes(self):
        args = fixture(2)
        args[0]["allocation"]["B"] = "DIGITAL"
        r = decide(*args)
        self.assertEqual(r["assessment"]["reason"], "RESOURCE_CONFLICT")
        self.assertEqual(r["approvals"]["agreement"], "AGREED")
        self.assertEqual(r["execution"], "DEFERRED")

    def test_different_proposal_or_version_cannot_authorize(self):
        for key, value in [("proposal_id", "other"), ("proposal_version", 0),
                           ("evidence_version", "exp-old")]:
            args = fixture()
            args[1][0][key] = value
            r = decide(*args)
            self.assertEqual(r["execution"], "DEFERRED")
            self.assertEqual(r["approvals"]["states"]["A"], "MISSING")
            self.assertTrue(r["approvals"]["issues"])

    def test_stale_snapshot_keeps_actual_agreement(self):
        args = list(fixture())
```

```
        args[-1] = "exp-8"
        r = decide(*args)
        self.assertEqual(r["approvals"]["agreement"], "AGREED")
        self.assertEqual(r["assessment"]["reason"], "STALE_EVIDENCE")
        self.assertEqual(r["execution"], "DEFERRED")

    def test_duplicate_votes_not_counted_as_another_person(self):
        for action in ("ACCEPT", "HOLD"):
            args = fixture()
            duplicate = deepcopy(args[1][0])
            duplicate["action"] = action
            args[1].append(duplicate)
            r = decide(*args)
            self.assertEqual(r["approvals"]["states"]["A"], "UNRESOLVED")
            self.assertEqual(len(r["approvals"]["receipts"]), 4)
            self.assertEqual(r["execution"], "DEFERRED")

    def test_unknown_actor_and_unsupported_action(self):
        for key, value in [("author", "OUTSIDER"), ("action", "COUNTER")]:
            args = fixture()
            args[1][0][key] = value
            r = decide(*args)
            self.assertEqual(r["execution"], "DEFERRED")
            self.assertTrue(r["approvals"]["issues"])

    def test_empty_or_duplicate_participant_set(self):
        p, receipts, *_ = fixture()
        for roles in [(), ("A", "A")]:
            with self.assertRaises(ValueError):
                project_votes(p, receipts, roles)

    def test_receipt_order_does_not_change_states(self):
        args = list(fixture())
        for ordered in permutations(args[1]):
            args[1] = list(ordered)
            r = decide(*args)
            self.assertEqual(r["execution"], "AUTHORIZED")
            self.assertEqual(set(r["approvals"]["states"].values()), {"ACCEPT"})

    def test_tie_and_exact_fraction(self):
        for lower in (False, True):
            args = fixture(2, lower)
            args[2]["A"]["PAPER"] = Fraction(20, 3)
            args[2]["A"]["DIGITAL"] = Fraction(20, 3)
            args[2]["B"]["PAPER"] = Fraction(10, 3)
            args[2]["B"]["DIGITAL"] = Fraction(10, 3)
            r = decide(*args)
            self.assertEqual(r["assessment"]["total"], 10)
            self.assertEqual(r["execution"], "AUTHORIZED")

    def test_candidate_scope_and_catalog(self):
        args = fixture()
        args[0]["allocation"] = {"A": "PAPER", "B": "PAPER", "C": "DIGITAL"}
        self.assertEqual(decide(*args)["assessment"]["reason"], "OUTSIDE_CANDIDATES")
        args[3].clear()
        self.assertEqual(decide(*args)["assessment"]["reason"], "INVALID_CANDIDATE_CATALOG")

    def test_assignment_preserves_own_judgment(self):
        for n, role in [(2, "B"), (3, "A")]:
            state = {"role": role, "own_judgment": {"choice": "DIGITAL", "refs": ["e1"]}}
            before = deepcopy(state)
            out = record_assignment(state, decide(*fixture(n)))
            self.assertEqual(out["state"]["joint_assignment"]["medium"], "PAPER")
            self.assertEqual(out["state"]["own_judgment"], before["own_judgment"])
            self.assertEqual(state, before)
```

```
    def test_no_assignment_without_permission(self):
        args = fixture()
        args[1].pop()
        self.assertEqual(record_assignment({"role": "A"}, decide(*args))["status"], "DEFERRED")

    def test_no_input_mutation_or_output_alias(self):
        args = fixture()
        before = deepcopy(args)
        out = decide(*args)
        self.assertEqual(args, before)
        out["proposal"]["allocation"]["A"] = "CHANGED"
        out["approvals"]["receipts"][0]["action"] = "CHANGED"
        self.assertEqual(args, before)


def demonstrate():
    examples = {}
    for label in ("MISSING", "HOLD", "ACCEPT"):
        args = fixture()
        if label == "MISSING":
            args[1].pop(0)
        elif label == "HOLD":
            args[1][0]["action"] = "HOLD"
        result = decide(*args)
        examples[label] = {"states": result["approvals"]["states"],
                           "agreement": result["approvals"]["agreement"],
                           "execution": result["execution"]}
    examples["LOWER_WITH_ALL_ACCEPT"] = decide(*fixture(2, lower=True))
    state = {"role": "A", "own_judgment": {"choice": "DIGITAL", "refs": ["a1", "a2"]}}
    examples["OWN_AND_ASSIGNMENT"] = record_assignment(state, decide(*fixture()))
    examples["LATEST_ROW"] = select_observation(
        [observation("A", 2, "a2"), observation("B", 99, "b99"), observation("A", 3, "a3")], "A")
    print(json.dumps(examples, ensure_ascii=False, indent=2))


if __name__ == "__main__":
    demonstrate()
    suite = unittest.defaultTestLoader.loadTestsFromTestCase(DesignTests)
    result = unittest.TextTestRunner(verbosity=2).run(suite)
    if not result.wasSuccessful():
        raise SystemExit(1)
```

D.3 Interpreting the Output and Verification Scope

For the three-role higher-total proposal, varying A's vote gives the following results. B/C approvals and the own judgment are common to all conditions.

A's vote	A's state	Unanimous agreement	Execution decision
Absent from input	MISSING	PENDING	DEFERRED
Explicit HOLD	HOLD	PENDING	DEFERRED
Explicit ACCEPT	ACCEPT	AGREED	AUTHORIZED

For unanimous approval of the two-role total-37 proposal, the code returns LOWER_TOTAL and DEFERRED while preserving the original votes and AGREED. For the authorized total-91 proposal, it records PAPER as A's joint assignment while retaining DIGITAL as A's own recommendation. The observation-selection example selects A's revision 3 without admitting B's revision 99 into A's candidates.

The 18 tests cover valid two- and three-role progress, missing votes versus explicit holds, prevention of treating

a proposal as a vote, unanimous approval of suboptimal and resource-violating proposals, wrong-version and duplicate votes, candidate-set scope, ties and fractions, own-field preservation, and nonmutation of inputs. A function signature that accepts arbitrary role labels is distinct from measured model performance with more roles. These are illustrative two- and three-role inputs; test counts are not added to research response counts.